



CHULA **ENGINEERING** **COMPUTER**
Foundation toward Innovation



Practical Machine Learning

Prof. Peerapon Vateekul, Ph.D.
Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University
Peerapon.v@chula.ac.th
www.cp.eng.chula.ac.th/~peerapon/



Outlines

- Introduction
- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning



Introduction



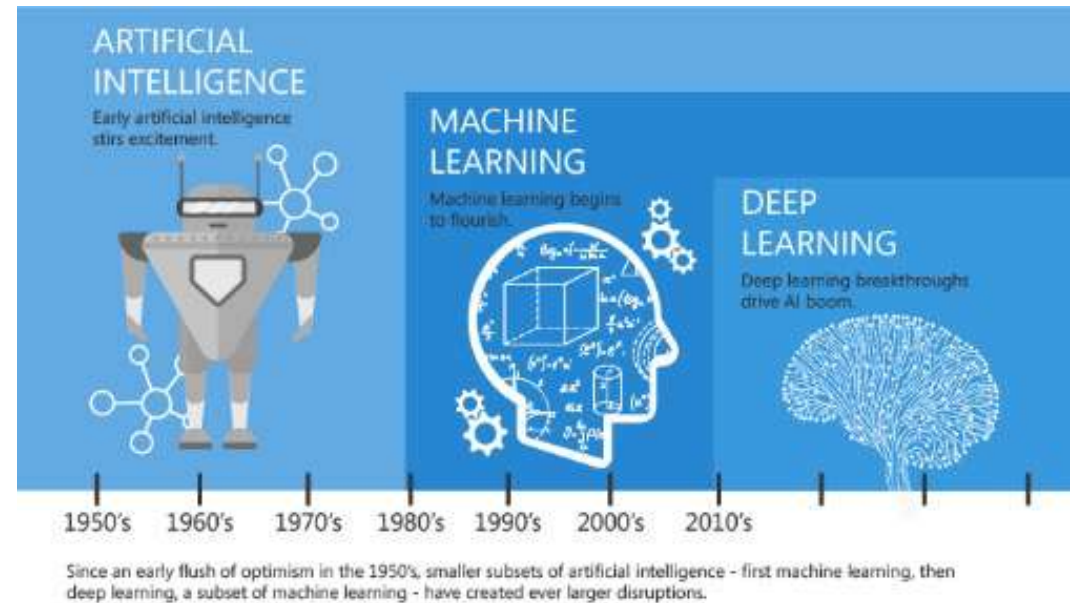
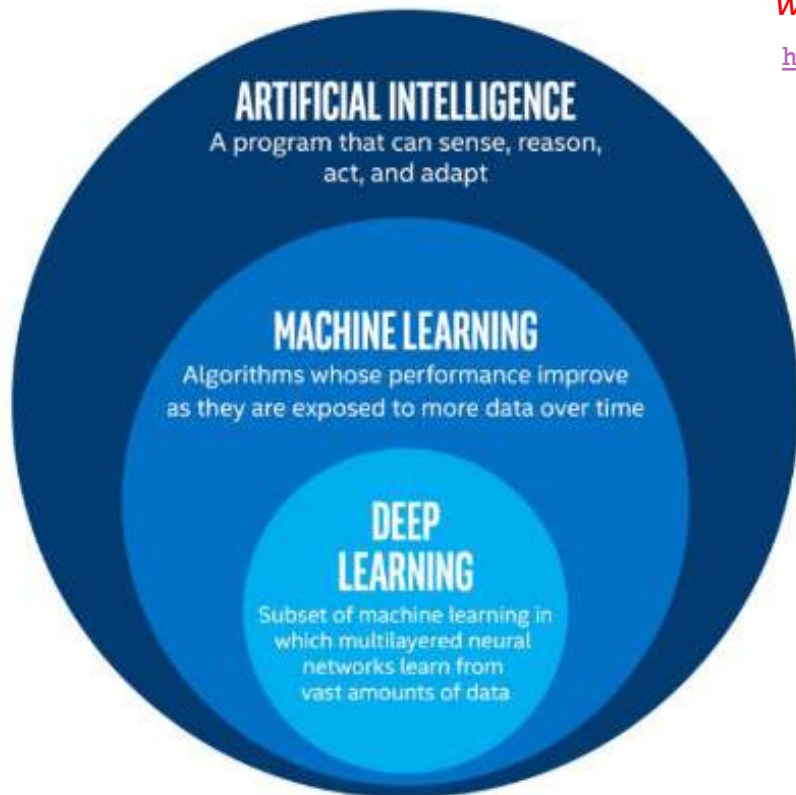
AI, Machine Learning, and Deep Learning

4

- **Machine Learning (ML)** is a subfield in AI focusing on making to learn by itself **without human intervention**.

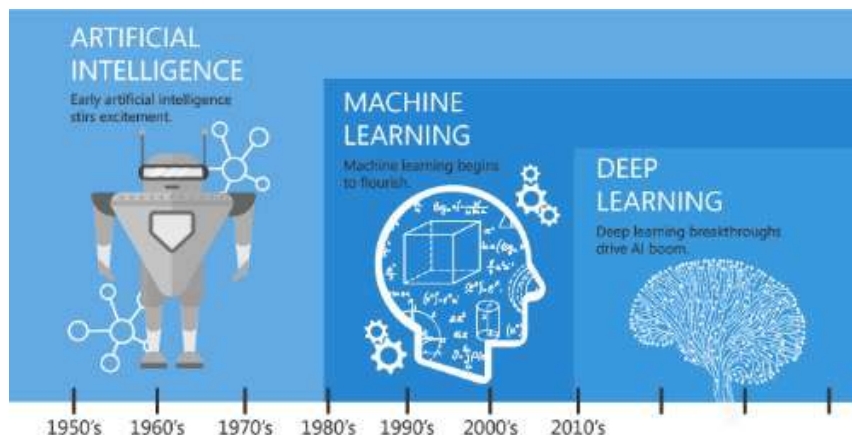
“Machine learning is the science of getting computers to act *without being explicitly programmed*.” — [Stanford University](https://towardsdatascience.com/cousins-of-artificial-intelligence-dda4edc27b55)

<https://towardsdatascience.com/cousins-of-artificial-intelligence-dda4edc27b55>

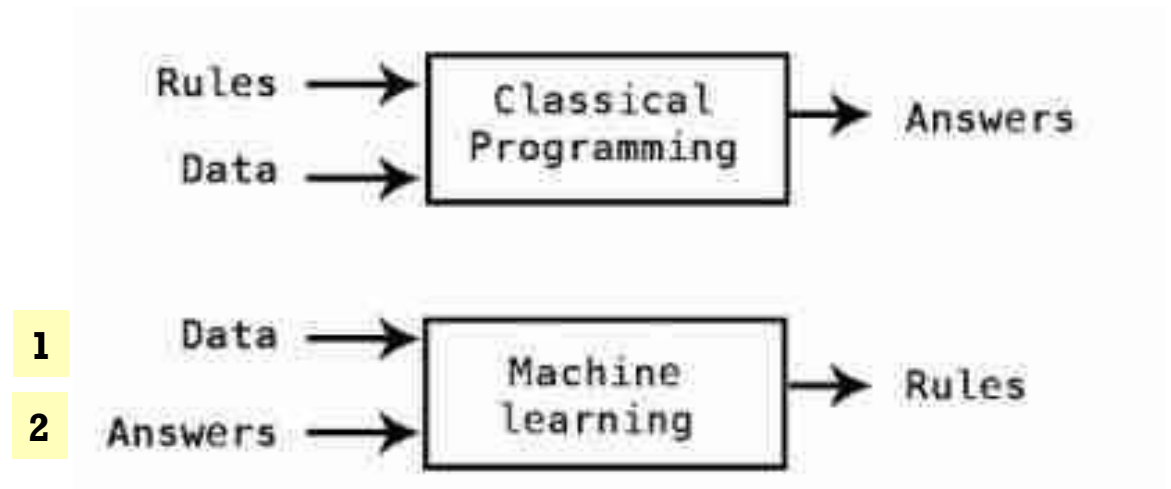


AI = Automation

- 1) Rule-based AI (Symbolic AI)
- 2) Machine Learning



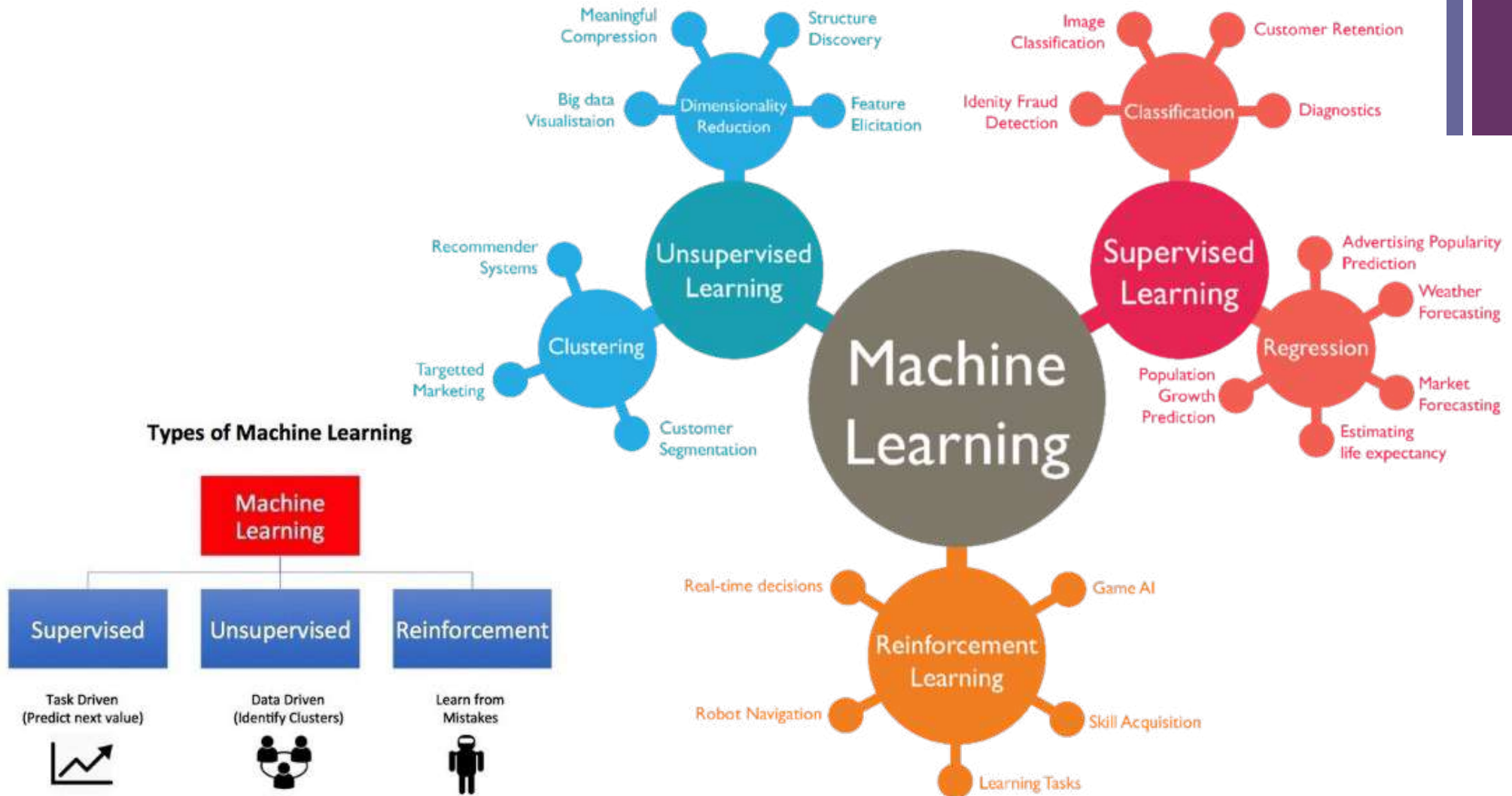
Since an early flush of optimism in the 1950's, smaller subsets of artificial intelligence - first machine learning, then deep learning, a subset of machine learning - have created ever larger disruptions.



<https://mc.ai/machine-learning-basics-artificial-intelligence-machine-learning-and-deep-learning/>

+ Machine Learning (cont.)

6





scikit-learn

Machine Learning in Python

[Getting Started](#)[What's New in 0.22.2](#)[GitHub](#)

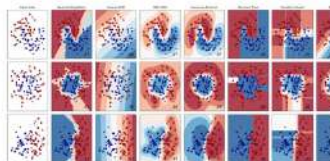
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: SVM, nearest neighbors, random forest, and more...



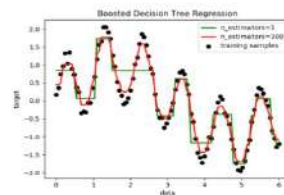
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, nearest neighbors, random forest, and more...



Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, and more...



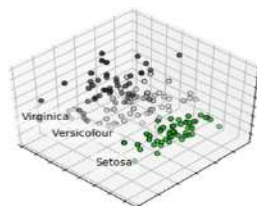
Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: k-Means, feature selection, non-negative matrix factorization, and more...



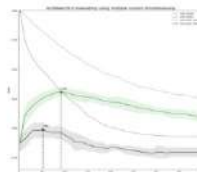
Examples

Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning

Algorithms: grid search, cross validation, metrics, and more...



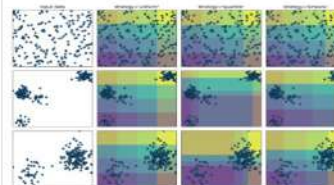
Examples

Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: preprocessing, feature extraction, and more...



Examples



Terminology: Data table

inputs				target
Age	Income	Gender	Province	Purchase
25	25,000	Female	Bangkok	Yes
35	50,000	Female	Nontaburi	Yes
32	35,000	Male	Bangkok	No

■ Row

- Example, instance, case, observation, subject

■ Column

- Feature, variable, attribute

■ Input

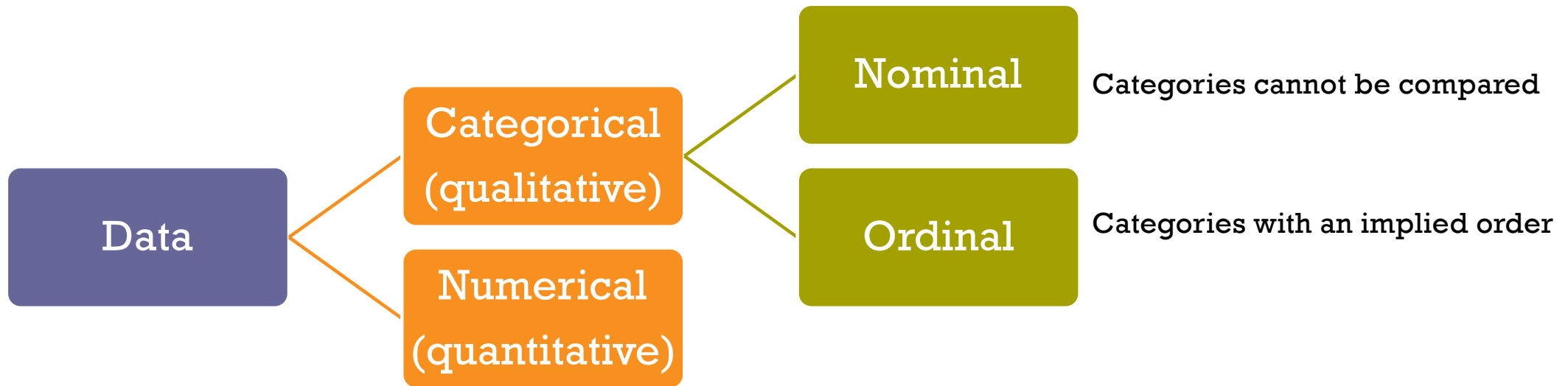
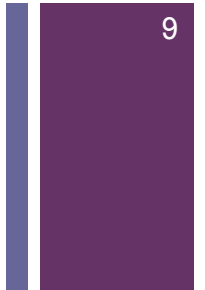
- Predictor, independent, explanatory variable

■ Target

- Output, outcome, response, dependent variable



Terminology: Kinds of data

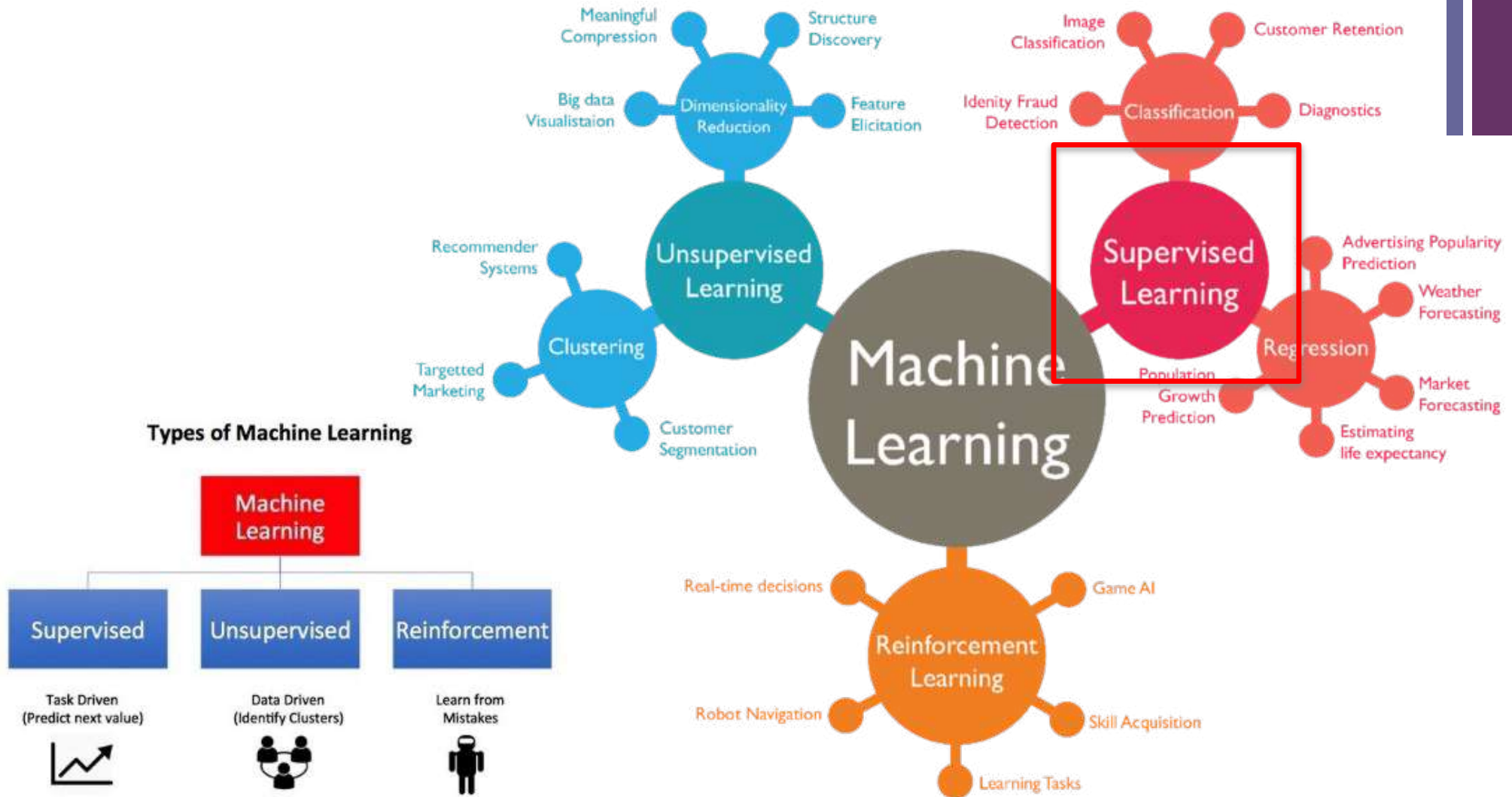




Supervised Learning (Predictive Task)

+ Machine Learning (cont.)

11





Task1: Supervised learning

Handcrafted features

12

Training Data



inputs				target
Age	Income	Gender	Province	Purchase
25	25,000	Female	Bangkok	Yes
35	50,000	Female	Nontaburi	Yes
32	35,000	Male	Bangkok	No

Testing Data

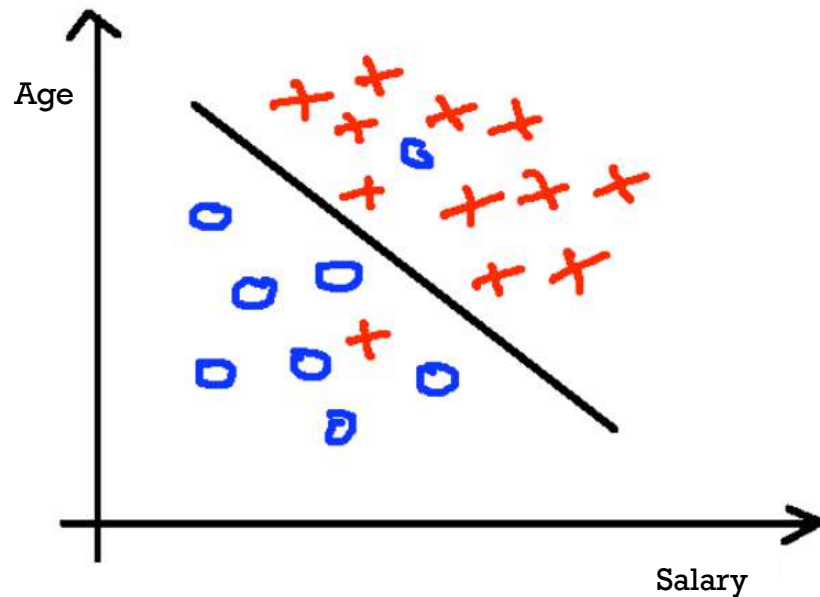


Age	Income	Gender	Province	Purchase
25	25,000	Female	Bangkok	?

Application: Direct Target Marketing

+ Classification: predicting a category

Logistic Regression



Predict targeted customers who tend to buy our product (yes/no)

■ Some techniques:

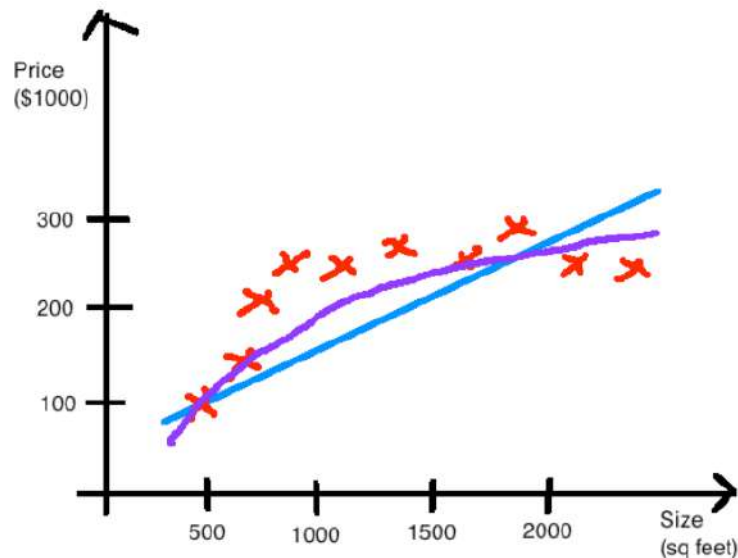
- Naïve Bayes
- Decision Tree
- Logistic Regression
- Support Vector Machines
- Neural Network
- Ensembles

■ Sample Applications

- Database marketing
- Fraud detection
- Pattern detection
- Churn customer detection

+ Regression: predict a continuous value

Linear Regression



Predict a sale price of each house

■ Some techniques:

- Linear Regression / GLM
- Decision Trees
- Support vector regression
- Neural Network
- Ensembles

■ Sample Applications

- Financial risk management
- Revenue forecasting





Scikit-learn: Machine learning library in Python



- Provides many machine learning tools with a common **Estimator interface**
- Built in helpers for common **ML tasks** (e.g., metrics, preprocessing)
- Easily combine algorithms to make **a complex pipeline**
- Relies heavily on numpy and scipy, often used with **pandas**

How do you pronounce the project name?

sy-kit learn. sci stands for science!

Why scikit?

There are multiple scikits, which are scientific toolboxes built around SciPy. You can find a list at <https://scikits.appspot.com/scikits>. Apart from scikit-learn, another popular one is scikit-image.

Classification

Identifying to which category an object belongs to.

Applications: Spam detection, Image recognition.

Algorithms: SVM, nearest neighbors, random forest, ... — Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, ridge regression, Lasso, ... — Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, ... — Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: PCA, feature selection, non-negative matrix factorization. — Examples

Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning

Modules: grid search, cross validation, metrics. — Examples

Preprocessing

Feature extraction and normalization.

Application: Transforming input data such as text for use with machine learning algorithms.

Modules: preprocessing, feature extraction. — Examples

<http://scikit-learn.org/stable/index.html>



Estimator Interface

Decision Trees

We'll start just by training a single decision tree.

```
In [8]: from sklearn.tree import DecisionTreeClassifier

In [9]: dtree = DecisionTreeClassifier(min_samples_leaf=10, criterion='entropy')

In [10]: dtree.fit(X_train,y_train)

Out[10]: DecisionTreeClassifier(class_weight=None, criterion='entropy', max_depth=None,
                                max_features=None, max_leaf_nodes=None,
                                min_impurity_decrease=0.0, min_impurity_split=None,
                                min_samples_leaf=10, min_samples_split=2,
                                min_weight_fraction_leaf=0.0, presort=False, random_state=None,
                                splitter='best')
```

Prediction and Evaluation

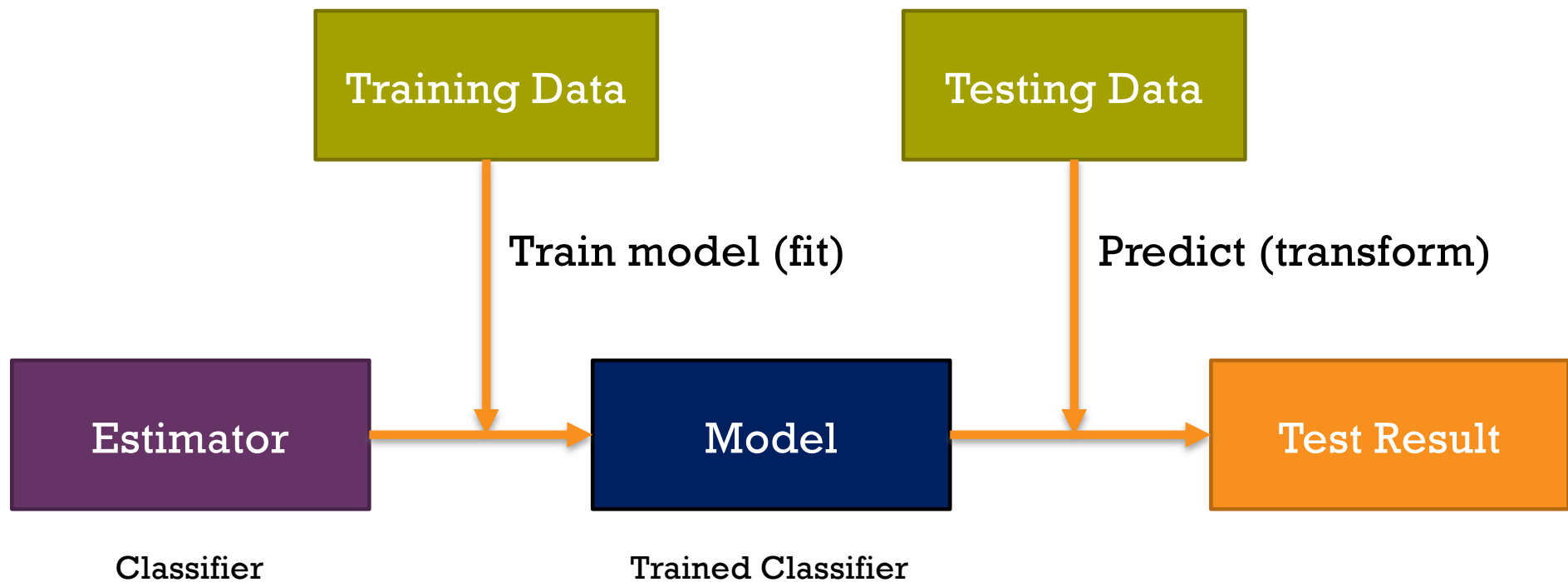
Let's evaluate our decision tree.

```
{11}: predictions = dtree.predict(X_test)

{12}: from sklearn.metrics import classification_report, confusion_matrix

{13}: print(classification_report(y_test,predictions))
```

	precision	recall	f1-score	support
absent	0.85	0.85	0.85	20
present	0.40	0.40	0.40	5
avg / total	0.76	0.76	0.76	25





Training Phase

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

cancer = load_breast_cancer()    # Get some data
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target,
    stratify=cancer.target, random_state=1337)

tree = DecisionTreeClassifier(random_state=7331)
tree.fit(X_train, y_train)    # Learn a Decision Function
```



Testing Phase (Prediction)

```
y_pred = tree.predict(X_test)
```

```
>>> from sklearn.metrics import classification_report
>>> y_true = [0, 1, 2, 2, 2]
>>> y_pred = [0, 0, 2, 2, 1]
>>> target_names = ['class 0', 'class 1', 'class 2']
>>> print(classification_report(y_true, y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

class 0	0.50	1.00	0.67	1
class 1	0.00	0.00	0.00	1
class 2	1.00	0.67	0.80	3

micro avg	0.60	0.60	0.60	5
macro avg	0.50	0.56	0.49	5
weighted avg	0.70	0.60	0.61	5



Prediction Algorithms

- Decision Tree
- (Logistic) Regression
- Neural Networks (NN)
- kNN
- Support Vector Machine
- Deep Learning

BASIC REGRESSION

- LINEAR** `linear_model.LinearRegression()`
Lots of numerical data
- LOGISTIC** `linear_model.LogisticRegression()`
Target variable is categorical

CLASSIFICATION

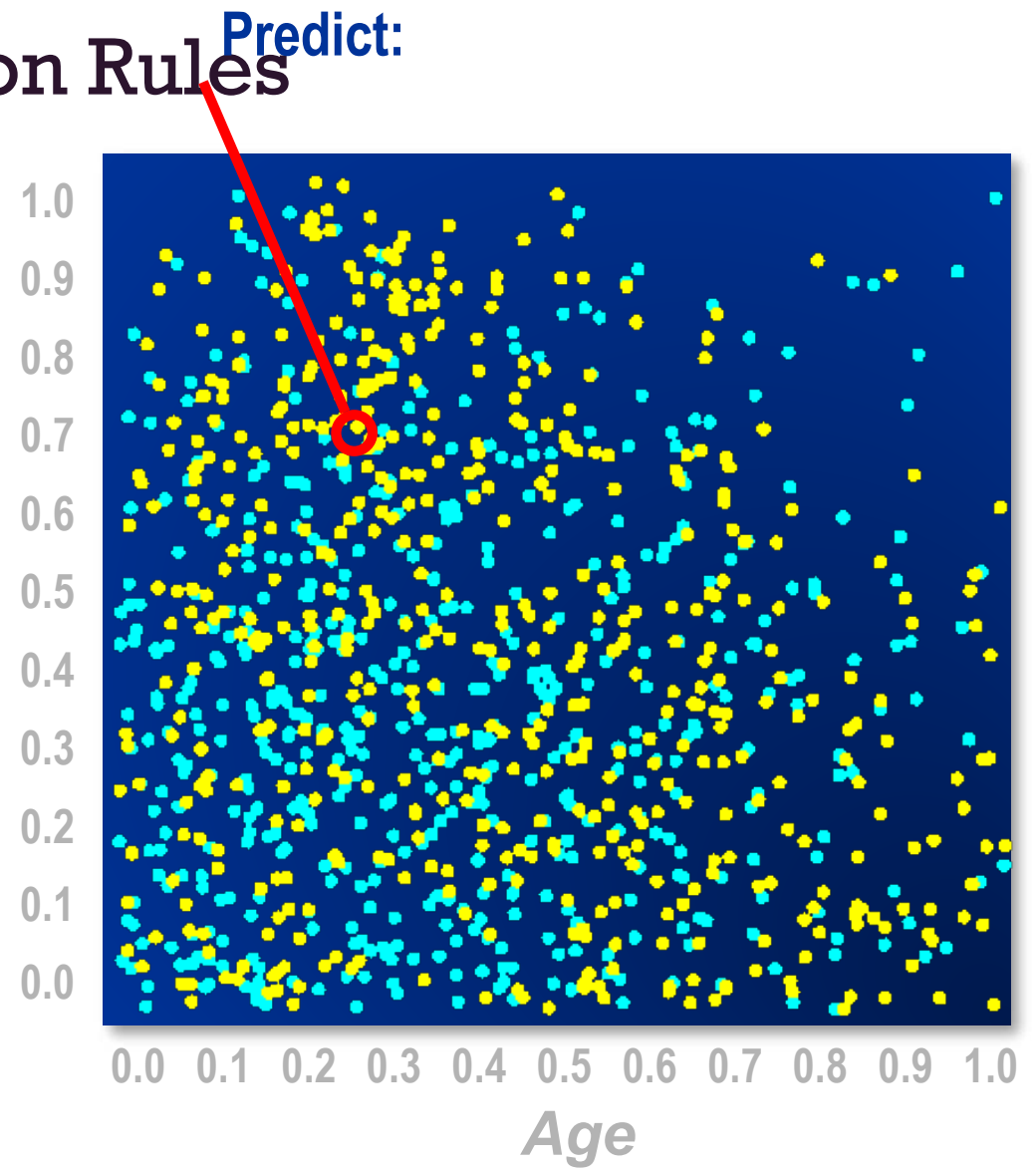
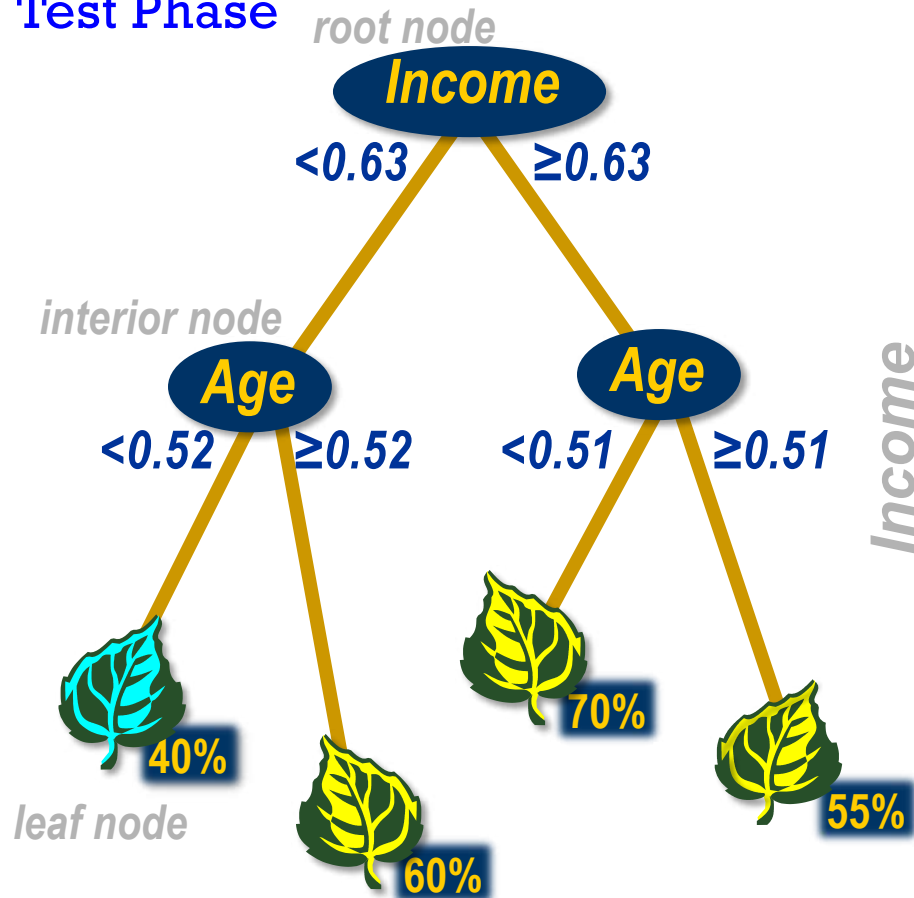
- NEURAL NET** `neural_network.MLPClassifier()`
Complex relationships. Prone to overfitting
Basically magic.
- K-NN** `neighbors.KNeighborsClassifier()`
Group membership based on proximity
- DECISION TREE** `tree.DecisionTreeClassifier()`
If/then/else. Non-contiguous data
Can also be regression
- RANDOM FOREST** `ensemble.RandomForestClassifier()`
Find best split randomly
Can also be regression
- SVM** `svm.SVC()` `svm.LinearSVC()`
Maximum margin classifier. Fundamental
Data Science algorithm
- NAIVE BAYES** `GaussianNB()` `MultinomialNB()` `BernoulliNB()`
Updating knowledge step by step with new info



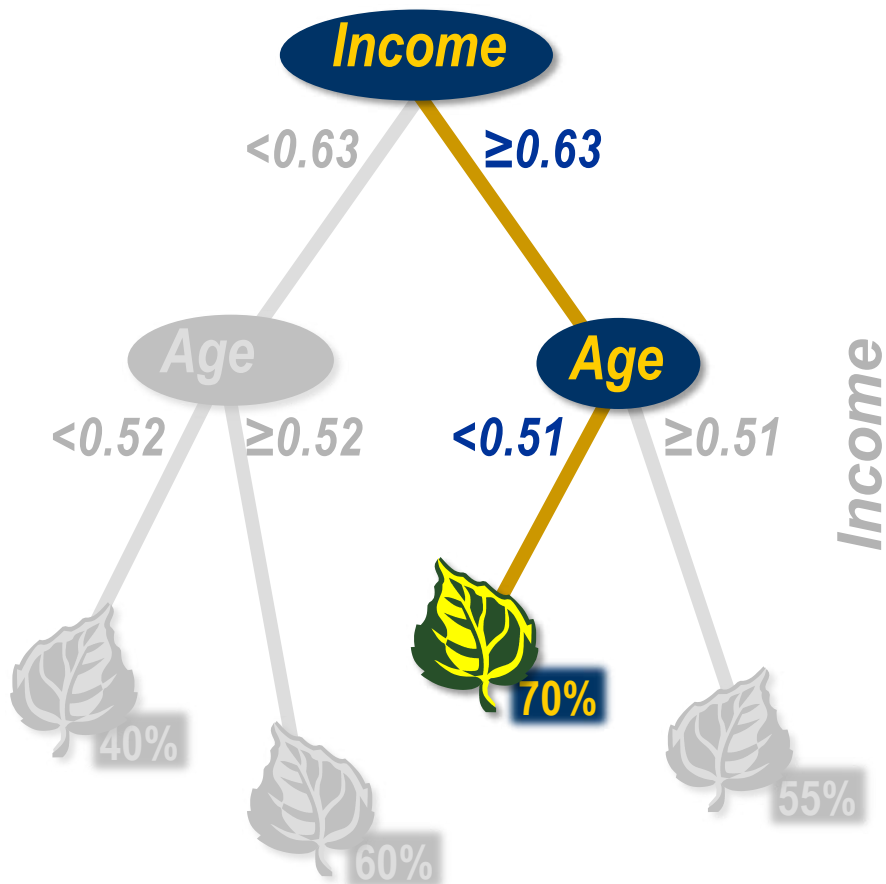
1) Decision Tree

1) Decision Tree Prediction Rules

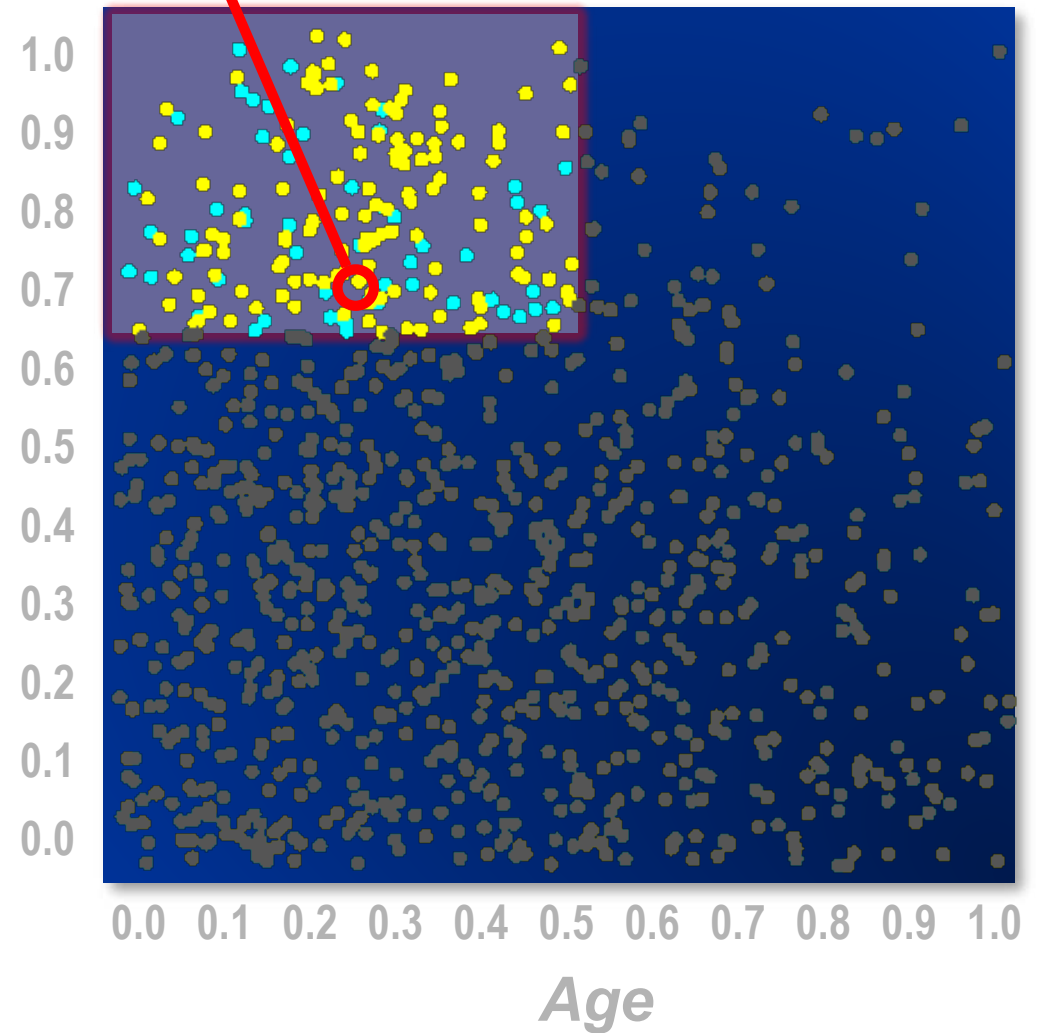
Test Phase



Decision Tree Prediction Rules



Predict: Decision = ●
Estimate = 0.70



Model Essentials: Decision Trees

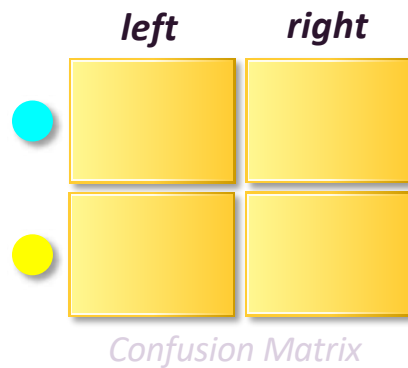
► Predict cases.

Prediction rules

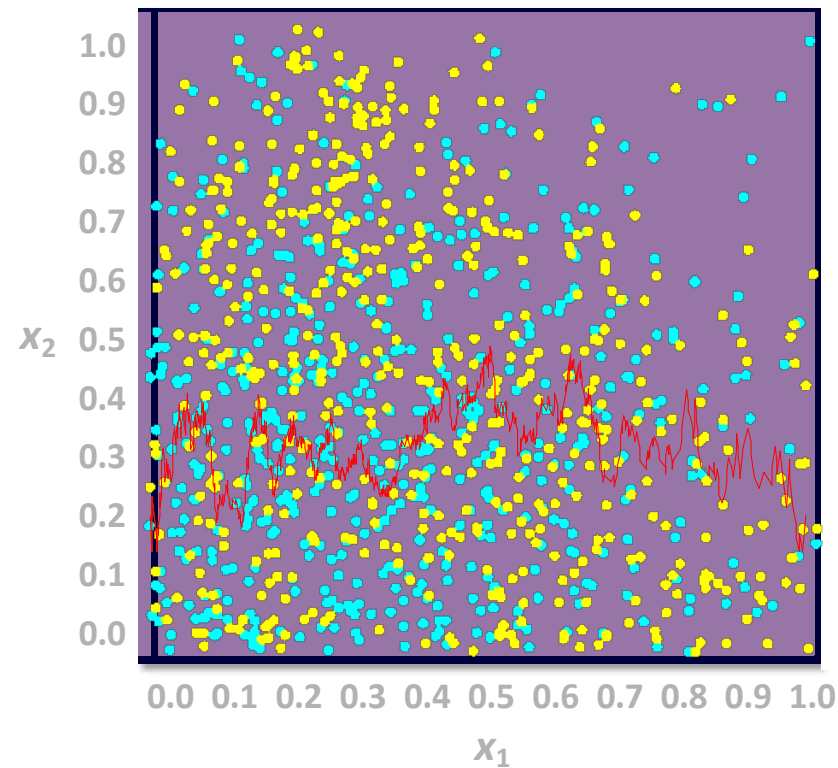
► Select useful predictors.

Split search

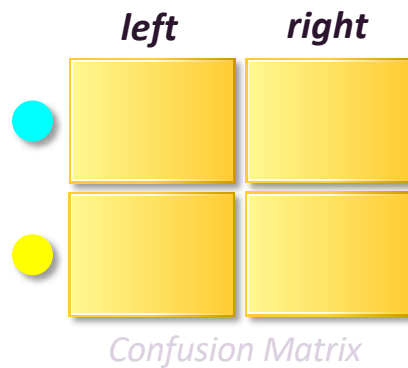
Decision Tree Split Search



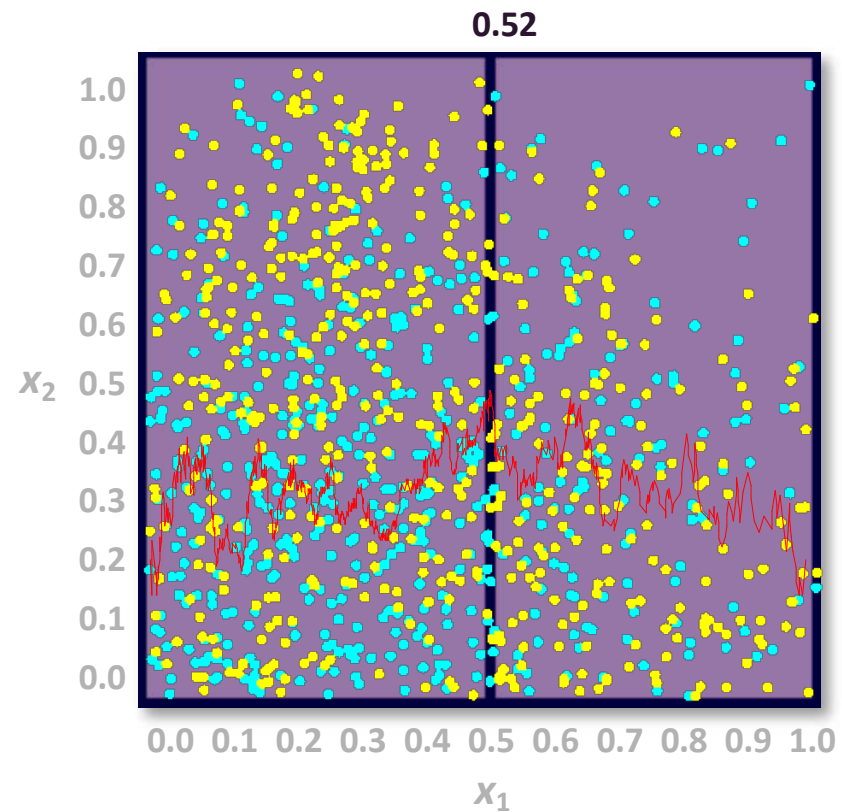
**Calculate information
gain on partitions
on input x_1 .**



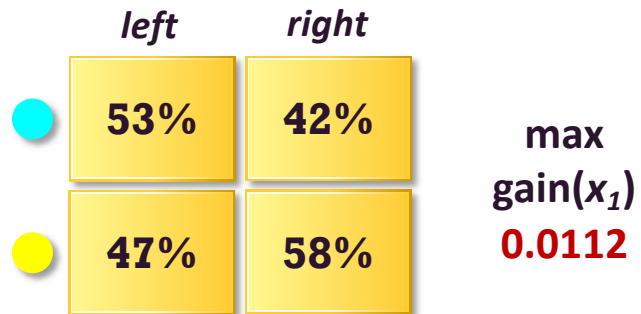
Decision Tree Split Search



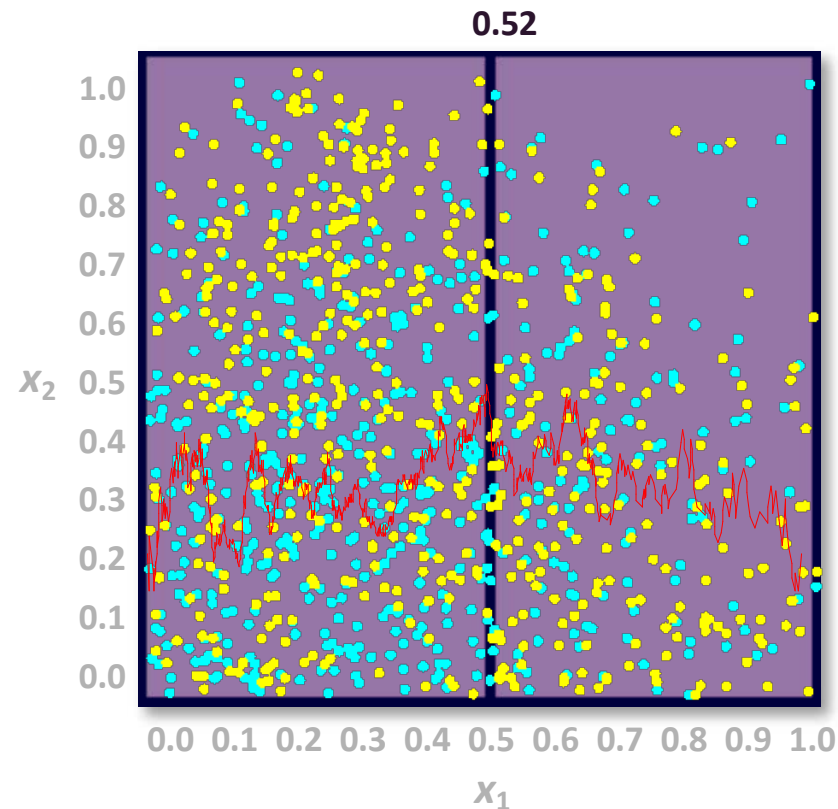
**Calculate the gain
of every partition
on input x_1 .**



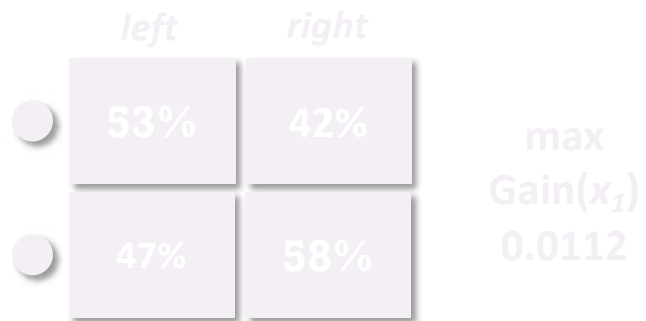
Decision Tree Split Search



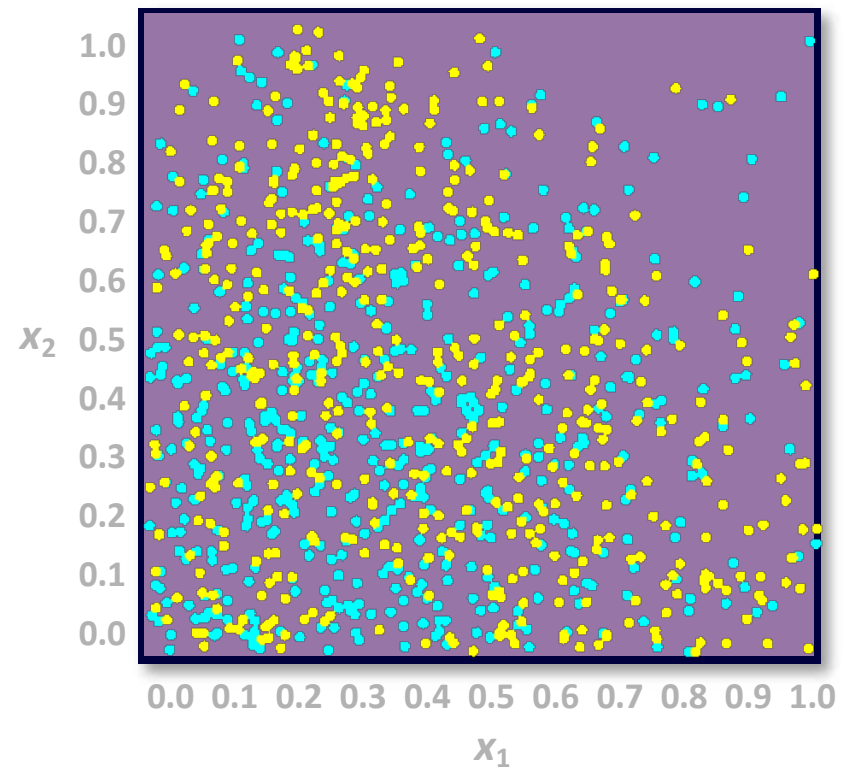
Select the partition
with the maximum
gain.



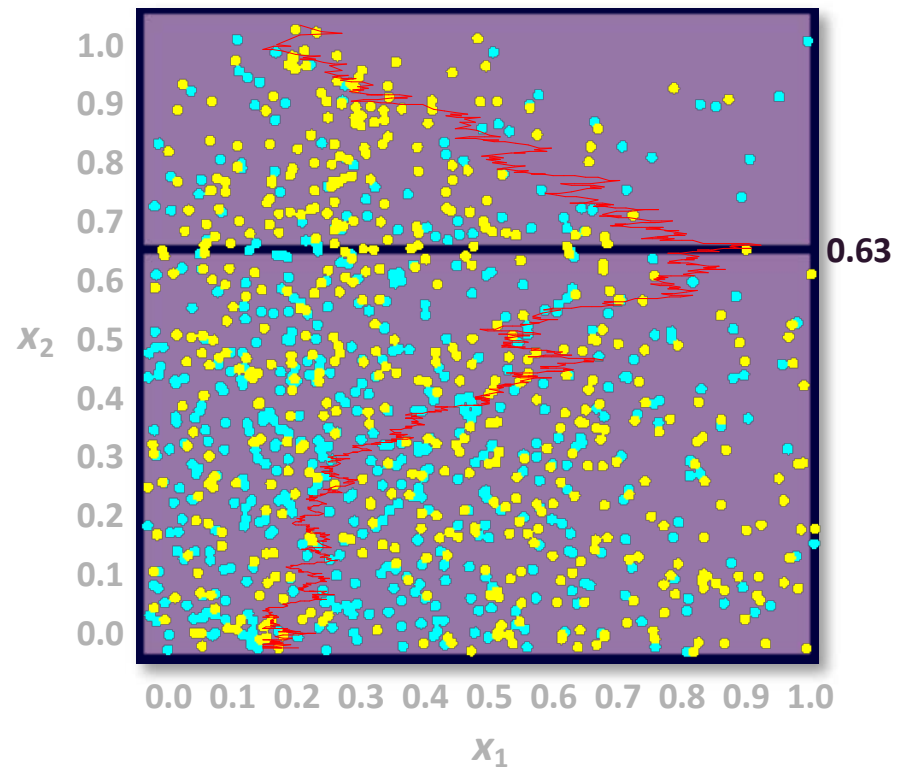
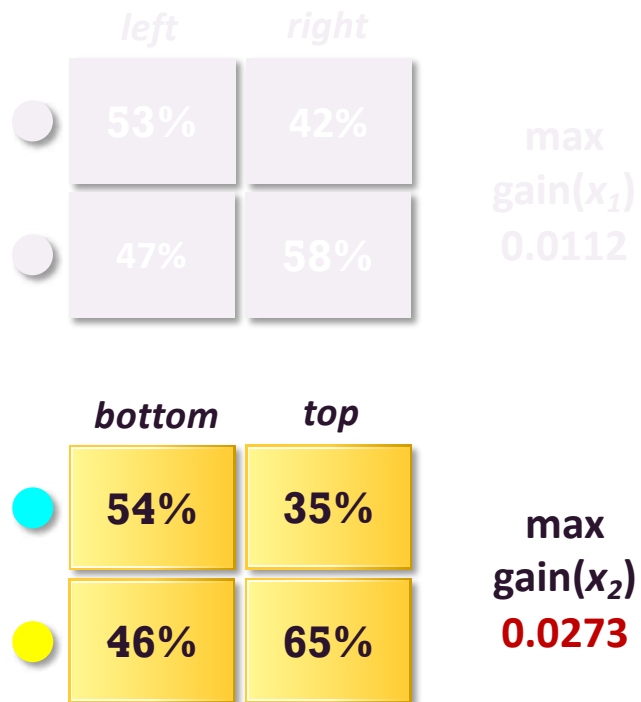
Decision Tree Split Search



Repeat for input x_2 .



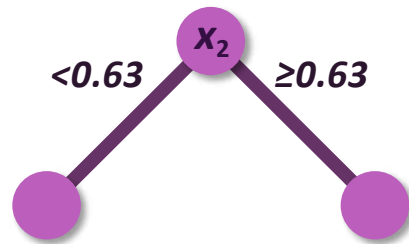
Decision Tree Split Search



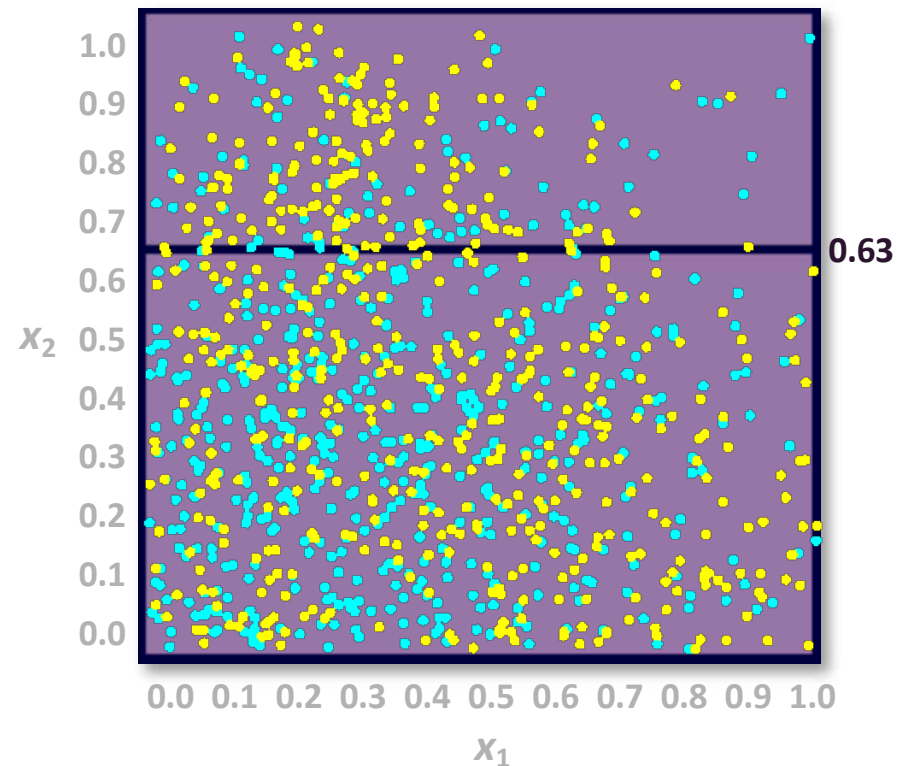
Decision Tree Split Search



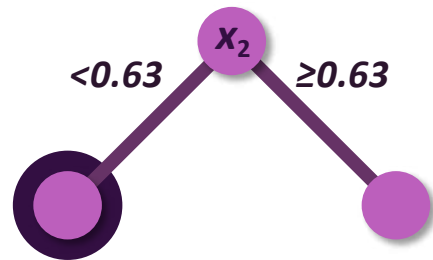
Decision Tree Split Search



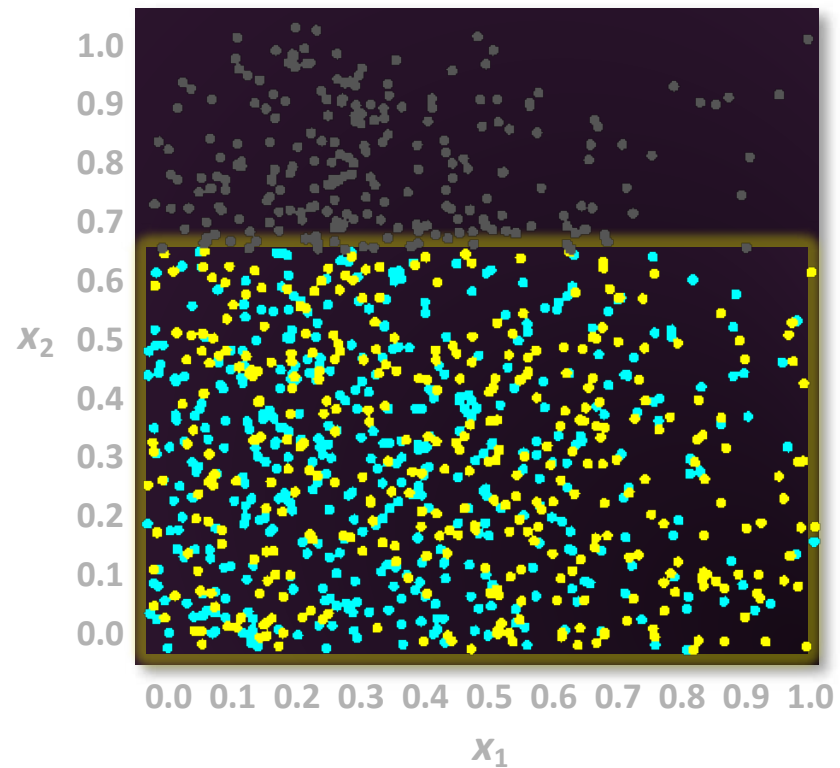
Create a **partition** rule from the best partition across all inputs.



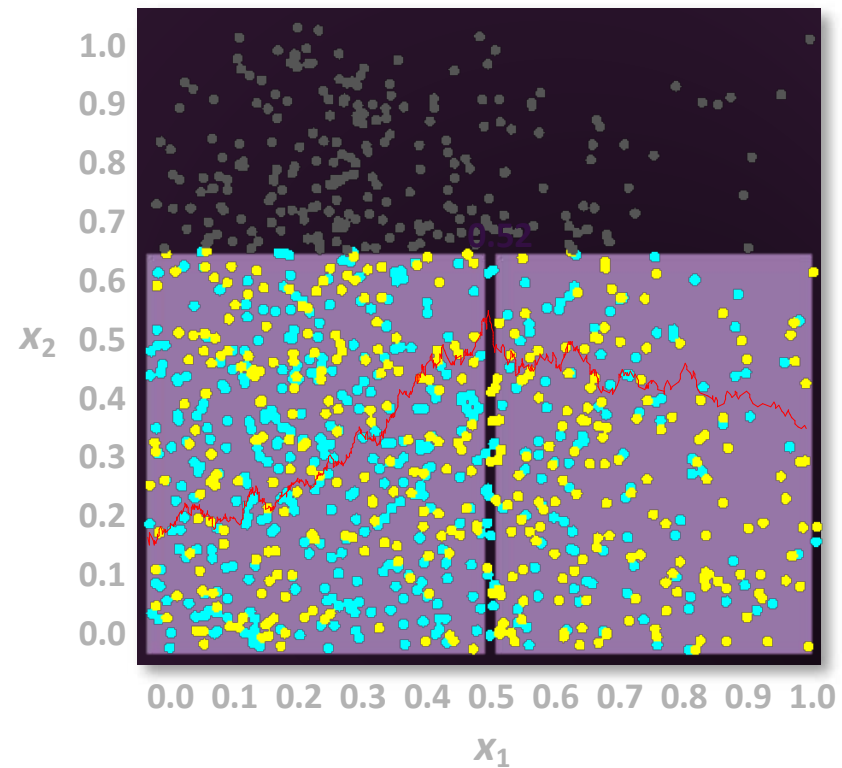
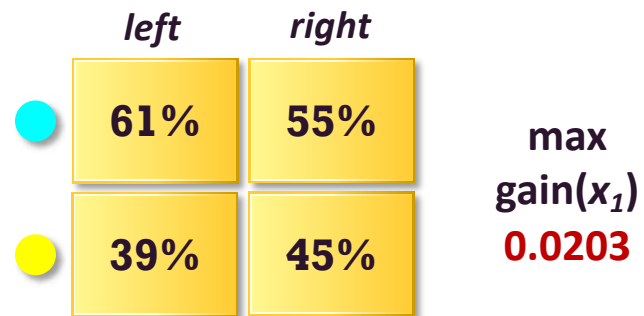
Decision Tree Split Search



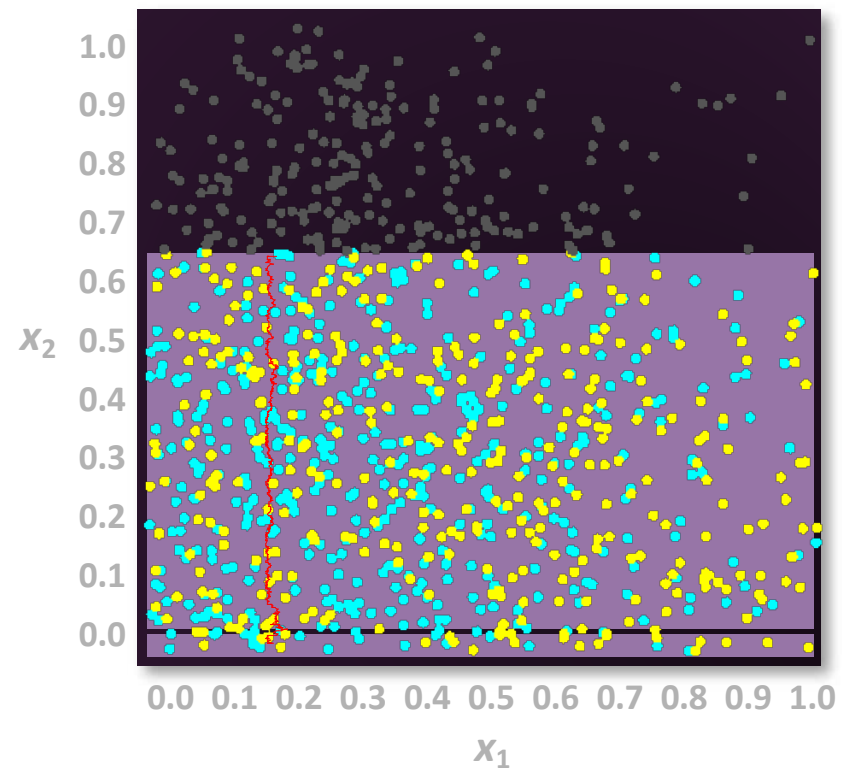
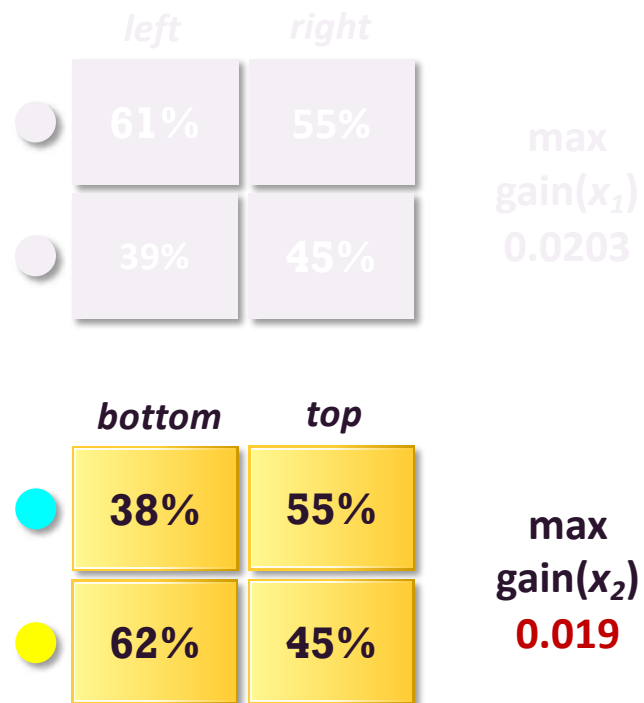
Repeat the process
in each subset.



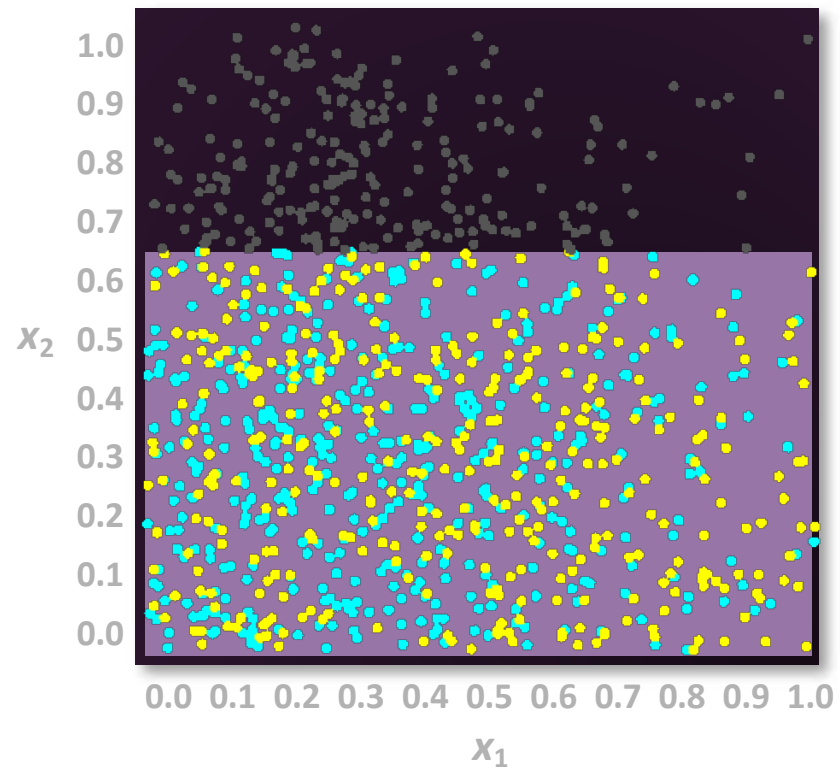
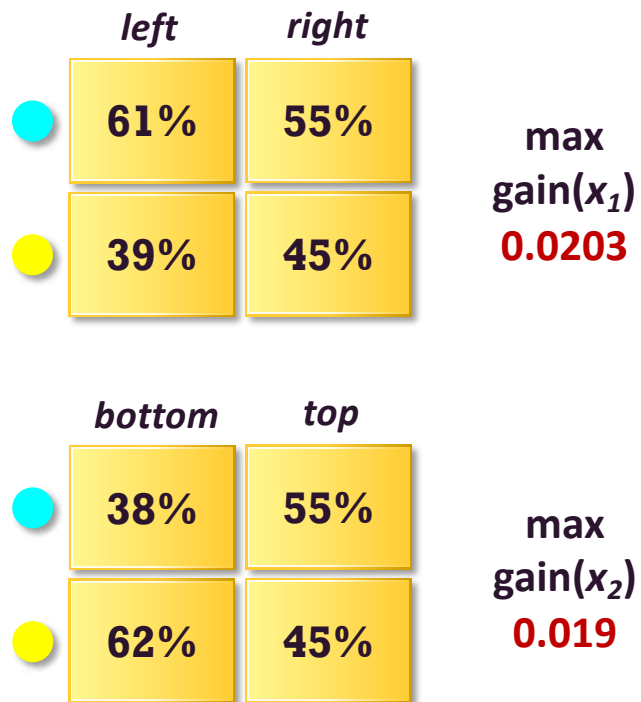
Decision Tree Split Search



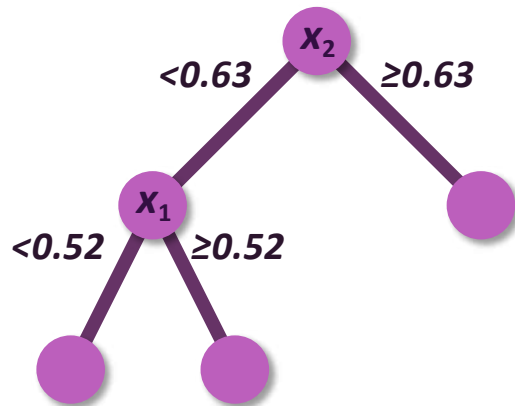
Decision Tree Split Search



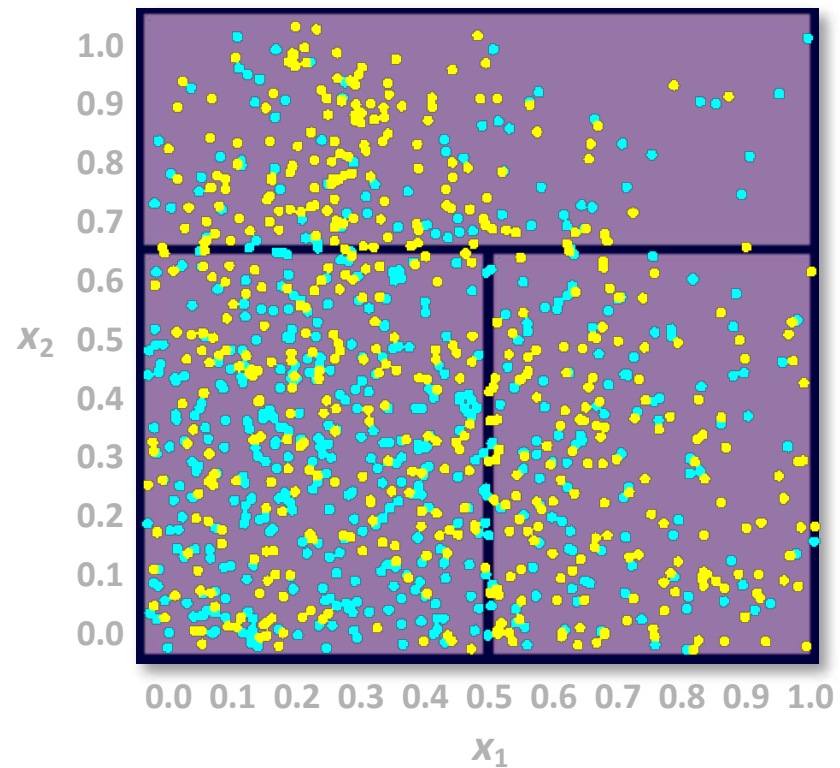
Decision Tree Split Search



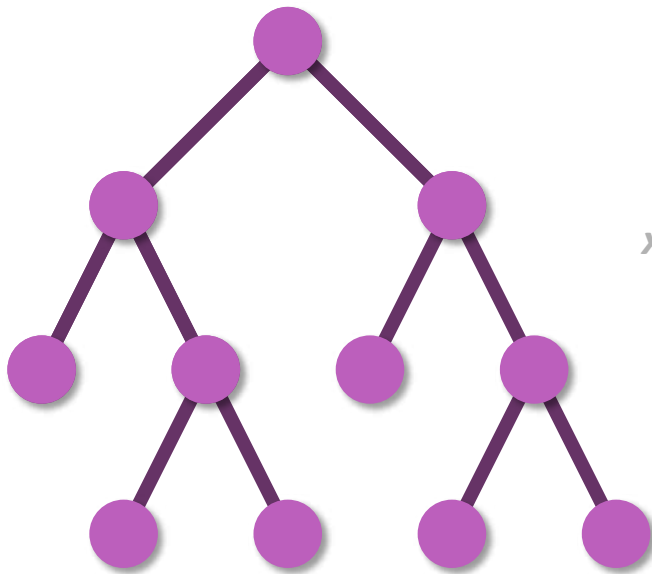
Decision Tree Split Search



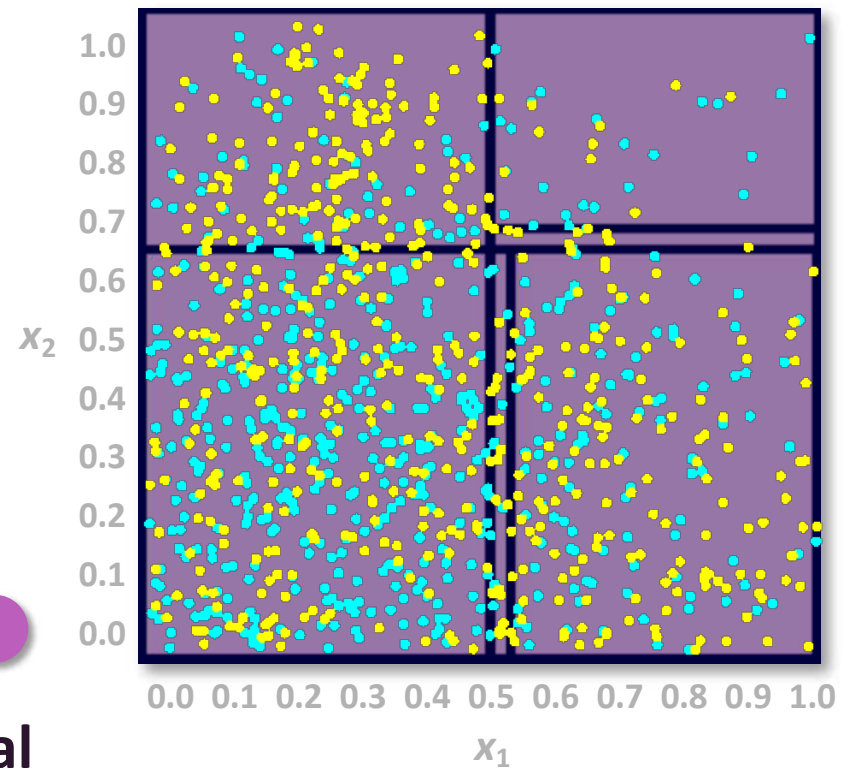
**Create a second
partition rule.**



Decision Tree Split Search



Repeat to form a maximal tree.





1) Entropy (impurity)

- **Entropy** is a measure of disorder or uncertainty and the goal of machine learning models and general is to reduce uncertainty.

$$Entropy = \sum_{i=1}^n -p_i \log_2 p_i$$

scipy.stats.entropy

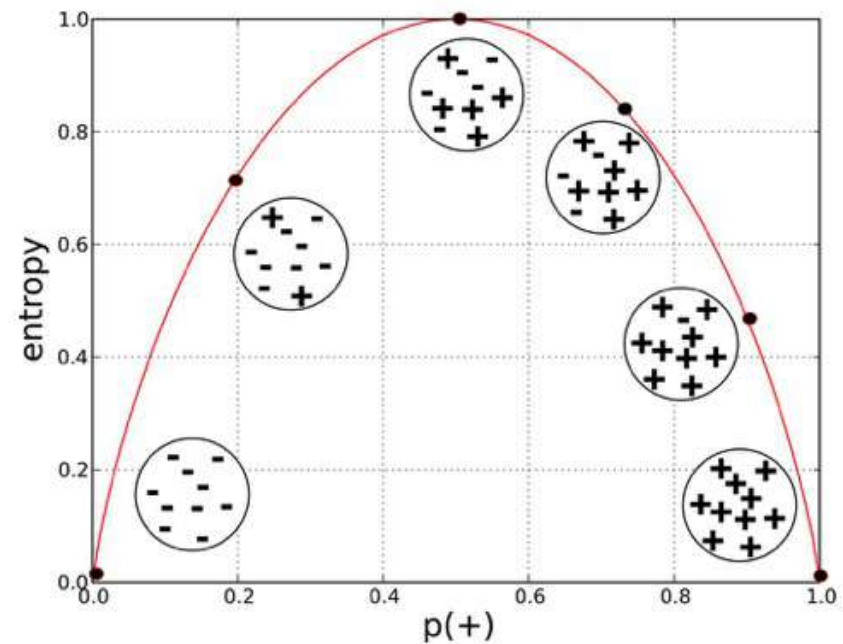
`scipy.stats.entropy(pk, qk=None, base=None, axis=0)`

Calculate the entropy of a distribution for given probability values.

If only probabilities pk are given, the entropy is calculated as $S = -\sum(pk * \log(pk))$, `axis=axis`.

If qk is not None, then compute the Kullback-Leibler divergence $S = \sum(pk * \log(pk / qk))$, `axis=axis`.

This routine will normalize pk and qk if they don't sum to 1.



Source: Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking



Information Gain

Before - After

38

- Which one is Better ?

Split w/ Age: 70
 $\sum$ Entropy: 0.350

Split w/ Age: 50
 $\sum$ Entropy: 0.348

- Information Gain: measure the reduction of this disorder in our target variable/class given additional information

$$InformationGain = Entropy(before) - \sum Entropy(after)$$

- “Before” = Entropy of Parent Node
“After” = Entropy of Child Nodes



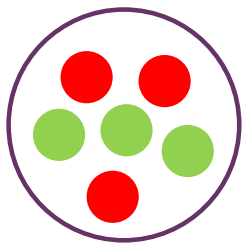
2) Gini Impurity

Gini Reduction = Before - After

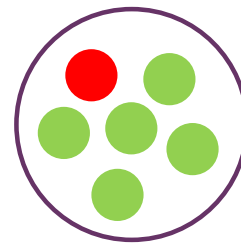
- Another way to measure how well a splitting feature.

$$Gini = 1 - \sum_{i=1}^n (P_i)^2$$

When P is the probability of class i in data-set.



$$Gini = 1 - ((3/6)^2 + (3/6)^2) \\ = 0.5$$

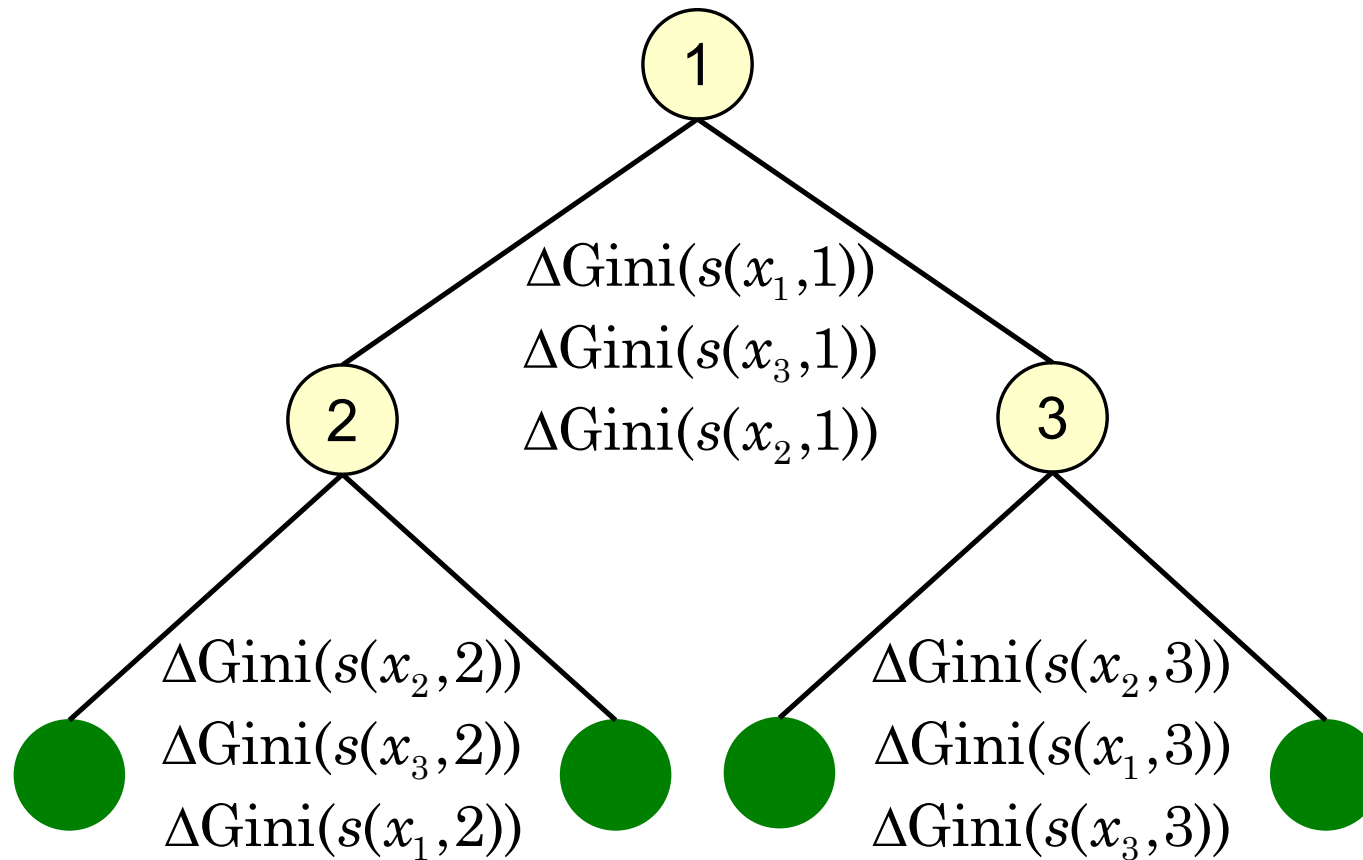


$$Gini = 1 - ((5/6)^2 + (1/6)^2) \\ = 0.28$$

- Easy to calculation, may take less time to build in large dataset.

Source: <https://towardsdatascience.com/understanding-decision-tree-classification-with-scikit-learn-2ddf272731bd>

+ Variable Importance



Types of Decision Tree

Algorithm	Splitting Measure
ID3	Entropy
C4.5	Gain Ratio
CART	Gini index
CHAID	Chi-squared test



`sklearn.tree.DecisionTreeClassifier`

```
class sklearn.tree.DecisionTreeClassifier(criterion='gini', splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, class_weight=None, presort='deprecated', ccp_alpha=0.0)
```

[\[source\]](#)

A decision tree classifier.

Read more in the [User Guide](#).

Parameters:

criterion : {"gini", "entropy"}, default="gini"

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain.

splitter : {"best", "random"}, default="best"

The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split.

max_depth : int, default=None

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : int or float, default=2

The minimum number of samples required to split an internal node:



Decision Tree with Regression

`sklearn.tree.DecisionTreeRegressor`

```
class sklearn.tree.DecisionTreeRegressor(criterion='mse', splitter='best', max_depth=None, min_samples_split=2,
min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None, max_leaf_nodes=None,
min_impurity_decrease=0.0, min_impurity_split=None, presort='deprecated', ccp_alpha=0.0)
```

[\[source\]](#)

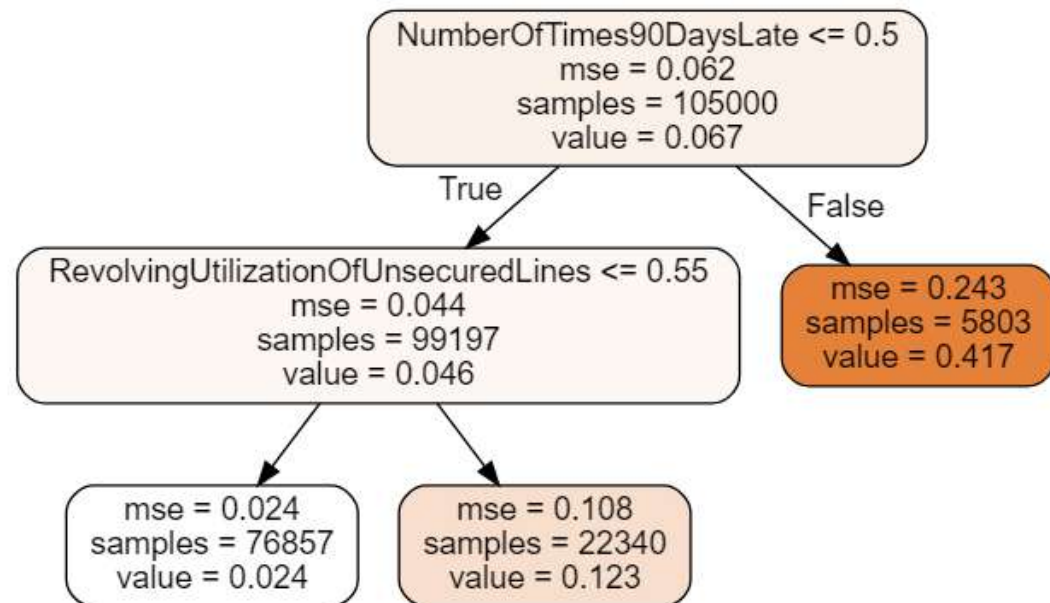
A decision tree regressor.

Read more in the [User Guide](#).

Parameters: criterion : {"mse", "friedman_mse", "mae"},

MSE: Mean Square Error

MAE: Mean Absolute Error



+ Build your Decision Tree (cont.)

44

■ Code Section 2.4

```
select_features = 'age' #@param ['age']
target_column = 'SeriousDlqin2yrs' #@param ['SeriousDlqin2yrs']

select_features = [select_features]
# Setup Features & Target
features = train_df[select_features]
target = train_df[target_col]

# Setup Parameters
Criterion = 'Entropy' #@param ['Entropy', 'Gini']
max_depth = 1
dtree = tree.DecisionTreeClassifier(
    criterion=Criterion.lower(),
    max_depth=max_depth,
)

# Training you tree
dtree.fit(features, target)
```

DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None, criterion='entropy', max_depth=1, max_features=None, max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, min_samples_leaf=1, min_samples_split=2, min_weight_fraction_leaf=0.0, presort='deprecated', random_state=None, splitter='best')

select_features: age
target_column: SeriousDlqin2yrs
Criterion: Entropy

+ Model Visualization (**read** the model)

45

Basic Description

1. `get_params` : Get all model parameters
2. `get_depth` : Get depth of trained tree
3. `get_n_leaves` : Get number of nodes
4. `tree_.impurity` : Get Gini/Entropy values

```
1 Parameters: {  
  "ccp_alpha": 0.0,  
  "class_weight": null,  
  "criterion": "entropy",  
  "max_depth": 1,  
  "max_features": null,  
  "max_leaf_nodes": null,  
  "min_impurity_decrease": 0.0,  
  "min_impurity_split": null,  
  "min_samples_leaf": 1,  
  "min_samples_split": 2,  
  "min_weight_fraction_leaf": 0.0,  
  "presort": "deprecated",  
  "random_state": null,  
  "splitter": "best"  
}  
2 Tree Depth: 1  
  Number of Node: 2  
3  
4 Entropy of Nodes: [0.35401168 0.42690946 0.2175608 ]
```



Model Visualization (cont.)

46

sklearn.tree.plot_tree

```
sklearn.tree.plot_tree(decision_tree, *, max_depth=None, feature_names=None, class_names=None, label='all', filled=False, impurity=True, node_ids=False, proportion=False, rounded=False, precision=3, ax=None, fontsize=None)
```

[\[source\]](#)

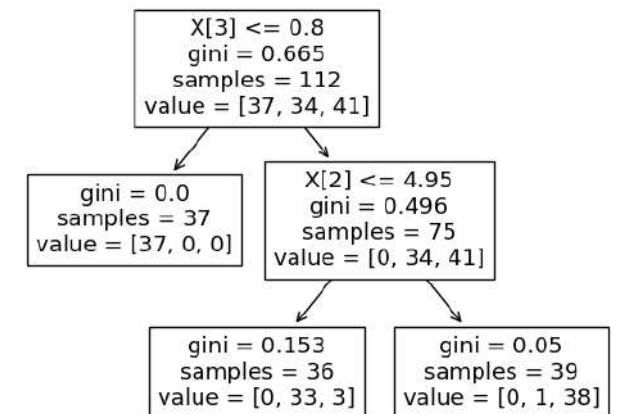
Plot a decision tree.

The sample counts that are shown are weighted with any sample_weights that might be present.

The visualization is fit automatically to the size of the axis. Use the `figsize` or `dpi` parameter to control the size of the rendering.

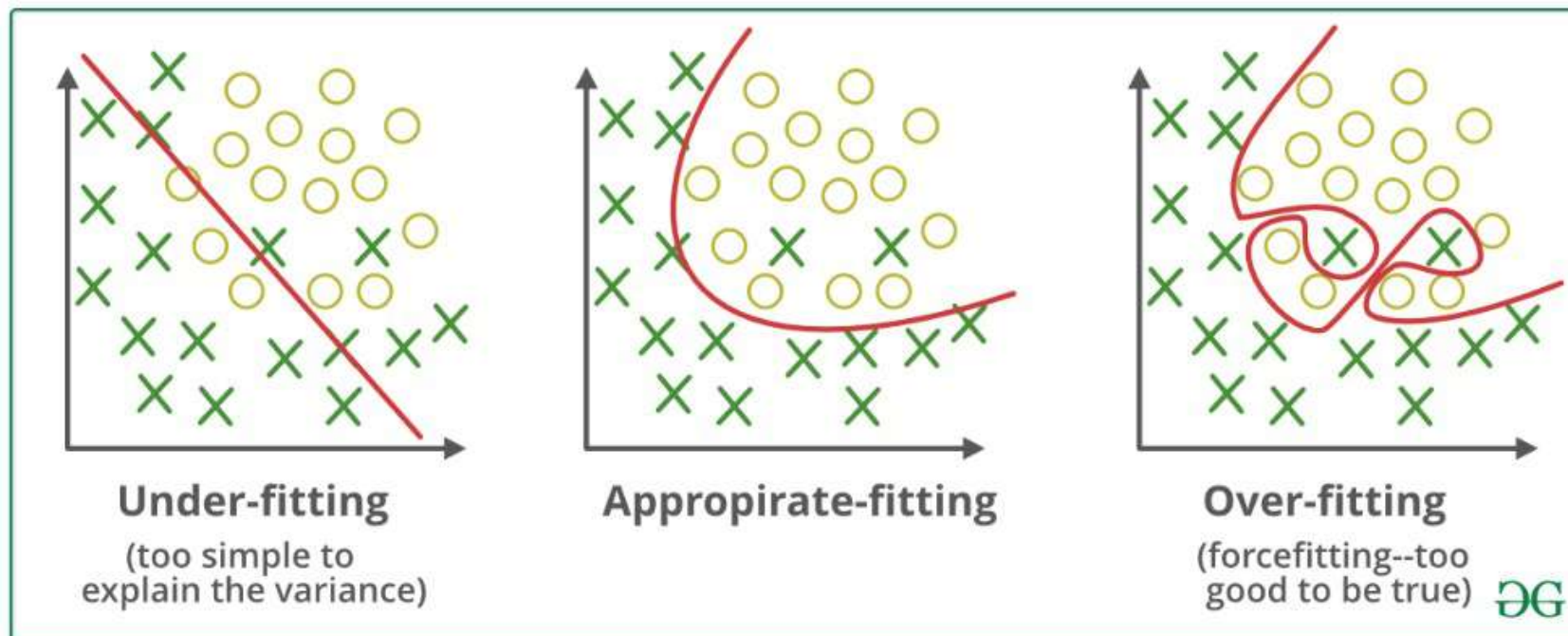
Read more in the [User Guide](#).

New in version 0.21.



+ Overfitting

47



Source: <https://towardsdatascience.com/underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6fe4a8a49dbf>

+ Pruning Tree

ccp_alpha : non-negative float, default=0.0

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. See [Minimal Cost-Complexity Pruning](#) for details.

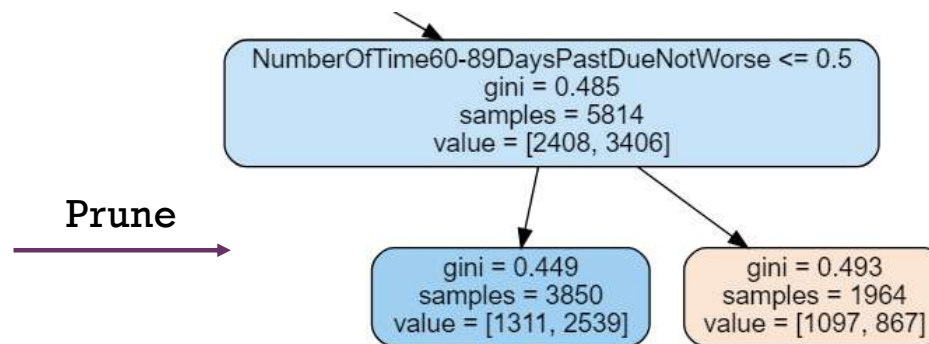
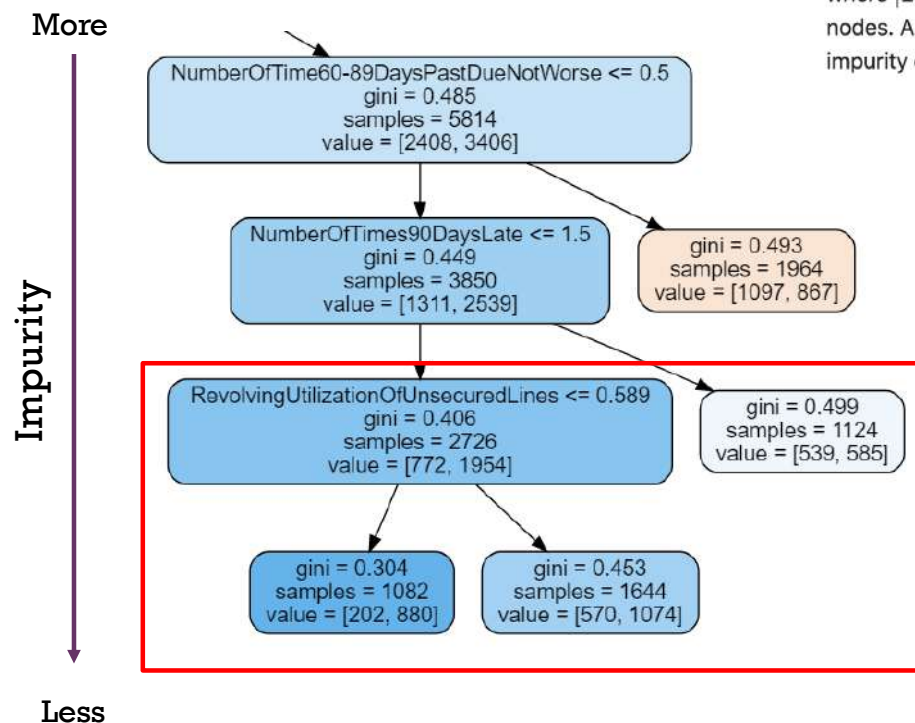
New in version 0.22.

<https://scikit-learn.org/stable/modules/tree.html#minimal-cost-complexity-pruning/>

48

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

where $|T|$ is the number of terminal nodes in T and $R(T)$ is traditionally defined as the total misclassification rate of the terminal nodes. Alternatively, scikit-learn uses the total sample weighted impurity of the terminal nodes for $R(T)$. As shown above, the impurity of a node depends on the criterion. Minimal cost-complexity pruning finds the subtree of T that minimizes $R_{\alpha}(T)$.



Test pruning parameter by `.cost_complexity_pruning_path`

```
dtree.cost_complexity_pruning_path(features, target)
```

```
{'ccp_alphas': array([0.00000000e+00, 8.91619910e-05, 1.17039010e-04, 1.29787452e-04,
1.94525732e-04, 3.18283592e-04, 4.15157131e-04, 5.84363721e-04,
6.81710606e-04, 1.05951846e-03, 1.17765555e-03, 1.83295138e-03,
3.24166605e-03, 1.41432990e-02]),
'impurities': array([0.10075641, 0.10084557, 0.10096261, 0.1010924 , 0.10128692,
0.10160521, 0.10202036, 0.10260473, 0.10328644, 0.10434596,
0.10552361, 0.10735656, 0.11059823, 0.12474153])}
```

ccp_alphas: Values which affect pruning (use lower bound value)

Impurities: Impurity after pruning (default=0; no pruning)

When impurity is too low, it cause more overfitting



Important Parameters in Decision Tree



49

- Splitting measure (criterion) : gini / entropy
- Maximum depth : ~5-10 (depend on number of feature)
- Maximum leaf nodes : depend on number of class (target) and feature
- Minimum sample split : 5 – 20% (depend on number of data)
- Minimum impurity decrease : (default 0)
- Pruning adjustment (ccp_alpha) : 0.001 - 0.01 (depend on size of tree)

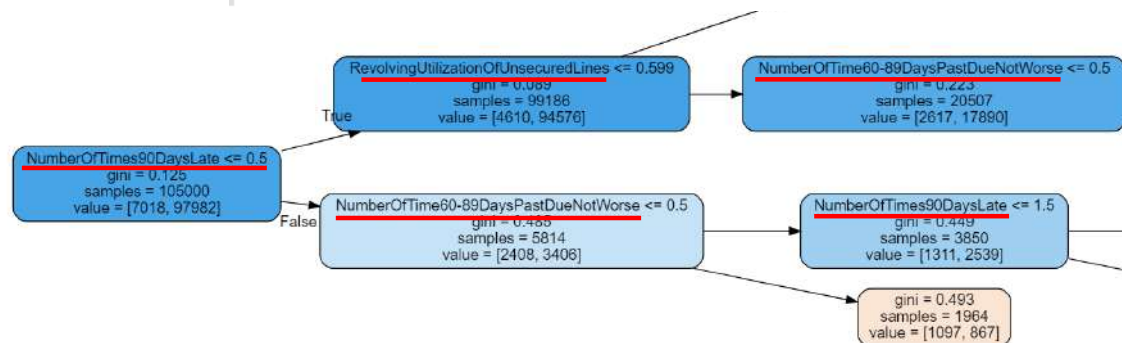


Which features are important ?

- Check how important by `.feature_importances_`
- The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as **the gini importance**.

```
dtree.feature_importances_
```

```
array([0.15330291, 0.00371739, 0.07500437, 0.        , 0.        ,
       0.        , 0.61403332, 0.        , 0.15394201, 0.        ])
```

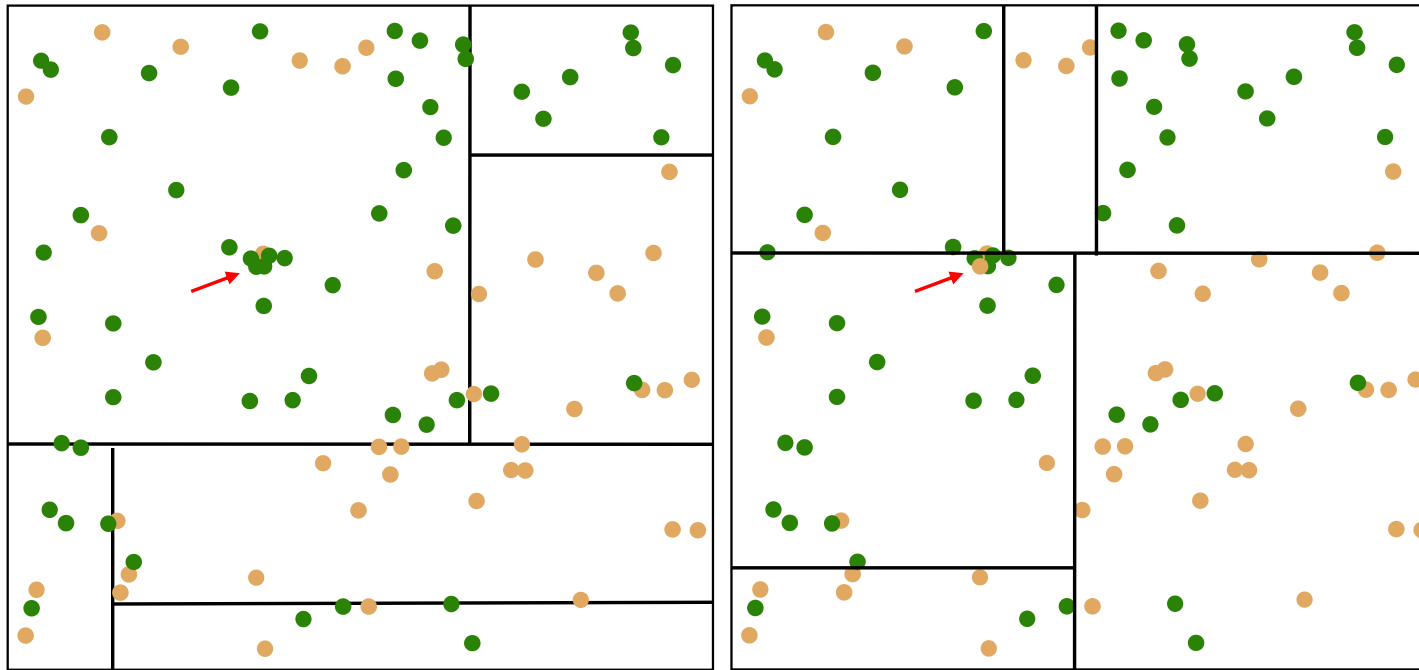


===== Features Important =====

0.614 : Numberoftimes90dayslate
 0.154 : Numberoftime60-89dayspastduenotworse
 0.153 : Revolvingutilizationofunsecuredlines
 0.075 : Numberoftime30-59dayspastduenotworse
 0.004 : Age
 0.000 : Debtratio
 0.000 : Monthlyincome
 0.000 : Numberofopencreditlinesandloans
 0.000 : Numberrealestateloansorlines
 0.000 : Numberofdependents

+ Instability

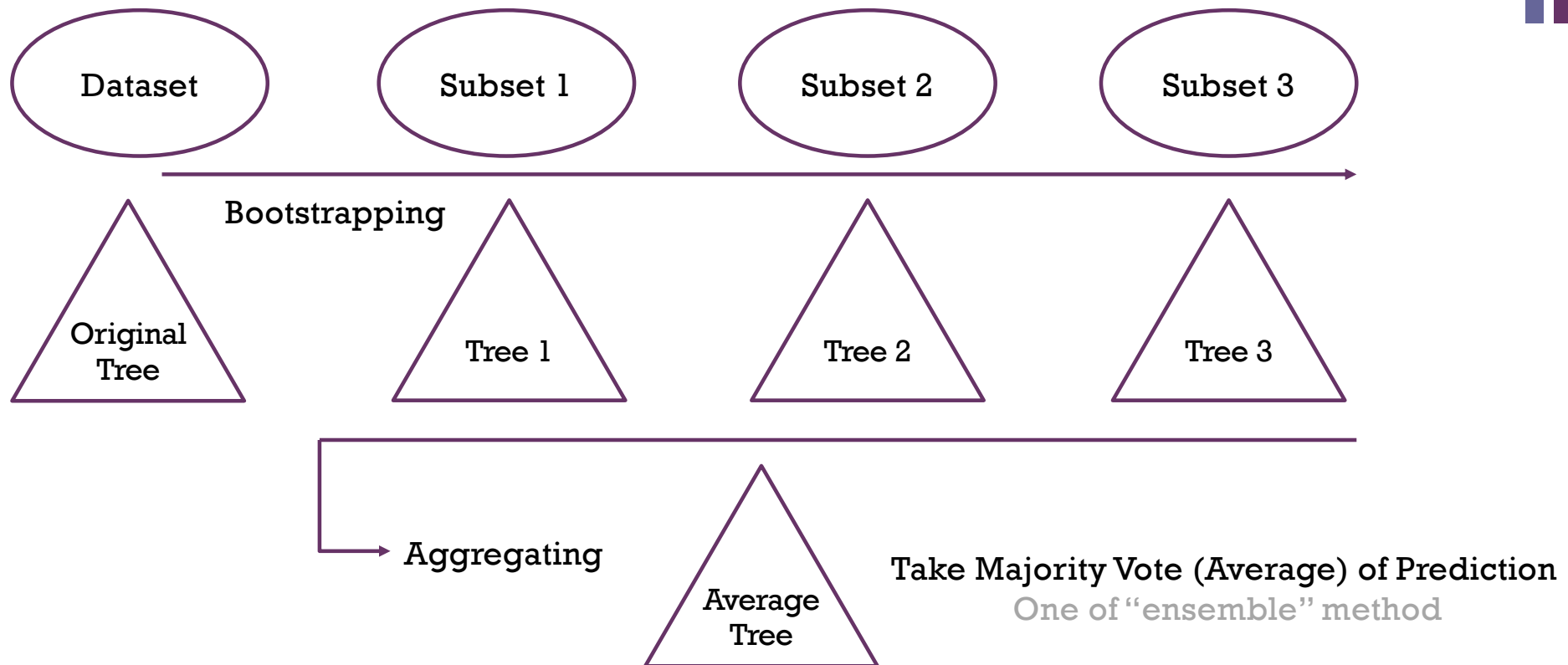
One reversal



Accuracy = 81%

Accuracy = 80%

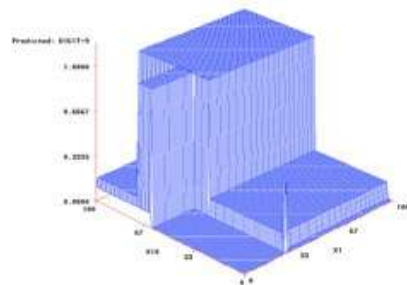
+ Bagging (Bootstrap Aggregation)



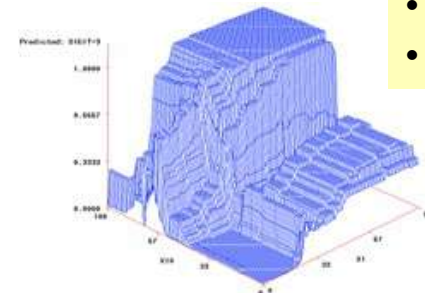
[illegible]

Random Forest Algorithm (RF)

- Forest algorithm samples the **rows** and the **columns** at each step.
- Forest takes bootstrap samples of the **rows** in training data (sampling with replacement)
- At each step, a set of variables (**columns**) is sampled.
- This increases variation among trees in the **ensemble** often leads to improved prediction accuracy.



Single Tree

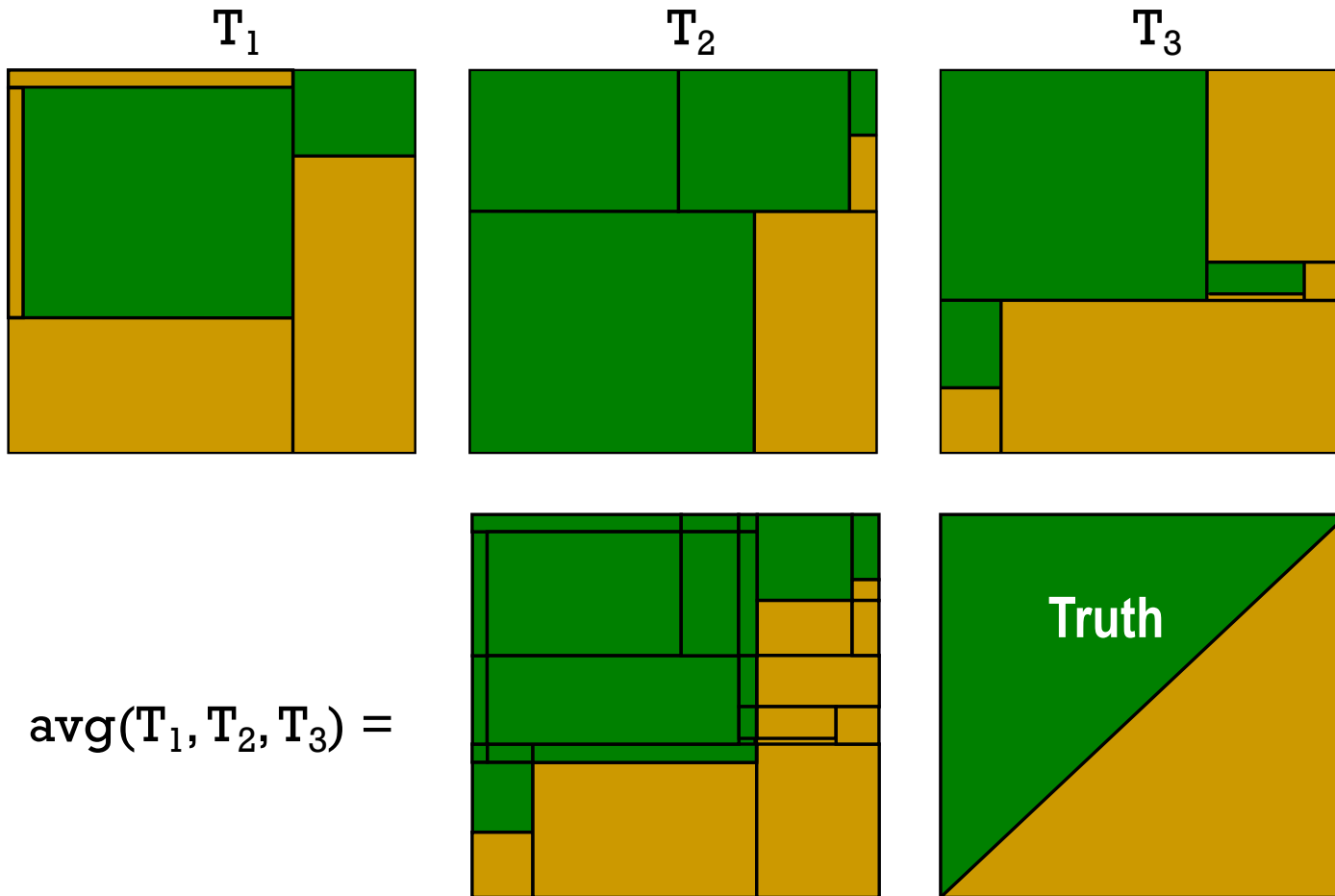


Random Forest

Important Params:

- #Tree
- #Columns (variables)
- #Rows (examples)

Combine



3.2.4.3.1.

`sklearn.ensemble.RandomForestClassifier`

3.2.4.3.1.1. Examples using

`sklearn.ensemble.RandomForestClassifier`

3.2.4.3.1. `sklearn.ensemble.RandomForestClassifier`

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None, min_impurity_decrease=0.0, min_impurity_split=None, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False, class_weight=None, ccp_alpha=0.0, max_samples=None)
```

[\[source\]](#)

A random forest classifier.

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. The sub-sample size is always the same as the original input sample size but the samples are drawn with replacement if `bootstrap=True` (default).

Read more in the [User Guide](#).

Parameters:

`n_estimators` : integer, optional (default=100)

The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

`criterion` : string, optional (default="gini")

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "entropy" for the information gain. Note: this parameter is tree-specific.

3.2.4.3.2.

`sklearn.ensemble.RandomForestRegressor`

3.2.4.3.2.1. Examples using

`sklearn.ensemble.RandomForestRegressor`

3.2.4.3.2. `sklearn.ensemble.RandomForestRegressor`

```
class sklearn.ensemble.RandomForestRegressor(n_estimators=100, criterion='mse', max_depth=None, min_samples_split=2,
min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None, min_impurity_decrease=0.0,
min_impurity_split=None, bootstrap=True, oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False,
ccp_alpha=0.0, max_samples=None) ¶
```

[\[source\]](#)

A random forest regressor.

A random forest is a meta estimator that fits a number of classifying decision trees on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. The sub-sample size is always the same as the original input sample size but the samples are drawn with replacement if `bootstrap=True` (default).

Read more in the [User Guide](#).

Parameters:

`n_estimators` : integer, optional (default=10)

The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

`criterion` : string, optional (default="mse")

The function to measure the quality of a split. Supported criteria are "mse" for the mean squared error, which is equal to variance reduction as feature selection criterion, and "mae" for the mean absolute error.

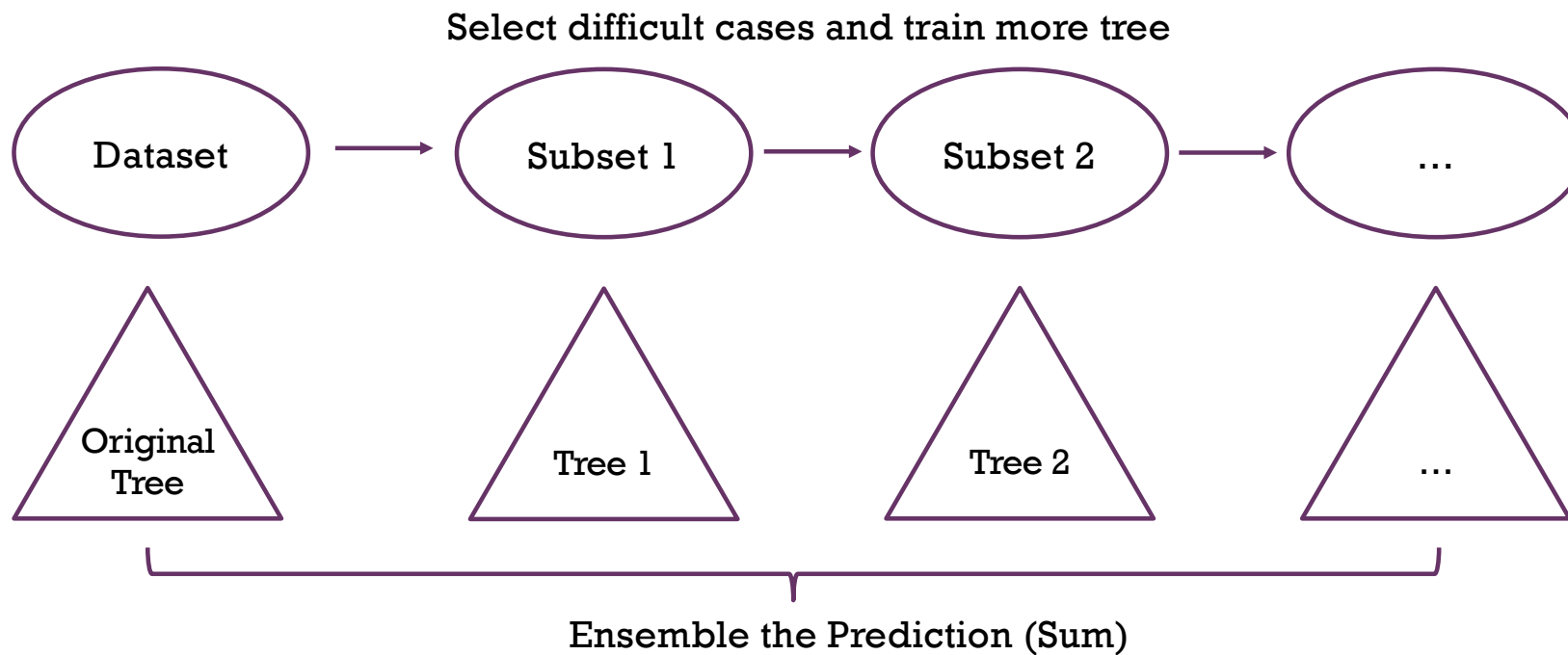
New in version 0.18: Mean Absolute Error (MAE) criterion.



Boosting

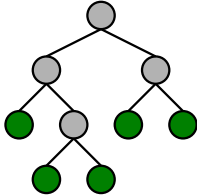
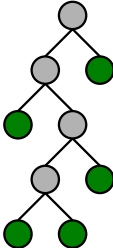
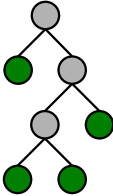
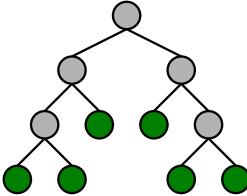
58

- Boosting is a method of converting **weak learners into strong learners**.
- In boosting, each new tree is a fit on a modified version of the original data set.



Boosting (cont.)

	k=1		k=2		k=3		k=4 ...
<u>case</u>	<u>freq</u>	<u>m</u>	<u>freq</u>	<u>m</u>	<u>freq</u>	<u>m</u>	<u>freq</u>
1	1	1	1.5	1	.5	2	.97
2	1	0	.75	0	.25	0	.06
3	1	1	1.5	2	4.25	3	4.69
4	1	0	.75	1	.5	1	.11
5	1	0	.75	0	.25	0	.06
6	1	0	.75	0	.25	1	.11

*Shown is Arc-x4, one method of boosting



Boosting in sklearn

- AdaBoost, short for “Adaptive Boosting”

`sklearn.ensemble.AdaBoostClassifier`

```
class sklearn.ensemble.AdaBoostClassifier(base_estimator=None, *, n_estimators=50, learning_rate=1.0, algorithm='SAMME.R', random_state=None)
```

[\[source\]](#)

`sklearn.ensemble.AdaBoostRegressor`

```
class sklearn.ensemble.AdaBoostRegressor(base_estimator=None, *, n_estimators=50, learning_rate=1.0, loss='linear', random_state=None)
```

[\[source\]](#)

Example

```
>>> from sklearn.ensemble import AdaBoostClassifier
>>> from sklearn.tree import DecisionTreeClassifier
>>> boosting = AdaBoostClassifier(DecisionTreeClassifier(),
...                               max_samples=0.5, max_features=0.5)
```

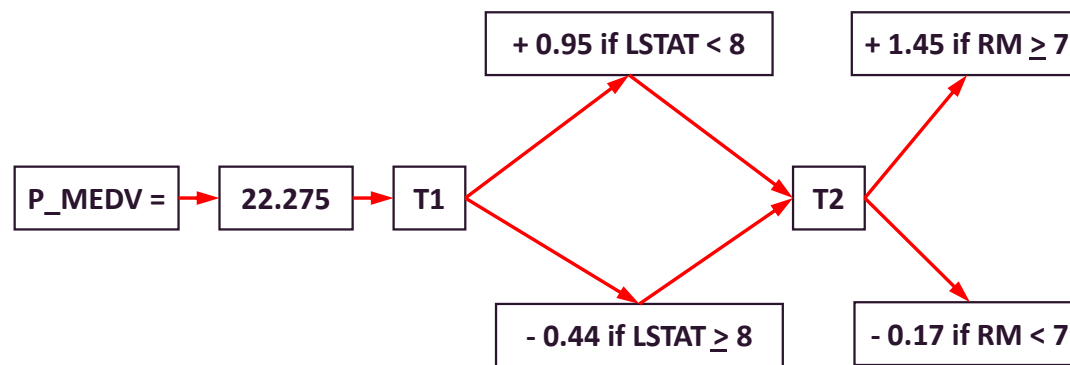
>>>

<https://towardsdatascience.com/boosting-algorithm-adaboost-b6737a9ee60c#:~:text=AdaBoost,classification%20can%20be%20represented%20as>



Gradient Boosting Tree

- Boosting the performance by using **ensemble trees**
- Each tree in the next step aims to **predict error**.
- So, the prediction result is **the summation of all trees** (not average any more).
- In the final step, **weights** are assigned to all leaf nodes using **gradient descent** in order to finally reduce errors.



Boston House Price Prediction

- 'RM' is the average number of rooms among homes in the neighborhood.
- 'LSTAT' is the percentage of homeowners in the neighborhood considered "lower class" (working poor).

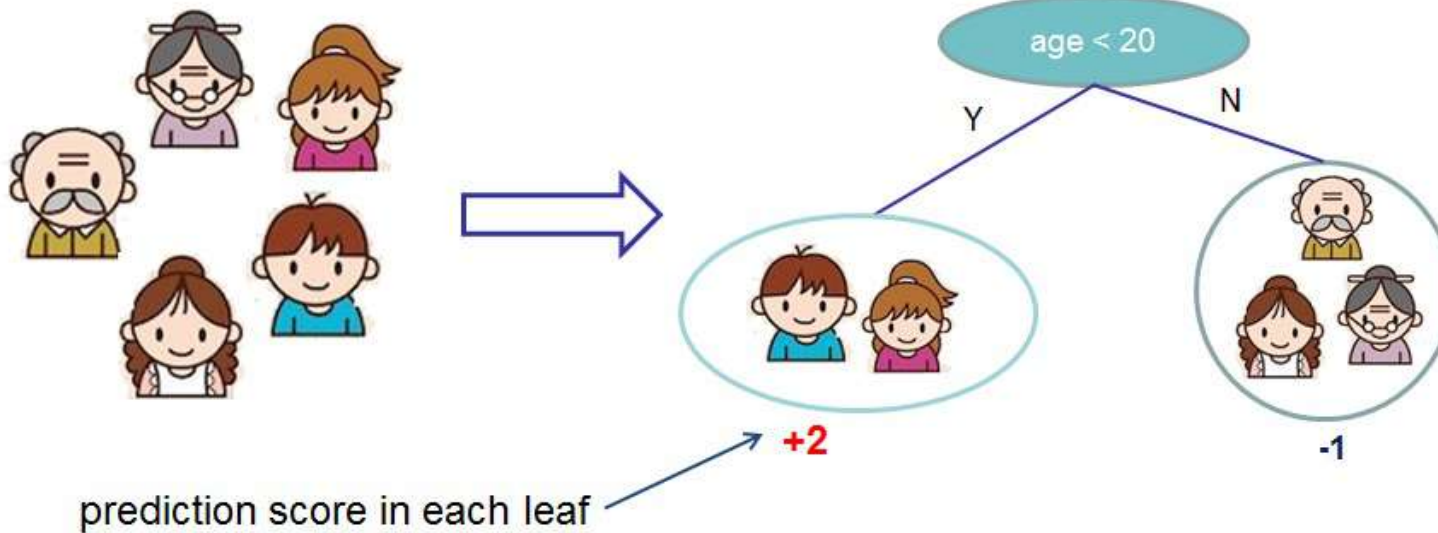


Intro to Gradient Boosting Tree

■ Decision Tree

Input: age, gender, occupation, ...

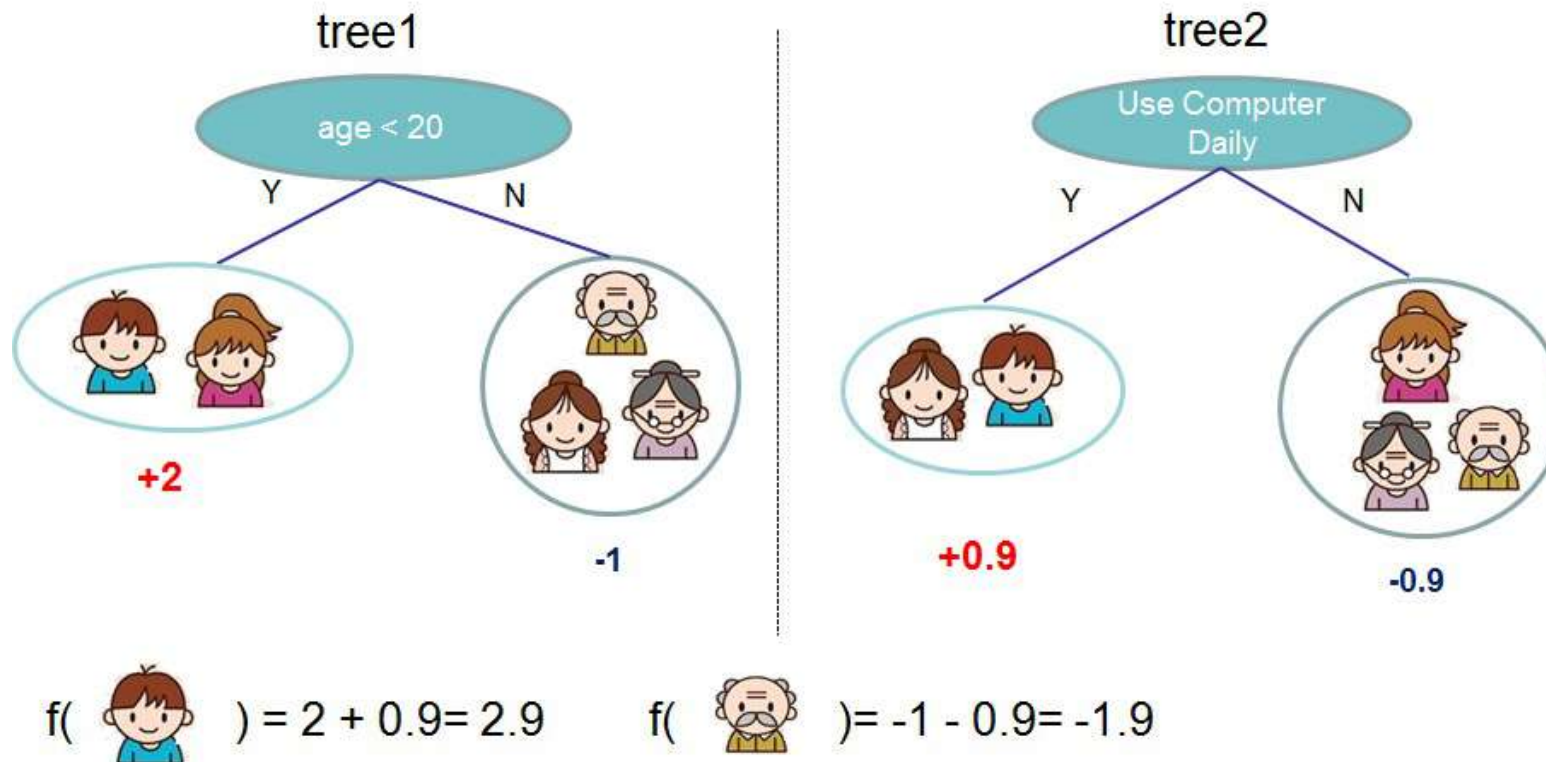
Like the computer game X





Intro to Gradient Boosting Tree (cont.)

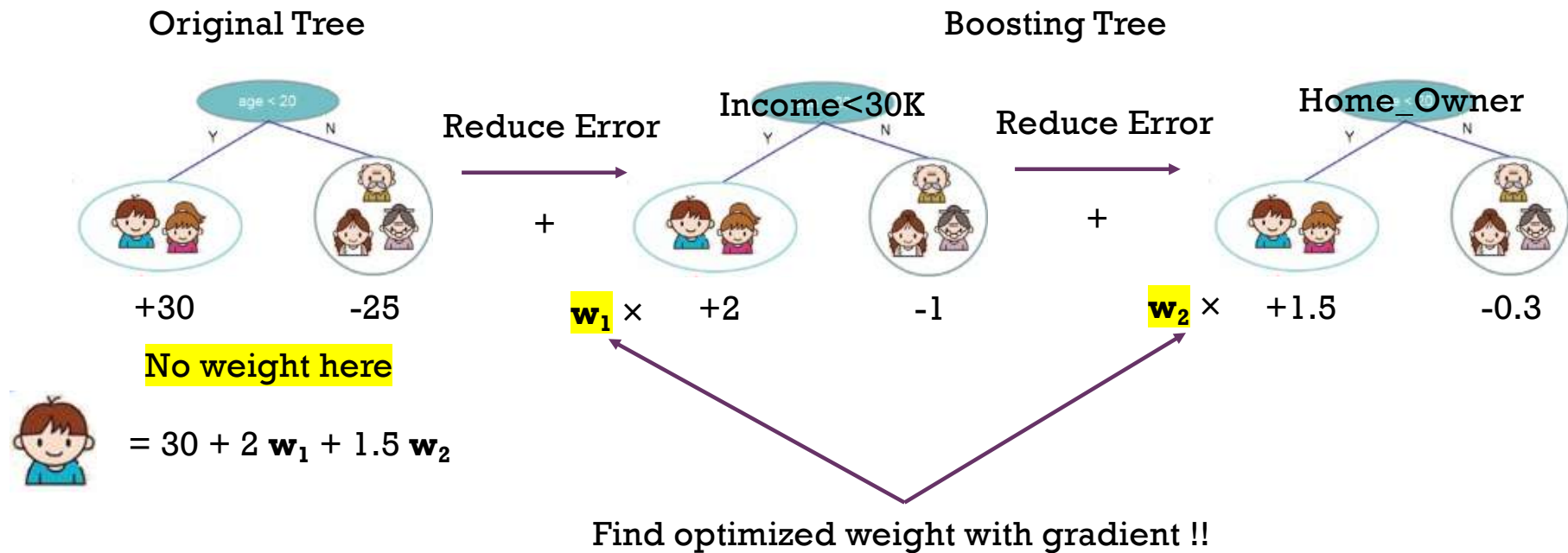
- Ensemble by weight on leaf nodes





Intro to Gradient Boosting Tree (cont.)

- Optimize weight of leaf nodes by gradient descent





Gradient Boosting Tree in sklearn

65

 [Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#)

[BaggingRegressor](#)
[ExtraTreesClassifier](#)
[ExtraTreesRegressor](#)
[GradientBoostingClassifier](#)
[GradientBoostingRegressor](#)
[HistGradientBoostingClassifier](#)
[HistGradientBoostingRegressor](#)
[IsolationForest](#)
[RandomForestClassifier](#)
[RandomForestRegressor](#)
[RandomTreesEmbedding](#)
[StackingClassifier](#)

[API Reference](#) > [sklearn.ensemble](#) > [GradientBoostingClassifier](#)

GradientBoostingClassifier

```
class sklearn.ensemble.GradientBoostingClassifier(*, loss='log_loss',
learning_rate=0.1, n_estimators=100, subsample=1.0,
criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0,
init=None, random_state=None, max_features=None, verbose=0,
max_leaf_nodes=None, warm_start=False, validation_fraction=0.1,
n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)
```

[\[source\]](#) [Examples](#) [Community](#) [More](#)

Gradient Boosting for classification.

[BaggingRegressor](#)
[ExtraTreesClassifier](#)
[ExtraTreesRegressor](#)
[GradientBoostingClassifier](#)
[GradientBoostingRegressor](#)
[HistGradientBoostingClassifier](#)
[HistGradientBoostingRegressor](#)
[IsolationForest](#)
[RandomForestClassifier](#)
[RandomForestRegressor](#)
[RandomTreesEmbedding](#)

[API Reference](#) > [sklearn.ensemble](#) > [GradientBoostingRegressor](#)

GradientBoostingRegressor

```
class sklearn.ensemble.GradientBoostingRegressor(*,
loss='squared_error', learning_rate=0.1, n_estimators=100, subsample=1.0,
criterion='friedman_mse', min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_depth=3, min_impurity_decrease=0.0,
init=None, random_state=None, max_features=None, alpha=0.9, verbose=0,
max_leaf_nodes=None, warm_start=False, validation_fraction=0.1,
n_iter_no_change=None, tol=0.0001, ccp_alpha=0.0)
```

[\[source\]](#)

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>



eXtreme Gradient Boosting Tree (XGBoost)

66

- XGBoost stands for “**Extreme** Gradient Boosting”, where the term “Gradient Boosting” originates from the paper *Greedy Function Approximation: A Gradient Boosting Machine*, by Friedman.
- XGBoost is exactly a tool motivated by the formal principle introduced in this tutorial! More importantly, it is developed with both deep consideration in terms of **systems optimization** and **principles in machine learning**. The goal of this library is to push the extreme of the computation limits of machines to provide **a scalable, portable and accurate library**

Aspect	Traditional GBDT	XGBoost
Tree building	Sequential	Sequential
In-tree parallelism	Limited	✓ Yes
Split strategy	Exact greedy	Histogram-based

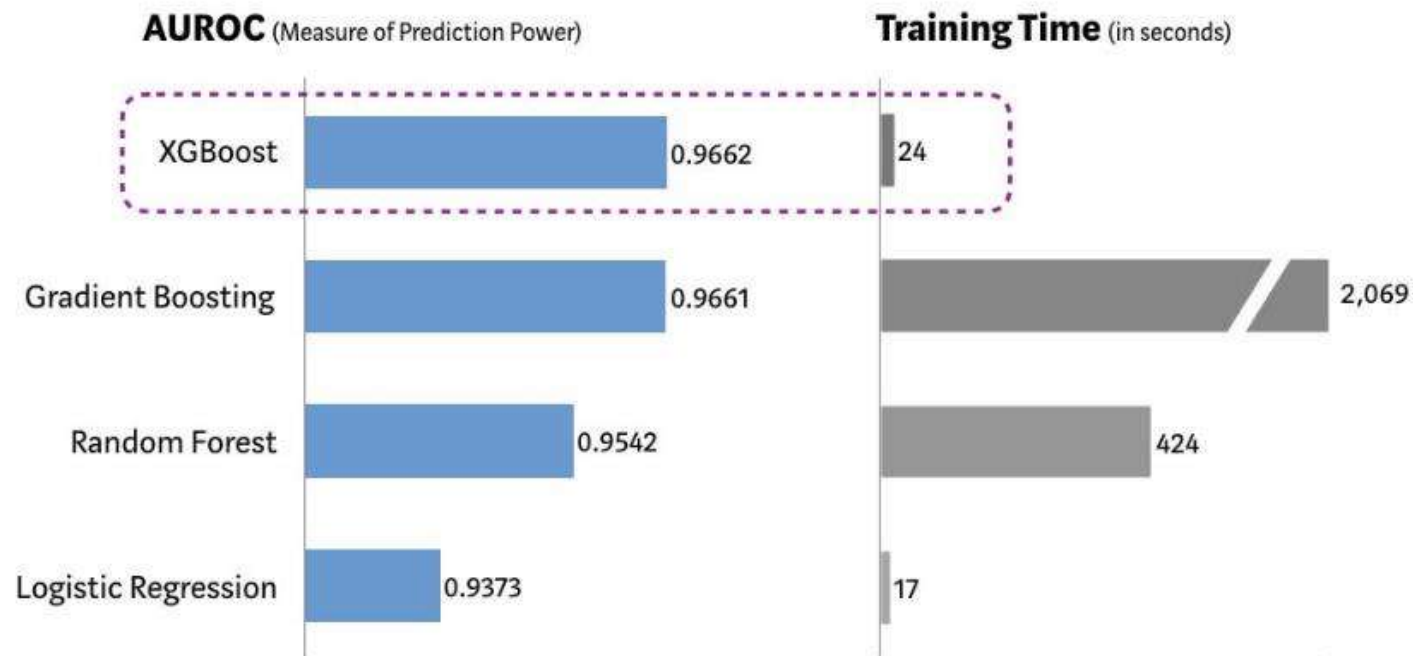


Models Performance

67

Performance Comparison using SKLearn's 'Make_Classification' Dataset

(5 Fold Cross Validation, 1MM randomly generated data sample, 20 features)





XGBoost Package

- Multiple Language Support 
- Distributed Model on Cloud
 - AWS
 - Kubernetes
 - Spark
 - Dask
- Compatible with sklearn interface

Python package

R package
JVM package
Ruby package
Swift package
Julia package
C Package



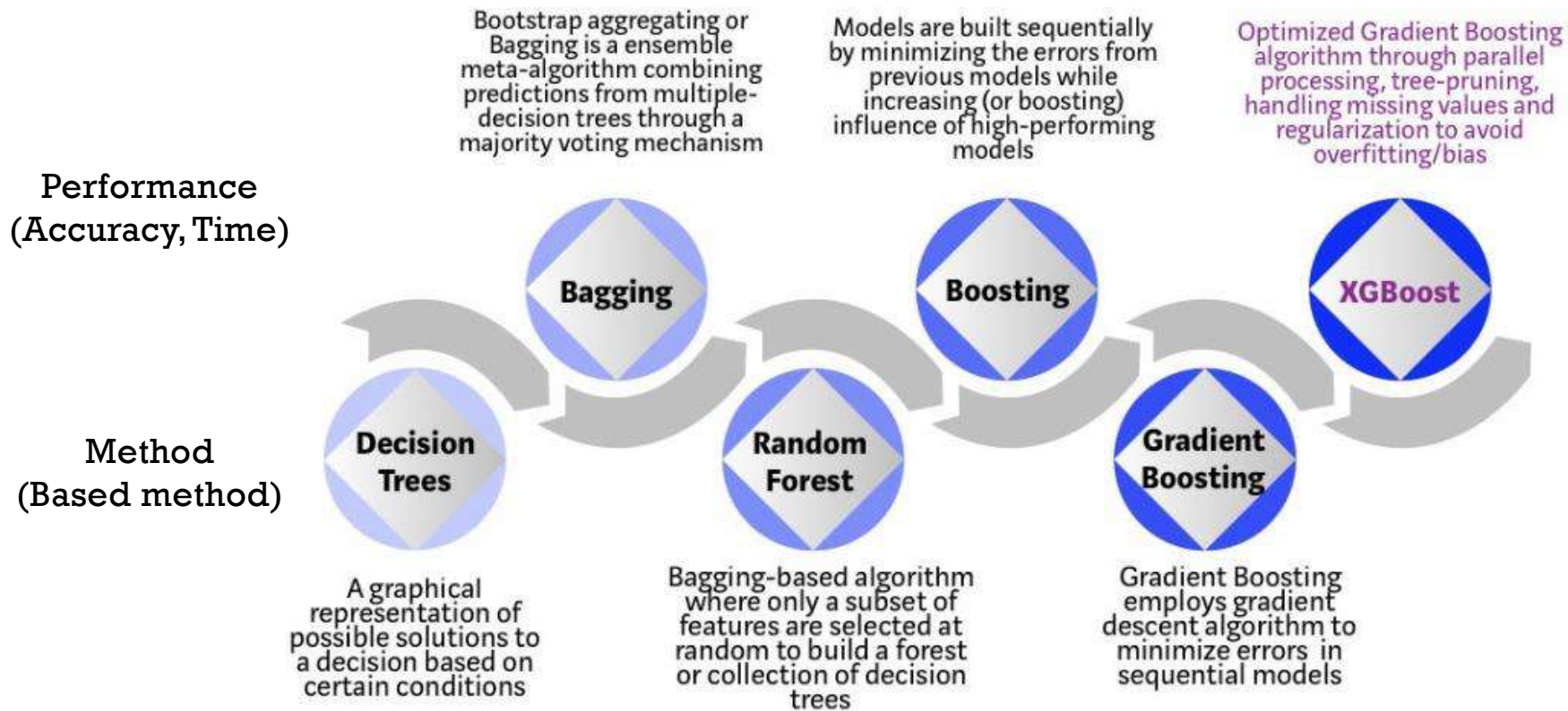
XGBoost Important Parameter



- `n_jobs` : Number of parallel tasks (-1 = all cores)
- `objective` : Type of Prediction
 - for Classification `'binary:hinge'`, `'binary:logistic'` (prob) or `'multi:softmax'` (multi-class)
 - for Regression `'reg:squarederror'`
- `learning_rate` : Step size used in update weights
- `n_estimators` : Number of trees
- `subsample` (0, 1]: Number of training data to growing trees, will prevent overfitting
- `importance_type` : Get feature importance of each feature.
 - `'weight'`: the number of times a feature is used to split the data across all trees.
 - `'gain'`: the average gain across all splits the feature is used in.
 - `'cover'`: the average coverage across all splits the feature is used in.
 - `'total_gain'`: the total gain across all splits the feature is used in.
 - `'total_cover'`: the total coverage across all splits the feature is used in.



Evolution of Decision Trees



Source: <https://towardsdatascience.com/https-medium-com-vishalmorde-xgboost-algorithm-long-she-may-rein-edd9f99be63d>



Note: Always set random_state in tree-based model

- **Randomness in Splitting Criteria:** During training, if multiple splits have the same quality (e.g., information gain, Gini impurity), the algorithm chooses one randomly. This randomness can cause variations in the resulting tree structure.

Without random_state:

python

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris

# Load dataset
data = load_iris()
X, y = data.data, data.target

# Train decision tree multiple times
model1 = DecisionTreeClassifier()
model2 = DecisionTreeClassifier()
model1.fit(X, y)
model2.fit(X, y)

# Check if results are the same
print(model1.feature_importances_)
print(model2.feature_importances_)
```

You might get **different feature importances** or splits each time you run the code.

With random_state:

python

```
model1 = DecisionTreeClassifier(random_state=42)
model2 = DecisionTreeClassifier(random_state=42)
model1.fit(X, y)
model2.fit(X, y)

# Check if results are the same
print(model1.feature_importances_)
print(model2.feature_importances_)
```

Now, the results are consistent.

sklearn.feature_selection

PrevUpNext

scikit-learn 1.2.1
Other versions

Please cite us if you use the software.

sklearn.feature_selection.SelectFromModel
SelectFromModel
Examples using
sklearn.feature_selection.SelectFromModel

InstallUser GuideAPIExamplesCommunityMore

Go

sklearn.feature_selection.SelectFromModel ¶

```
class sklearn.feature_selection.SelectFromModel(estimator, *, threshold=None, prefit=False, norm_order=1, max_features=None, importance_getter='auto')
```

[source]

Meta-transformer for selecting features based on importance weights.

New in version 0.17.

Read more in the [User Guide](#).

Parameters:	estimator : object The base estimator from which the transformer is built. This can be both a fitted (if <code>prefit</code> is set to <code>True</code>) or a non-fitted estimator. The estimator should have a <code>feature_importances_</code> or <code>coef_</code> attribute after fitting. Otherwise, the <code>importance_getter</code> parameter should be used.
	threshold : str or float, default=None The threshold value to use for feature selection. Features whose absolute importance value is greater or equal are kept while the others are discarded. If "median" (resp. "mean"), then the <code>threshold</code> value is the median (resp. the mean) of the feature importances. A scaling factor (e.g., "1.25*mean") may also be used. If <code>None</code> and if the estimator has a parameter <code>penalty</code> set to <code>l1</code> , either explicitly or implicitly (e.g, Lasso), the threshold used is <code>1e-5</code> . Otherwise, "mean" is used by default.
	prefit : bool, default=False Whether a prefit model is expected to be passed into the constructor directly or not. If <code>True</code> , <code>estimator</code>

Toggle Menu

https://scikit-learn.org/stable/modules/classes.html#module-sklearn.feature_selection

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectFromModel.html



```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_selection import SelectFromModel
import pandas as pd

# Load dataset
data = load_iris()
X = data.data # Features
y = data.target # Target
feature_names = data.feature_names

# Train a Decision Tree Classifier
tree_model = DecisionTreeClassifier(random_state=42)
tree_model.fit(X, y)

# Use SelectFromModel with threshold=0
selector = SelectFromModel(tree_model, prefit=True, threshold=0)

# Transform the dataset to keep only selected features
X_selected = selector.transform(X)

# Get selected feature names
selected_features = [feature_names[i] for i in selector.get_support(indices=True)]

# Display results
print("Selected Features (Threshold = 0):")
print(selected_features)

# Optionally, display feature importances for reference
feature_importances = pd.DataFrame({
    'Feature': feature_names,
    'Importance': tree_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

print("\nFeature Importances:")
print(feature_importances)

print("\nShape of Original Dataset:", X.shape)
print("Shape of Dataset After Feature Selection:", X_selected.shape)

```

threshold=0

Selected Features (Threshold = 0):
 ['sepal length (cm)', 'petal length (cm)', 'petal width (cm)']

Feature Importances:

	Feature	Importance
2	petal length (cm)	0.734006
3	petal width (cm)	0.239972
0	sepal length (cm)	0.026022
1	sepal width (cm)	0.000000

Shape of Original Dataset: (150, 4)
 Shape of Dataset After Feature Selection: (150, 3)



```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_selection import SelectFromModel
import pandas as pd

# Load the Iris dataset (replace this with your own dataset if needed)
data = load_iris()
X = data.data # Features
y = data.target # Target
feature_names = data.feature_names

# Train a Decision Tree Classifier
tree_model = DecisionTreeClassifier(random_state=42)
tree_model.fit(X, y)

# Extract feature importances
feature_importances = tree_model.feature_importances_

# Display feature importances for better understanding
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': feature_importances
}).sort_values(by='Importance', ascending=False)

print("Feature Importances:")
print(importance_df)

# Use SelectFromModel for feature selection
selector = SelectFromModel(tree_model, prefit=True, threshold="mean")

# Transform the dataset to keep only selected features
X_selected = selector.transform(X)

# Get selected feature names
selected_features = [feature_names[i] for i in selector.get_support(indices=True)]

# Output results
print("\nSelected Features:")
print(selected_features)

print("\nShape of Original Dataset:", X.shape)
print("Shape of Dataset After Feature Selection:", X_selected.shape)
```

Default threshold=mean

74

	Feature	Importance
2	petal length (cm)	0.734006
3	petal width (cm)	0.239972
0	sepal length (cm)	0.026022
1	sepal width (cm)	0.000000

Selected Features:
['petal length (cm)', 'petal width (cm)']

Shape of Original Dataset: (150, 4)
Shape of Dataset After Feature Selection: (150, 2)



```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_selection import SelectFromModel
import numpy as np
import pandas as pd

# Load the Iris dataset
data = load_iris()
X = data.data # Features
y = data.target # Target
feature_names = data.feature_names

# Train a Decision Tree Classifier
tree_model = DecisionTreeClassifier(random_state=42)
tree_model.fit(X, y)

# Use SelectFromModel with threshold=-np.inf to consider all features and limit to top 2 features
selector = SelectFromModel(estimator=tree_model, prefit=True, threshold=-np.inf, max_features=2)
X_selected = selector.transform(X)

# Get the names of the selected features
selected_features = [feature_names[i] for i in selector.get_support(indices=True)]

# Display the results
print("Selected Features (Top 2):")
print(selected_features)

# Optionally, show the feature importances
feature_importances = pd.DataFrame({
    'Feature': feature_names,
    'Importance': tree_model.feature_importances_
}).sort_values(by='Importance', ascending=False)

print("\nFeature Importances:")
print(feature_importances)

print("\nShape of Original Dataset:", X.shape)
print("Shape of Dataset After Feature Selection:", X_selected.shape)
```

max_features=2

threshold=-inf

Ensures that all features are considered for selection

Selected Features (Top 2):
['petal length (cm)', 'petal width (cm)']

Feature Importances:

	Feature	Importance
2	petal length (cm)	0.734006
3	petal width (cm)	0.239972
0	sepal length (cm)	0.026022
1	sepal width (cm)	0.000000

Shape of Original Dataset: (150, 4)

Shape of Dataset After Feature Selection: (150, 2)

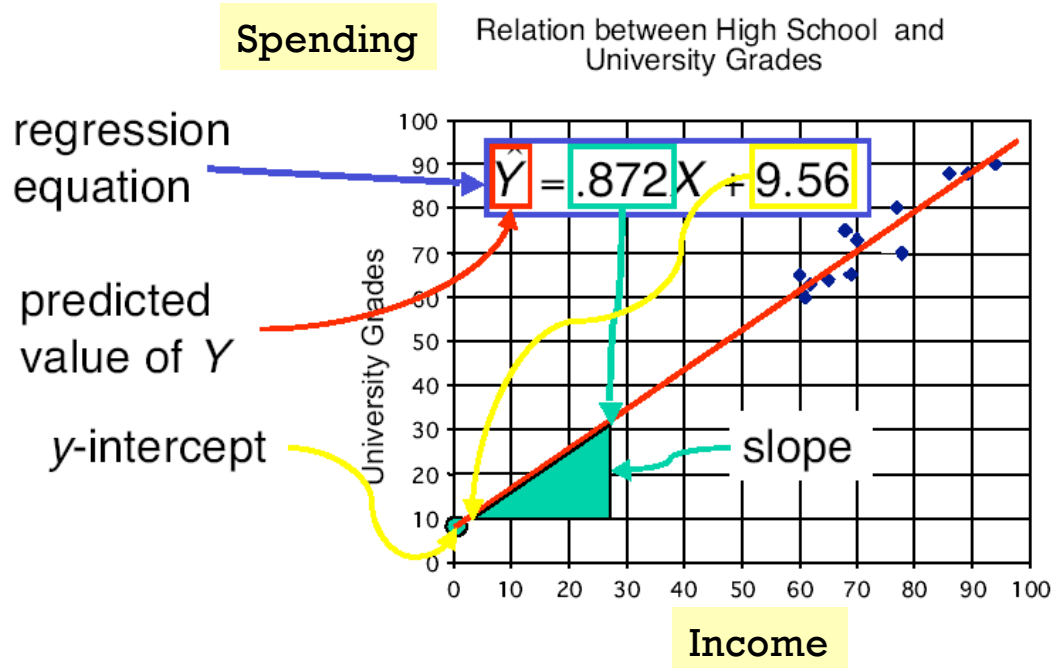


2) Regression



2) Regression

77



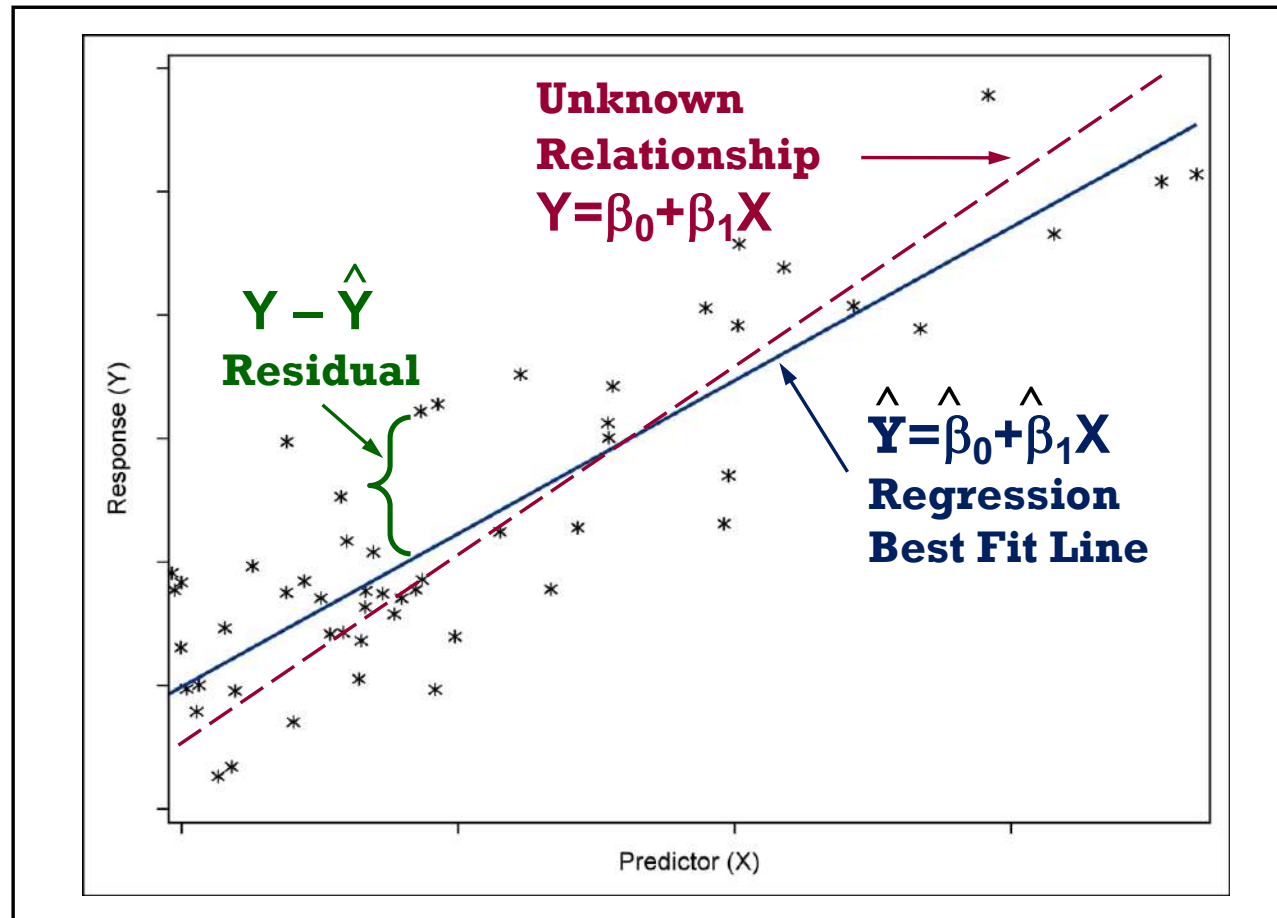
$$\hat{y} = \hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2$$

target weight, coefficient intercept input

- The least square method aims to minimize the following term

$$\sum_{\text{training data}} (y_i - \hat{y}_i)^2$$

Ordinary Least Squares (OLS) Regression



+ Multiple Linear Regression Model Matrix Multiplication Approach

	inputs	target
Age	Income in k\$	Spending
25	25	400
35	50	500
32	35	550

$$Y = \beta_0(1) + \beta_1 X_1 + \beta_2 X_2 + \varepsilon$$

$$[Y] = [X][\beta]$$

$$[\beta] = [X]^{-1}[Y]$$

General least-squares solution:

$$\beta = (X^T X)^{-1} X^T y$$

$$y = \begin{bmatrix} 400 \\ 500 \\ 550 \end{bmatrix} \quad X = \begin{bmatrix} 1 & 25 & 25 \\ 1 & 35 & 50 \\ 1 & 32 & 35 \end{bmatrix} \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

In this toy example, X is 3×3 and invertible, so it fits **exactly** and you can also use:

$$\beta = X^{-1}y$$

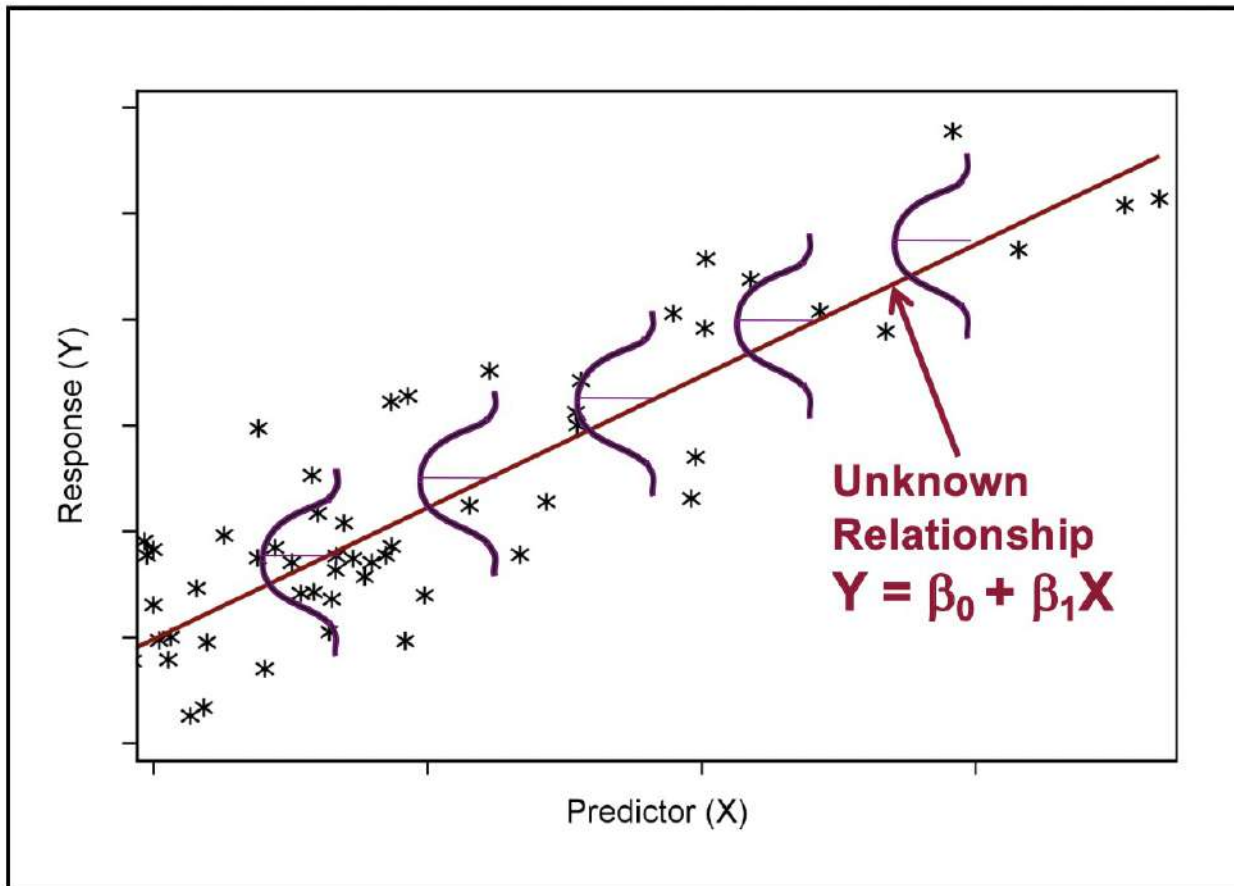
$$\beta = \begin{bmatrix} -250 \\ \frac{110}{3} \\ -\frac{32}{3} \end{bmatrix} \approx \begin{bmatrix} -250 \\ 36.6667 \\ -10.6667 \end{bmatrix}$$

$$\widehat{Spending} = -250 + 36.6667(\text{Age}) - 10.6667(\text{Income in k\$})$$



Linear Regression Assumption

$$\text{Spending} = 500 + 10 * \text{Income10K} + 2 * \text{Age}$$



- Linear relationship between (y, x_i) .
[Pearson correlation]
- Error is normal distributed. [remove outliers, log-transformation]
- Error has equal variance (homoscedasticity) [remove outliers, log-transformation]
- Errors are independent from each other. [design new data correction]



Prev Up Next

scikit-learn 0.22.1

[Other versions](#)Please [cite us](#) if you use the software.[sklearn.linear_model.LinearRegression](#)

Examples using

[sklearn.linear_model.LinearRegression](#)

sklearn.linear_model.LinearRegression

```
class sklearn.linear_model.LinearRegression(fit_intercept=True, normalize=False, copy_X=True, n_jobs=None)
```

[\[source\]](#)

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients $w = (w_1, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

Parameters:

fit_intercept : bool, optional, default True

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

normalize : bool, optional, default False

This parameter is ignored when `fit_intercept` is set to False. If True, the regressors X will be normalized before regression by subtracting the mean and dividing by the l2-norm. If you wish to standardize, please use `StandardScaler` before calling `fit` on an estimator with `normalize=False`.

[sklearn.preprocessing.StandardScaler](#)**normalize : bool, default=False**

This parameter is ignored when `fit_intercept` is set to False. If True, the regressors X will be normalized before regression by subtracting the mean and dividing by the l2-norm. If you wish to standardize, please use `StandardScaler` before calling `fit` on an estimator with `normalize=False`.

Deprecated since version 1.0: `normalize` was deprecated in version 1.0 and will be removed in 1.2.

Attributes::

coef_ : array of shape (n_features,) or (n_targets, n_features)

Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D),

intercept_ : float or array of shape (n_targets,)

Independent term in the linear model. Set to 0.0 if `fit_intercept = False`.

n_features_in_ : int

Number of features seen during `fit`.

New in version 0.24.

feature_names_in_ : ndarray of shape (n_features_in_,)

Names of features seen during `fit`. Defined only when X has feature names that are all strings.

New in version 1.0.

only one target is passed, this is a 1D array of

one means 1 unless in a `joblib.parallel_backend` context. `-1` means using all processors. For more details.

for $n_targets > 1$ and sufficient



Regression with Regularization

82

■ Lasso Regression (L1 Regularization)

Lasso Regression (Least Absolute Shrinkage and Selection Operator) adds “*absolute value of magnitude*” of coefficient as penalty term to the loss function.

$$\sum_{i=1}^n (Y_i - \sum_{j=1}^p X_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

Cost function

■ Ridge Regression (L2 Regularization)

Ridge regression adds “*squared magnitude*” of coefficient as penalty term to the loss function. Here the *highlighted* part represents L2 regularization element.

$$\sum_{i=1}^n (y_i - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Cost function

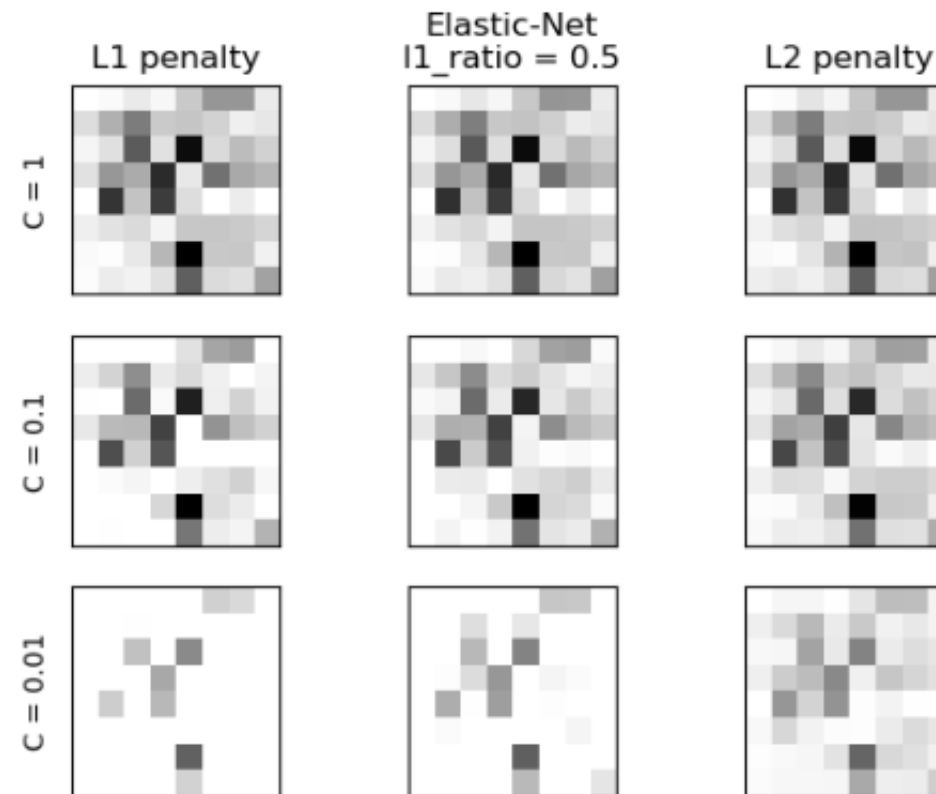
The function of both the regularization methods are almost the same. The **key difference** between these techniques is that Lasso shrinks the less important feature's coefficient to zero thus, removing some feature altogether. So, this works well for **feature selection** in case we have a huge number of features.

<https://towardsdatascience.com/l1-and-l2-regularization-methods-ce25e7fc831c>

ElasticNet

`l1_ratio : float, default=0.5`

The ElasticNet mixing parameter, with $0 \leq \text{l1_ratio} \leq 1$. For $\text{l1_ratio} = 0$ the penalty is an L2 penalty. For $\text{l1_ratio} = 1$ it is an L1 penalty. For $0 < \text{l1_ratio} < 1$, the penalty is a combination of L1 and L2.



[https://scikit-learn.org/stable/auto_examples/linear_model/plot_logistic_l1_l2_sparsity.html#:~:text=Comparison%20of%20the%20sparsity%20\(percentage,C%20constrain%20the%20model%20more](https://scikit-learn.org/stable/auto_examples/linear_model/plot_logistic_l1_l2_sparsity.html#:~:text=Comparison%20of%20the%20sparsity%20(percentage,C%20constrain%20the%20model%20more).

See also:

Ridge

Ridge regression addresses some of the problems of Ordinary Least Squares by imposing a penalty on the size of the coefficients with l2 regularization.

Lasso

The Lasso is a linear model that estimates sparse coefficients with l1 regularization.

ElasticNet

Elastic-Net is a linear regression model trained with both l1 and l2 -norm regularization of the coefficients.

sklearn.linear_model.Ridge

```
class sklearn.linear_model.Ridge(alpha=1.0, *, fit_intercept=True, max_iter=None, tol=0.001, solver='auto', positive=False, random_state=None)
```

Linear least squares with l2 regularization.

Minimizes the objective function:

$$||y - Xw||^2_2 + \alpha ||w||^2_2$$

This model solves a regression model where the loss function is the l2-norm. Also known as Ridge Regression or Tikhonov regression (i.e., when y is a 2d-array of shape $(n_{\text{samples}}, n_{\text{targets}})$).

Read more in the [User Guide](#).

Parameters:

- alpha** : *float, ndarray of shape (n_targets, n_targets)*
Constant that multiplies the L2 term, controlling regularization strength. `alpha` must be a float i.e. in $[0, \infty)$.

When `alpha = 0`, the objective is equivalent to the ordinary least squares, solved by the `LinearRegression` object. For numerical reasons, using `alpha` with the `LinearRegression` object is not advised.

If an array is passed, penalties are assumed to be applied to each target separately.

fit_intercept : *bool, default=True*
Whether to calculate the intercept for this model. If set to False, no intercept will be calculated (i.e. data is expected to be centered).
- normalize** : *bool, default=False*
This parameter is ignored when `fit_intercept` is set to False. If True, the regressor before regression by subtracting the mean and dividing by the l2-norm. If you wish to use `StandardScaler` before calling `fit` on an estimator with `normalize=False`.

sklearn.linear_model.Lasso

```
class sklearn.linear_model.Lasso(alpha=1.0, *, fit_intercept=True, normalize='deprecated', precompute=None, copy_X=True, max_iter=1000, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

Linear Model trained with L1 prior as regularizer (aka the Lasso).

The optimization objective for Lasso is:

$$(1 / (2 * n_{\text{samples}})) * ||y - Xw||^2_2 + \alpha ||w||_1$$

Technically the Lasso model is optimizing the same objective function as the Elastic Net with `l1_ratio=1`.

Read more in the [User Guide](#).

Parameters:

- alpha** : *float, default=1.0*
Constant that multiplies the L1 term, controlling regularization strength. `alpha` must be a float i.e. in $[0, \infty)$.

When `alpha = 0`, the objective is equivalent to ordinary least squares, solved by the `LinearRegression` object. For numerical reasons, using `alpha = 0` with the `Lasso` object is not advised, use the `LinearRegression` object.
- fit_intercept** : *bool, default=True*
Whether to calculate the intercept for this model. If set to False, no intercept will be calculated (i.e. data is expected to be centered).
- normalize** : *bool, default=False*
This parameter is ignored when `fit_intercept` is set to False. If True, the regressor before regression by subtracting the mean and dividing by the l2-norm. If you wish to use `StandardScaler` before calling `fit` on an estimator with `normalize=False`.

Deprecated since version 1.0: `normalize` was deprecated in version 1.0 and will be removed in version 1.2.

sklearn.linear_model.ElasticNet

```
class sklearn.linear_model.ElasticNet(alpha=1.0, *, l1_ratio=0.5, fit_intercept=True, normalize='deprecated', precompute=None, max_iter=1000, copy_X=True, tol=0.0001, warm_start=False, positive=False, random_state=None, selection='cyclic')
```

[\[source\]](#)

Linear regression with combined L1 and L2 priors as regularizer.

Minimizes the objective function:

$$(1 / (2 * n_{\text{samples}})) * ||y - Xw||^2_2 + \alpha * l1_ratio * ||w||_1 + 0.5 * \alpha * (1 - l1_ratio) * ||w||^2_2$$

If you are interested in controlling the L1 and L2 penalty separately, keep in mind that this is equivalent to:

$$a * ||w||_1 + 0.5 * b * ||w||^2_2$$

where:

$$\alpha = a + b \text{ and } l1_ratio = a / (a + b)$$

The parameter `l1_ratio` corresponds to `alpha` in the `glmnet` R package while `alpha` corresponds to the `lambda` parameter in `glmnet`. Specifically, `l1_ratio = 1` is the lasso penalty. Currently, `l1_ratio <= 0.01` is not reliable, unless you supply your own sequence of `alpha`.

Read more in the [User Guide](#).

Parameters:

- alpha** : *float, default=1.0*
Constant that multiplies the penalty terms. Defaults to 1.0. See the notes for the exact mathematical meaning of this parameter. `alpha = 0` is equivalent to an ordinary least square, solved by the `LinearRegression` object. For numerical reasons, using `alpha = 0` with the `Lasso` object is not advised. Given this, you should use the `LinearRegression` object.
- l1_ratio** : *float, default=0.5*
The ElasticNet mixing parameter, with $0 \leq l1_ratio \leq 1$. For `l1_ratio = 0` the penalty is an L2 penalty. For `l1_ratio = 1` it is an L1 penalty. For $0 < l1_ratio < 1$, the penalty is a combination of L1 and L2.
- fit_intercept** : *bool, default=True*
Whether the intercept should be estimated or not. If `False`, the data is assumed to be already centered.
- normalize** : *bool, default=False*

Adjust input ranges

StandardScaler

```
class sklearn.preprocessing.StandardScaler(*, copy=True, with_mean=True,
with_std=True) \[source\]
```

Standardize features by removing the mean and scaling to unit variance.

The standard score of a sample x is calculated as:

$$z = (x - \mu) / s$$

where μ is the mean of the training samples or zero if `with_mean=False`, and s is the standard deviation of the training samples or one if `with_std=False`.

Centering and scaling happen independently on each feature by computing the relevant statistics on the samples in the using [transform](#)

```
>>> from sklearn.preprocessing import StandardScaler
>>> data = [[0, 0], [0, 0], [1, 1], [1, 1]]
>>> scaler = StandardScaler()
>>> print(scaler.fit(data))
StandardScaler()
>>> print(scaler.mean_)
[0.5 0.5]
>>> print(scaler.transform(data))
[[-1. -1.]
 [-1. -1.]
 [ 1.  1.]
 [ 1.  1.]]
>>> print(scaler.transform([[2, 2]]))
[[3. 3.]]
```

<https://scikit-learn.org/1.6/modules/generated/sklearn.preprocessing.StandardScaler.html>

MinMaxScaler

```
class sklearn.preprocessing.MinMaxScaler(feature_range=(0, 1), *, copy=True,
clip=False) \[source\]
```

Transform features by scaling each feature to a given range.

This estimator scales and translates each feature individually such that it is in the given range on the training set, e.g. between zero and one.

The transformation is given by:

$$X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))$$
$$X_scaled = X_std * (max - min) + min$$

where min, max = f

This transformation

`MinMaxScaler` doe

range, where the la

one corresponds to

[with other scalers.](#)

Read more in the [U](#)

```
>>> from sklearn.preprocessing import MinMaxScaler
>>> data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
>>> scaler = MinMaxScaler()
>>> print(scaler.fit(data))
MinMaxScaler()
>>> print(scaler.data_max_)
[ 1. 18.]
>>> print(scaler.transform(data))
[[0.  0. ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.   1.  ]]
>>> print(scaler.transform([[2, 2]]))
[[1.5 0.  ]]
```

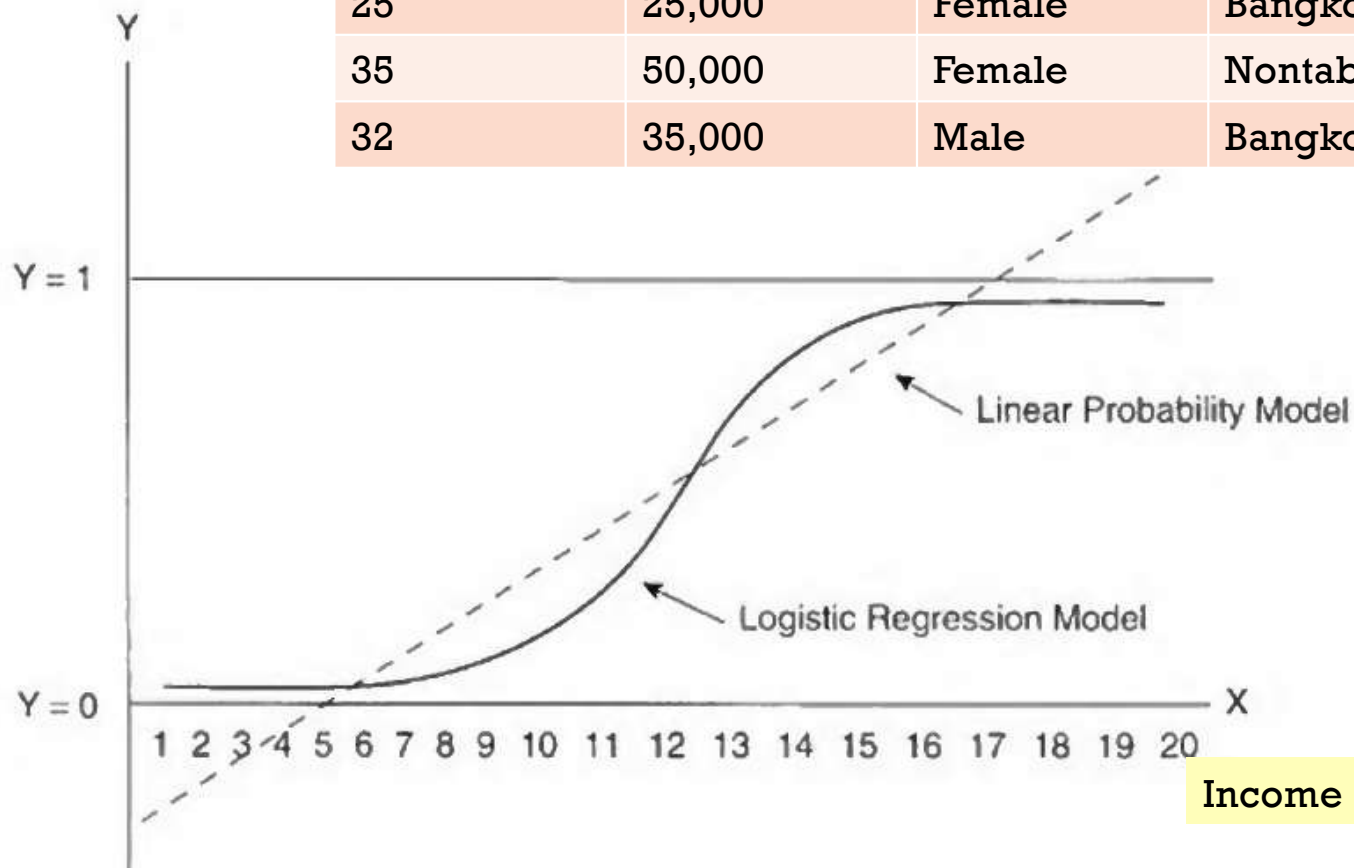
<https://scikit-learn.org/1.6/modules/generated/sklearn.preprocessing.MinMaxScaler.html>



Classification Problem

86

Prob. of purchase	Age	Income	Gender	Province	Purchase
	25	25,000	Female	Bangkok	Yes (1)
	35	50,000	Female	Nontaburi	Yes (1)
	32	35,000	Male	Bangkok	No (0)

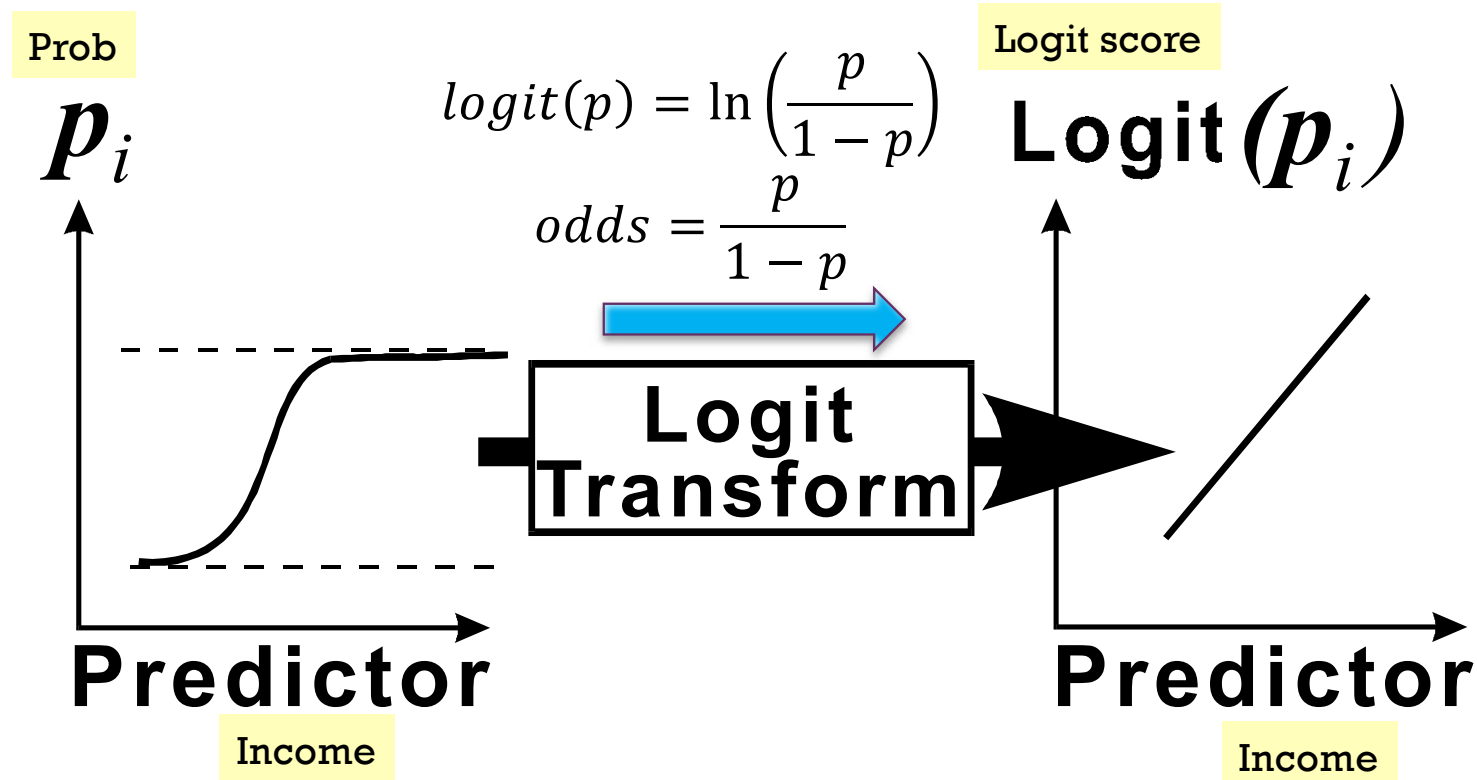


Logistic function
Sigmoid function



Logistic Regression (forward pass)

Logit link function: Non-linear to Linear



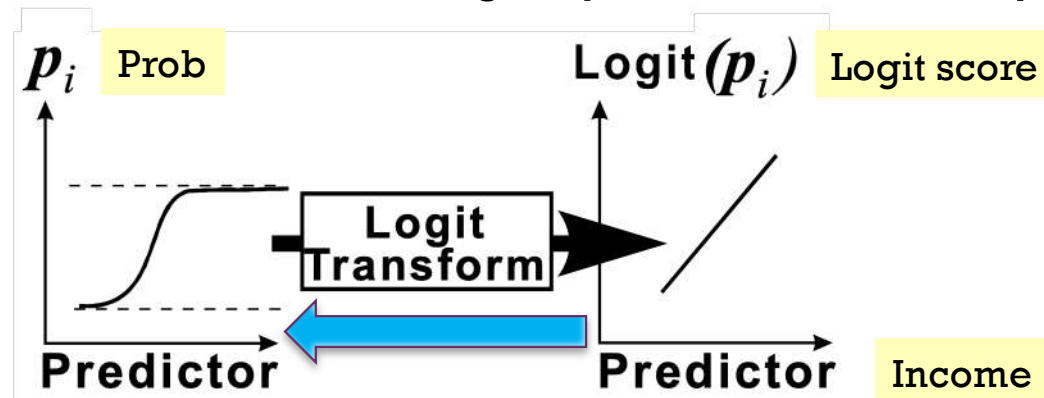
+ Logit Link Function (backward pass)

$$\log \left(\frac{\hat{p}}{1 - \hat{p}} \right) = \hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2 = \text{logit}(\hat{p})$$

■ Maximum likelihood estimates

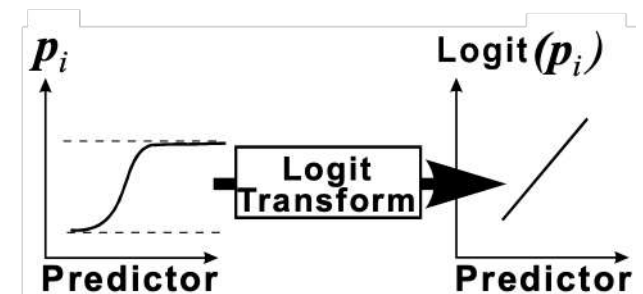
$$\hat{p} = \frac{1}{1 + e^{-\text{logit}(\hat{p})}}$$

To obtain prediction estimates, the logit equation is solved for $\hat{p}$.



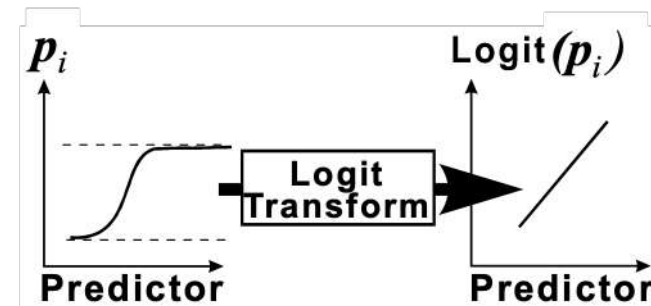
Assumptions for Logistic Regression

- The mean of the logit is accurately modeled by a linear function of the X_i .
 - (logit, x_i) = linear relationship
- ~~■ The random error term, ϵ , is assumed to have a normal distribution with a mean of zero.~~
- ~~■ The random error term, ϵ , is assumed to have a constant variance, σ^2 .~~
 - ~~■ Not skew~~
- The errors are independent.





Quiz



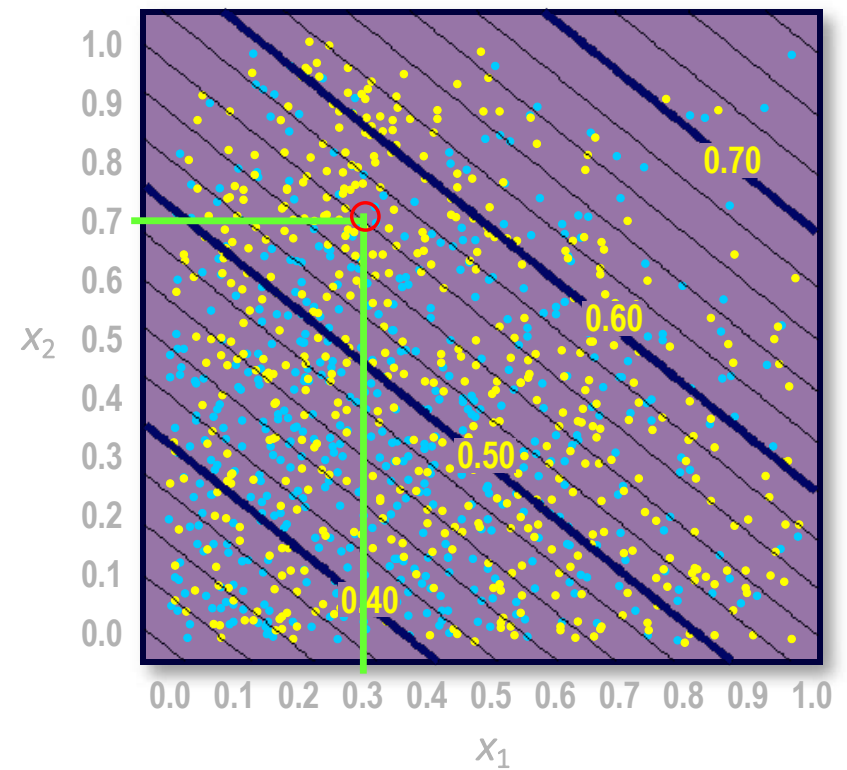
90

■ What is the logistic regression prediction for the indicated point?

- a. 0.243
- b. 0.56
- c. yellow
- d. It depends.

$$\text{logit}(\hat{p}) = -0.81 + 0.92x_1 + 1.11x_2$$

$$\hat{p} = \frac{1}{1 + e^{-\text{logit}(\hat{p})}}$$



sklearn.linear_model.LogisticRegression

```
class sklearn.linear_model.LogisticRegression(penalty='l2', dual=False, tol=0.0001, C=1.0, fit_intercept=True,
intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0,
warm_start=False, n_jobs=None, l1_ratio=None) ¶
```

[\[source\]](#)

Logistic Regression (aka logit, MaxEnt) classifier.

In the multiclass case, the training algorithm uses the one-vs-rest (OvR) scheme if the 'multi_class' option is set to 'ovr', and uses the cross-entropy loss if the 'multi_class' option is set to 'multinomial'. Note that the original implementation (using the 'lbfgs' solver) supports the 'sag', 'saga' and 'newton-cg' solvers.)

This class implements regularized logistic regression using the specified solver. **regularization is applied by default.** It can handle both dense and sparse data in the form of numpy arrays or scipy.sparse matrices; for optimal performance, use 64-bit floats for optimal performance; any other input format will be converted to 64-bit floats.

The 'newton-cg', 'sag', and 'lbfgs' solvers support only L2 regularization. The 'saga' solver supports both L1 and L2 regularization, with a dual formulation. The 'saga' solver also supports the 'l1_ratio' parameter, which allows the user to specify the proportion of L1 and L2 regularization to be used. The 'saga' solver is the only solver that supports the 'l1_ratio' parameter.

Read more in the [User Guide](#).

Examples

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.8...e-01, 1.8...e-02, 1.4...e-08],
       [9.7...e-01, 2.8...e-02, ...e-08]])
>>> clf.score(X, y)
0.97...
```

Parameters:

penalty : {'l1', 'l2', 'elasticnet', 'none'}, default='l2'

Used to specify the norm used in the penalization. The 'newton-cg', 'sag' and 'lbfgs' solvers support only l2 penalties. 'elasticnet' is only supported by the 'saga' solver. If 'none' (not supported by the liblinear solver), no regularization is applied.

New in version 0.19: l1 penalty with SAGA solver (allowing 'multinomial' + L1)

l1_ratio : float, default=None

The Elastic-Net mixing parameter, with $0 \leq \text{l1_ratio} \leq 1$. Only used if `penalty='elasticnet'`. Setting `l1_ratio=0` is equivalent to using `penalty='l2'`, while setting `l1_ratio=1` is equivalent to using `penalty='l1'`. For $0 < \text{l1_ratio} < 1$, the penalty is a combination of L1 and L2.

C : float, default=1.0

Inverse of regularization strength; must be a positive float. Like in support vector machines, smaller values specify stronger regularization.

Here, we are going to check how sparsity increases as we increase lambda (or decrease C, as $C = 1/\lambda$) when **L1** Regularizer is used.

```
from sklearn.linear_model import LogisticRegression
```

```
clf = LogisticRegression(C= 1000, penalty= 'l1')
clf.fit(x_train,y_train)
pred = clf.predict(x_test)
print("Non Zero weights:",np.count_nonzero(clf.coef_))
```

Non Zero weights: 50470

```
clf = LogisticRegression(C= 1, penalty= 'l1')
clf.fit(x_train,y_train)
pred = clf.predict(x_test)
print("Non Zero weights:",np.count_nonzero(clf.coef_))
```

Non Zero weights: 8228

```
clf = LogisticRegression(C= 0.01, penalty= 'l1')
clf.fit(x_train,y_train)
pred = clf.predict(x_test)
print("Non Zero weights:",np.count_nonzero(clf.coef_))
```

Non Zero weights:396

<https://medium.com/@aditya97p/l1-and-l2-regularization-237438a9caa6>

class_weight : dict or 'balanced', default=None

Weights associated with classes in the form {class_label: weight}. If not given, all classes are supposed to have weight one.

The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as $n_{\text{samples}} / (n_{\text{classes}} * \text{np.bincount}(y))$.

Note that these weights will be multiplied with sample_weight (passed through the fit method) if sample_weight is specified.

New in version 0.17: class_weight='balanced'

random_state : int, RandomState instance, default=None

Used when solver == 'sag', 'saga' or 'liblinear' to shuffle the data. See [Glossary](#) for details.

Examples

```
>>> np.bincount(np.arange(5))
array([1, 1, 1, 1, 1])
>>> np.bincount(np.array([0, 1, 1, 3, 2, 1, 7]))
array([1, 3, 1, 1, 0, 0, 0, 1])
```

- 100 (20:80)
- $W_{c1} = 100 / (2 * 20) = 2.5$
- $W_{c2} = 100 / (2 * 80) = 0.625$



Linear Regression Limitation

94

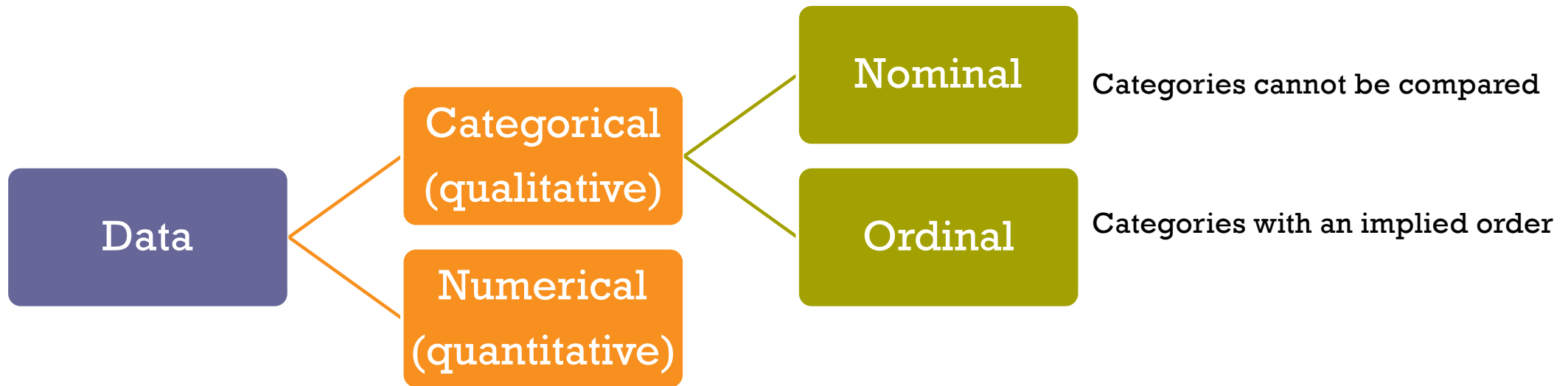
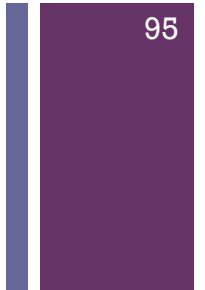
- Manage missing value
- Handle outliers (skewness)
- Handle categorical variables (dummy codes)
- Variable selection:
 - Forward, Backward, Stepwise (**not exist in scikit-learn**)
- Accounted for nonlinearities



Important Params:

- Require good data preparation
- Variable selection

+ Terminology: Kinds of data (recap)





Ordinal: Recode

Grade	GradeNum
A	4.00
B+	3.50
B	3.00
C+	2.50
C	2.00
D+	1.50
D	1.00
F	0.00

Size	SizeNum
XL	5
L	4
M	3
S	2
XS	1



Categorical

- Dummy coding = (n-1) dummy codes

Branch	BranchNum	D_BKK	B_Patum	D_Non	D_BKK	B_Patum	
BKK	1	1	0	0	1	0	
Patumtani	2	0	1	0	0	1	
Nontaburi	3	0	0	1	0	0	reference

[Input/output](#)
[General functions](#)
[Series](#)
[DataFrame](#)
[Pandas arrays](#)
[Panel](#)
[Index objects](#)
[Date offsets](#)
[Window](#)
[GroupBy](#)
[Resampling](#)
[Style](#)

pandas.get_dummies

pandas.get_dummies(data, prefix=None, prefix_sep='_', dummy_na=False, columns=None, sparse=False, drop_first=False, dtype=None) → 'DataFrame' [\[source\]](#)

Convert categorical variable into dummy/indicator variables.

Parameters: **data** : *array-like, Series, or DataFrame*

Data of which to get dummy indicators.

prefix : *str, list of str, or dict of str, default None*

String to append DataFrame column names. Pass a list with length equal to the number of columns when calling get_dummies on a DataFrame. Alternatively, *prefix* can be a dictionary mapping column names to prefixes.

prefix_sep : *str, default '_'*

If appending prefix, separator/delimiter to use. Or pass a list or dictionary as with *prefix*.

dummy_na : *bool, default False*

Add a column to indicate NaNs, if False NaNs are ignored.

columns : *list-like, default None*

Columns in the DataFrame to be encoded. If *columns* is None then all the columns with *object* or *category* are converted.

sparse : *bool, default False*

dummy-encoded columns should be backed by a **SparseArray** (True) or a regular NumPy array (False).

```
>>> pd.get_dummies(pd.Series(list('abcaa')), drop_first=True)
   b  c
0  0  0
1  1  0
2  0  1
3  0  0
4  0  0
```

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.get_dummies.html

sklearn.preprocessing.OneHotEncoder

```
class sklearn.preprocessing.OneHotEncoder(*, categories='auto', drop=None, sparse='deprecated',
sparse_output=True, dtype=<class 'numpy.float64'>, handle_unknown='error', min_frequency=None, max_categories=None)
```

[\[source\]](#)

Encode categorical features as a one-hot numeric array.

The input to this transformer should be an array-like of integers or strings, denoting the values taken on by categorical (discrete) features. The features are encoded using a one-hot (aka 'one-of-K' or 'dummy') encoding scheme. This creates a binary column for each category and returns a sparse matrix or dense array (depending on the `sparse_output` parameter).

By default, the encoder derives the categories based on the unique values in each feature. Alternatively, the categories manually.

This encoding is needed for feeding categorical data to many scikit-learn estimators, notably linear models and standard kernels.

Note: a one-hot encoding of y labels should use a `LabelBinarizer` instead.

drop : {'first', 'if_binary'} or an array-like of shape (n_features,), default=None

Specifies a methodology to use to drop one of the categories per feature. This is useful in situations where perfectly collinear features cause problems, such as when feeding the resulting data into an unregularized linear regression model.

However, dropping one category breaks the symmetry of the original representation and can therefore induce a bias in downstream models, for instance for penalized linear classification or regression models.

- `None` : retain all features (the default).
- `'first'` : drop the first category in each feature. If only one category is present, the feature will be dropped entirely.
- `'if_binary'` : drop the first category in each feature with two categories. Features with 1 or more than 2 categories are left intact.
- `array : drop[i]` is the category in feature `X[:, i]` that should be dropped.

New in version 0.21: The parameter `drop` was added in 0.21.

Changed in version 0.23: The option `drop='if_binary'` was added in 0.23.

Changed in version 1.1: Support for dropping infrequent categories.

```
>>> X = [['Male', 1], ['Female', 3], ['Female', 2]]
```

One can always drop the first column for each feature:

```
>>> drop_enc = OneHotEncoder(drop='first').fit(X)
>>> drop_enc.categories_
[array(['Female', 'Male'], dtype=object), array([1, 2, 3], dtype=object)]
>>> drop_enc.transform([['Female', 1], ['Male', 2]]).toarray()
array([[0., 0., 0.],
       [1., 1., 0.]])
```

Or drop a column for feature only having 2 categories:

```
>>> drop_binary_enc = OneHotEncoder(drop='if_binary').fit(X)
>>> drop_binary_enc.transform([['Female', 1], ['Male', 2]]).toarray()
array([[0., 1., 0., 0.],
       [1., 0., 1., 0.]])
```

sklearn.feature_selection

scikit-learn

Install

User Guide

API

Examples

Community

More ▾

Prev

Up

Next

scikit-learn 1.2.1

Other versions

Please cite us if you use the software.

sklearn.feature_selection.SelectFromModel

SelectFromModel

Examples using sklearn.feature_selection.SelectFromModel

Toggle Menu

https://scikit-learn.org/stable/modules/classes.html#module-sklearn.feature_selection

Go

00

sklearn.feature_selection.SelectFromModel

Recap

```
class sklearn.feature_selection.SelectFromModel(estimator, *, threshold=None, prefit=False, norm_order=1, max_features=None, importance_getter='auto')
```

[source]

Meta-transformer for selecting features based on importance weights.

New in version 0.17.

Read more in the [User Guide](#).

Parameters:

estimator : object

The base estimator from which the transformer is built. This can be both a fitted (if `prefit` is set to `True`) or a non-fitted estimator. The estimator should have a `feature_importances_` or `coef_` attribute after fitting. Otherwise, the `importance_getter` parameter should be used.

threshold : str or float, default=None

The threshold value to use for feature selection. Features whose absolute importance value is greater or equal are kept while the others are discarded. If "median" (resp. "mean"), then the `threshold` value is the median (resp. the mean) of the feature importances. A scaling factor (e.g., "1.25*mean") may also be used. If `None` and if the estimator has a parameter penalty set to `l1`, either explicitly or implicitly (e.g, Lasso), the threshold used is `1e-5`. Otherwise, "mean" is used by default.

prefit : bool, default=False

Whether a prefit model is expected to be passed into the constructor directly or not. If `True`, estimator

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectFromModel.html

Negative coefficients are handled by their absolute values

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Lasso
from sklearn.feature_selection import SelectFromModel
import pandas as pd
import numpy as np

# Load dataset
data = load_diabetes()
X = data.data # Features
y = data.target # Target
feature_names = data.feature_names

# Train a Lasso Regression Model
lasso = Lasso(alpha=0.1, random_state=42)
lasso.fit(X, y)

# Use SelectFromModel with threshold=0 to select all non-zero absolute coefficient features
selector = SelectFromModel(estimator=lasso, prefit=True, threshold=0)
X_selected = selector.transform(X)

# Get the names of the selected features
selected_features = [feature_names[i] for i in selector.get_support(indices=True)]

# Display results
print("Selected Features (Non-zero Coefficients):")
print(selected_features)

# Display feature coefficients for reference
feature_coefficients = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': lasso.coef_
}).sort_values(by='Coefficient', ascending=False)

print("\nFeature Coefficients:")
print(feature_coefficients)
```

Selected Features (Non-zero Coefficients):
['bmi', 's5']

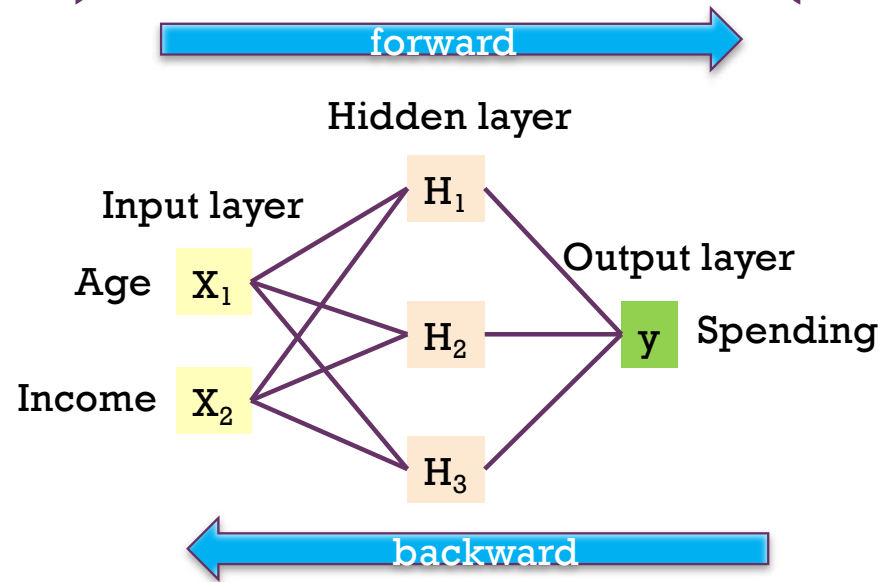
Feature Coefficients:

	Feature	Coefficient
2	bmi	502.679024
8	s5	293.543915
3	bp	0.000000
...		
1	sex	0.000000
4	tc	-0.000000
0	age	-0.000000



3) Neural Networks

3) Neural Networks (universal approximator)

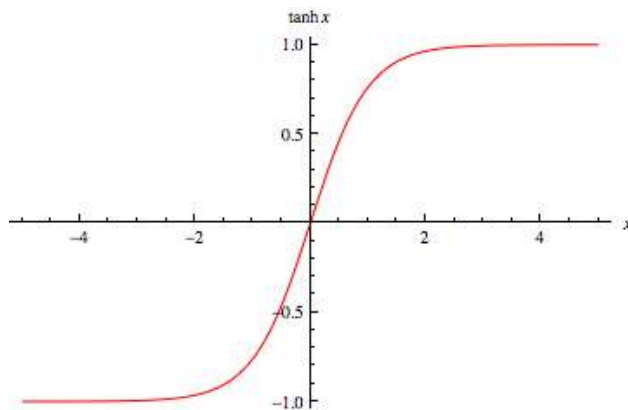


$$\text{Spending} = \hat{w}_0 + \hat{w}_1 H_1 + \hat{w}_2 H_2 + \hat{w}_3 H_3$$

$$H_1 = \tanh(\hat{w}_{10} + \hat{w}_{11}x_1 + \hat{w}_{12}x_2)$$

$$H_2 = \tanh(\hat{w}_{20} + \hat{w}_{21}x_1 + \hat{w}_{22}x_2)$$

$$H_3 = \tanh(\hat{w}_{30} + \hat{w}_{31}x_1 + \hat{w}_{32}x_2)$$



How to update weight

<https://mattmazor.com/2015/03/17/a-step-by-step-backpropagation-example/>



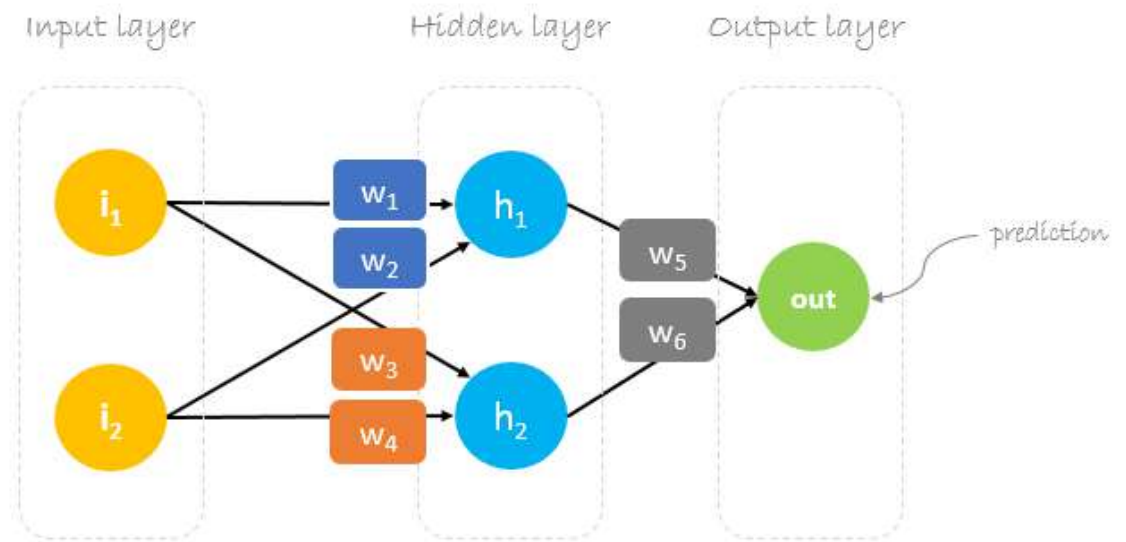
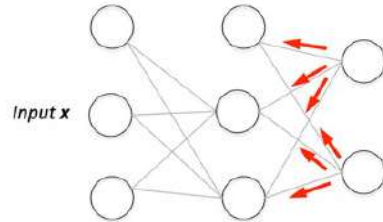
NN Example: Forward & backwork passes

104

HMKCODE

Backpropagation Step by Step

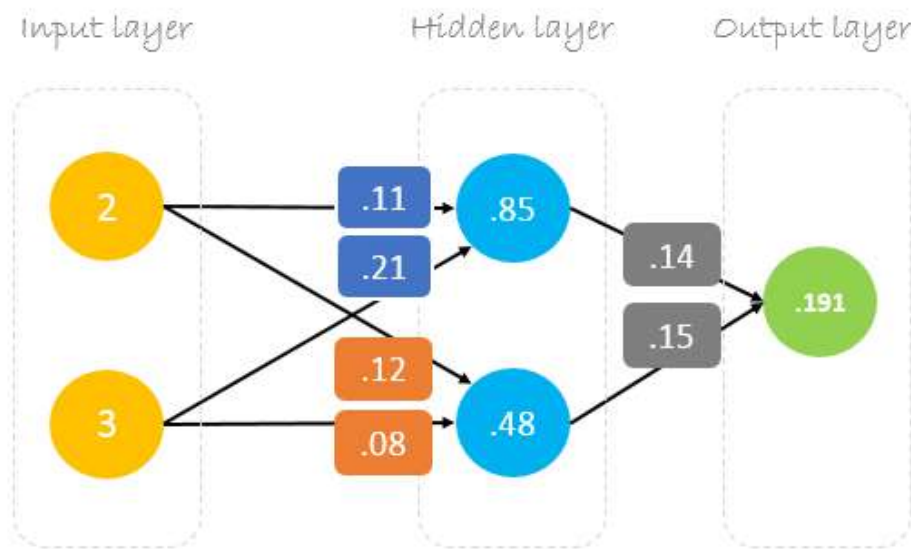
03 NOV 2019



<https://hmkcode.com/ai/backpropagation-step-by-step/>

+ 1) Forward pass (prediction)

Our single sample is as following `inputs=[2, 3]` and `output=[1]`.



Forward Pass

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = \begin{bmatrix} 0.85 & 0.48 \end{bmatrix} \cdot \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = \begin{bmatrix} 0.191 \end{bmatrix}$$

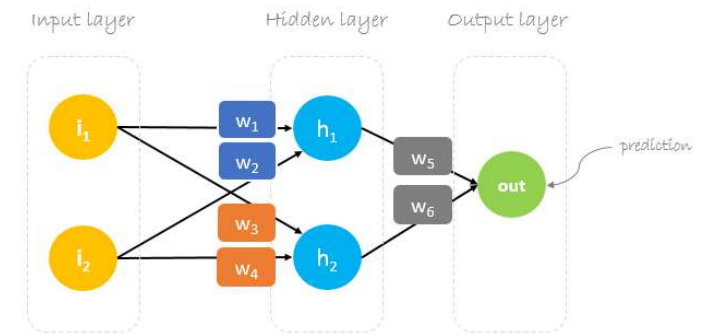
$$2 \times .11 + 3 \times .21 = .85$$

$$.85 \times .14 + .48 \times .15 = .191$$

$$2 \times .12 + 3 \times .08 = .48$$

Matrix multiplication

Details



Equations

$$h_1 = w_1 \cdot i_1 + w_2 \cdot i_2$$

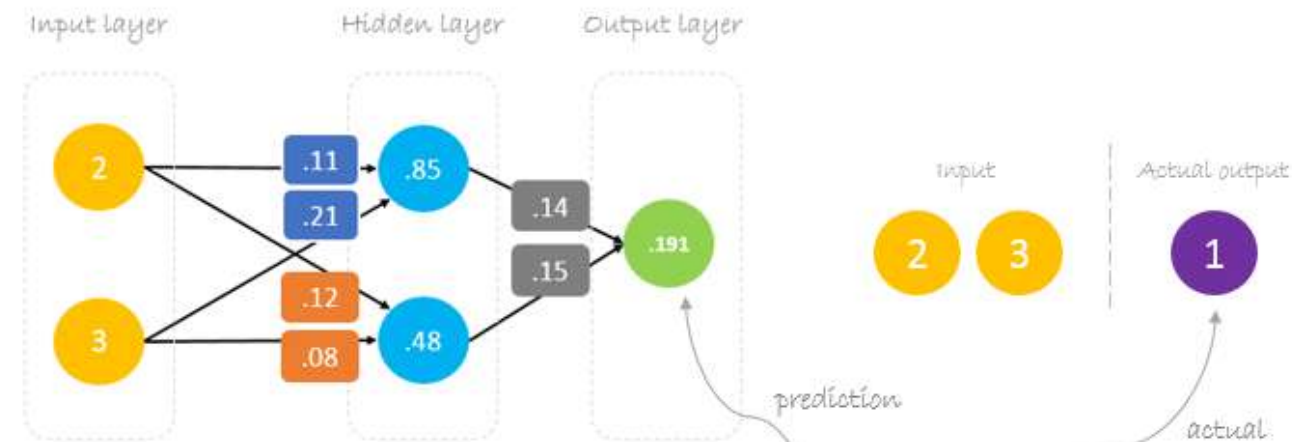
$$h_2 = w_3 \cdot i_1 + w_4 \cdot i_2$$

$$out = w_5 \cdot h_1 + w_6 \cdot h_2$$

+

2) Backward pass (aka. backpropagation)

Calculating the loss



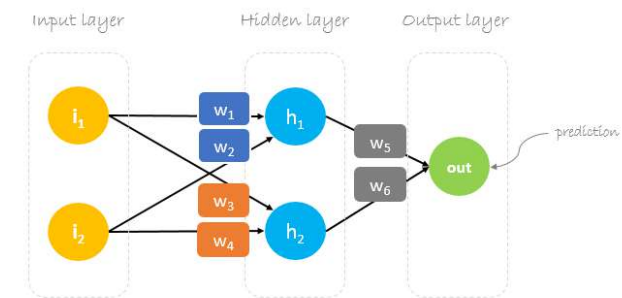
Loss = 0, if prediction = actual

$$\text{Loss} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

Loss is always positive because of the square

$\frac{1}{2}$ is added to ease the calculation of the derivative

$$\text{Loss} = \frac{1}{2}(\mathbf{0.191} - \mathbf{1.0})^2 = \mathbf{0.327}$$

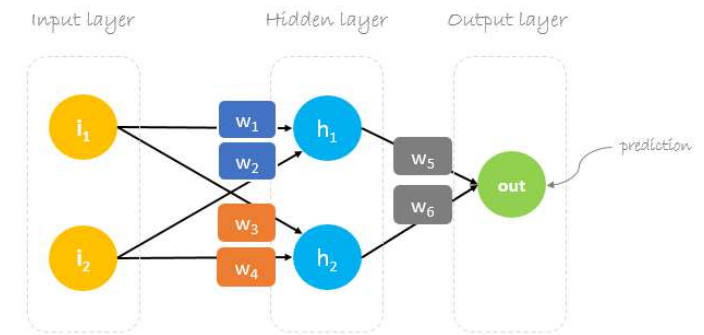
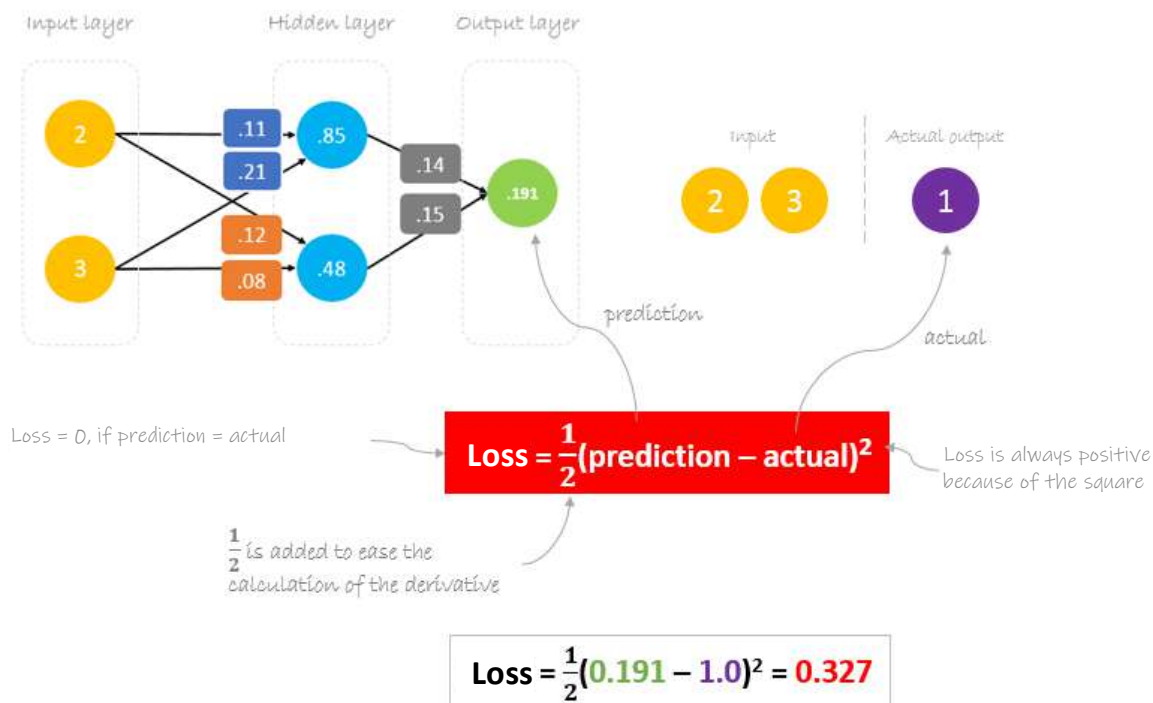


Step by step

1. Error = prediction - actual = $\mathbf{0.191} - \mathbf{1.0} = \mathbf{-0.809}$

2. Loss = $\frac{1}{2} \times \text{Error}^2 = \frac{1}{2} \times (-0.809)^2 = \mathbf{0.327}$

+ Reducing loss



$$\text{prediction} = \text{out}$$

$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$

$$\begin{aligned} h_1 &= i_1 w_1 + i_2 w_2 \\ h_2 &= i_1 w_3 + i_2 w_4 \end{aligned}$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

to change **prediction** value,
we need to change **weights**

$$h_1 = w_1 \cdot i_1 + w_2 \cdot i_2$$

$$h_2 = w_3 \cdot i_1 + w_4 \cdot i_2$$

$$\text{out} = w_5 \cdot h_1 + w_6 \cdot h_2$$

+ Backpropagation

w5, w6

prediction = out

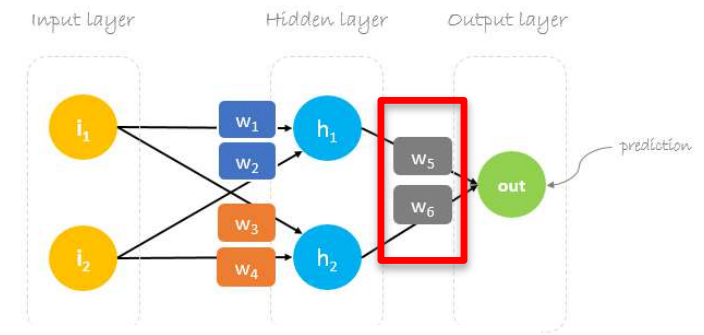
$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$h_2 = i_1 w_3 + i_2 w_4$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

to change **prediction** value,
we need to change **weights**



$$\frac{\partial \text{Loss}}{\partial w_6} = \frac{\partial \text{Loss}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial w_6}$$

chain rule

$$\text{Loss} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

$$\frac{\partial \text{Loss}}{\partial w_6} = \frac{\partial \frac{1}{2}(\text{prediction} - \text{actual})^2}{\partial \text{prediction}} * \frac{\partial ((i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6)}{\partial w_6}$$

$$\frac{\partial \text{Loss}}{\partial w_6} = 2 * \frac{1}{2}(\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (i_1 w_3 + i_2 w_4)$$

$$\frac{\partial \text{Loss}}{\partial w_6} = (\text{prediction} - \text{actual}) * (h_2)$$

$$\frac{\partial \text{Loss}}{\partial w_6} = \Delta h_2$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

$$h_2 = i_1 w_3 + i_2 w_4$$

$$\Delta = \text{prediction} - \text{actual}$$

delta

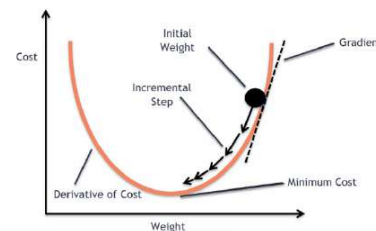
$$*W_x = W_x - a \left(\frac{\partial \text{Loss}}{\partial w_x} \right)$$

Old weight → New weight

Derivative of Loss with respect to weight → Learning rate

$$*W_6 = W_6 - a \Delta h_2$$

$$*W_5 = W_5 - a \Delta h_1$$



+ Backpropagation

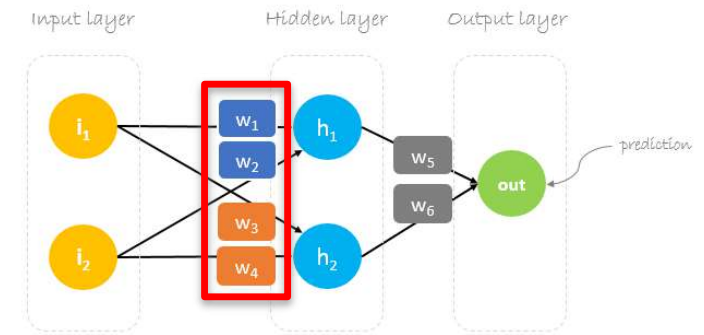
w1,w2,w3,w4

prediction = out

prediction = $(h_1) w_5 + (h_2) w_6$

prediction = $(i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$

to change **prediction** value,
we need to change **weights**



$$\frac{\partial \text{Loss}}{\partial W_1} = \frac{\partial \text{Loss}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial h_1} * \frac{\partial h_1}{\partial W_1} \quad \leftarrow \text{chain rule}$$

$$\frac{\partial \text{Loss}}{\partial W_1} = \frac{\partial \frac{1}{2} (\text{prediction} - \text{actual})^2}{\partial \text{prediction}} * \frac{\partial ((h_1) w_5 + (h_2) w_6)}{\partial h_1} * \frac{\partial (i_1 w_1 + i_2 w_2)}{\partial w_1}$$

$$\frac{\partial \text{Loss}}{\partial W_1} = 2 * \frac{1}{2} (\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (w_5) * (i_1)$$

$$\frac{\partial \text{Loss}}{\partial W_1} = (\text{prediction} - \text{actual}) * (w_5 i_1)$$

$$\frac{\partial \text{Loss}}{\partial W_1} = \Delta w_5 i_1$$

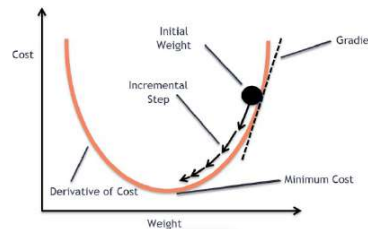
$$\text{Loss} = \frac{1}{2} (\text{prediction} - \text{actual})^2$$

$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$\text{New weight} = \text{Old weight} - \text{Learning rate} * \left(\frac{\partial \text{Loss}}{\partial W_x} \right)$$

$$\Delta = \text{prediction} - \text{actual} \quad \leftarrow \text{delta}$$



Updated weights

$$\begin{aligned} *w_6 &= w_6 - a (h_2 \cdot \Delta) \\ *w_5 &= w_5 - a (h_1 \cdot \Delta) \\ *w_4 &= w_4 - a (i_2 \cdot \Delta w_6) \\ *w_3 &= w_3 - a (i_1 \cdot \Delta w_6) \\ *w_2 &= w_2 - a (i_2 \cdot \Delta w_5) \\ *w_1 &= w_1 - a (i_1 \cdot \Delta w_5) \end{aligned}$$

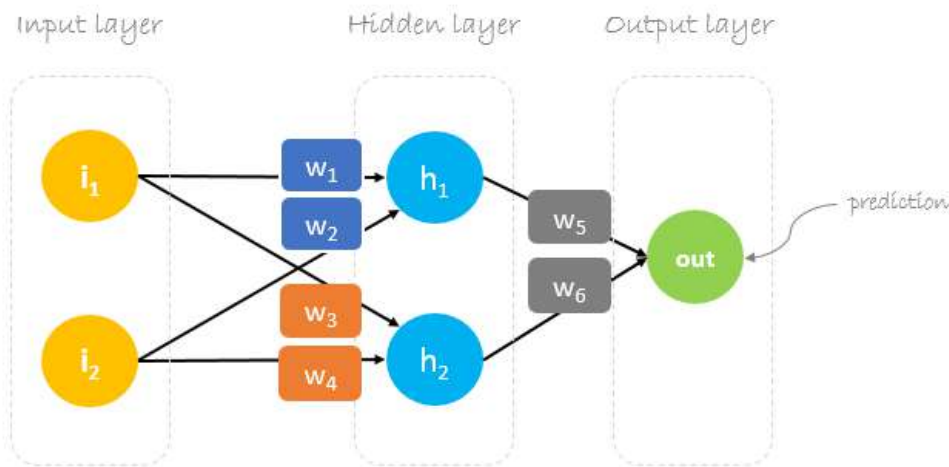


Backpropagation: Matrix Operations

110

$$*W_x = W_x - \underset{\substack{\text{Learning} \\ \text{rate}}}{a} \left(\underset{\substack{\text{Derivative of Loss} \\ \text{with respect to weight}}}{\frac{\partial \text{Loss}}{\partial W_x}} \right)$$

Old weight
New weight



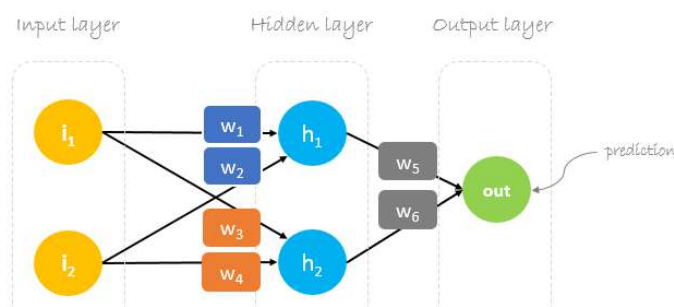
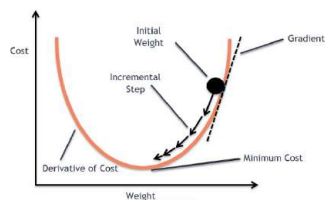
Updated weights

$$\begin{aligned}
 *w_6 &= w_6 - a(h_2 \cdot \Delta) \\
 *w_5 &= w_5 - a(h_1 \cdot \Delta) \\
 *w_4 &= w_4 - a(i_2 \cdot \Delta w_6) \\
 *w_3 &= w_3 - a(i_1 \cdot \Delta w_6) \\
 *w_2 &= w_2 - a(i_2 \cdot \Delta w_5) \\
 *w_1 &= w_1 - a(i_1 \cdot \Delta w_5)
 \end{aligned}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a \Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a \Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} \cdot [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a i_1 \Delta w_5 & a i_1 \Delta w_6 \\ a i_2 \Delta w_5 & a i_2 \Delta w_6 \end{bmatrix}$$

+ Backpropagation: Matrix Operations (cont.)



$$\Delta = 0.191 - 1 = -0.809 \quad \leftarrow \text{Delta = prediction - actual}$$

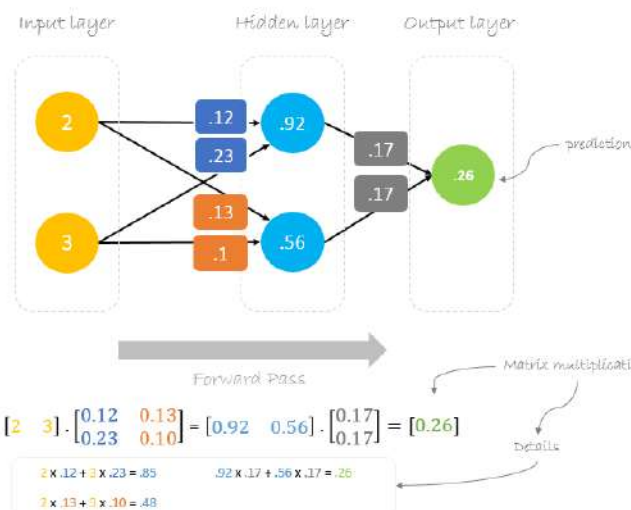
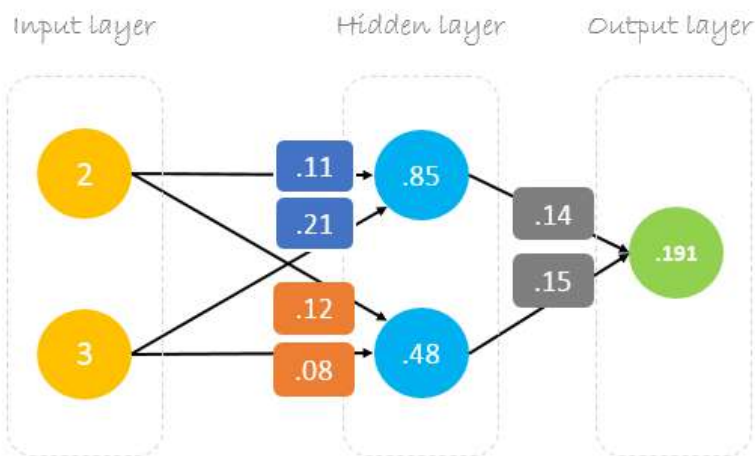
$$a = 0.05 \quad \leftarrow \text{Learning rate, we smartly guess this number}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} 0.14 & 0.15 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a \Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a \Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} \cdot \begin{bmatrix} w_5 & w_6 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a i_1 \Delta w_5 & a i_1 \Delta w_6 \\ a i_2 \Delta w_5 & a i_2 \Delta w_6 \end{bmatrix}$$

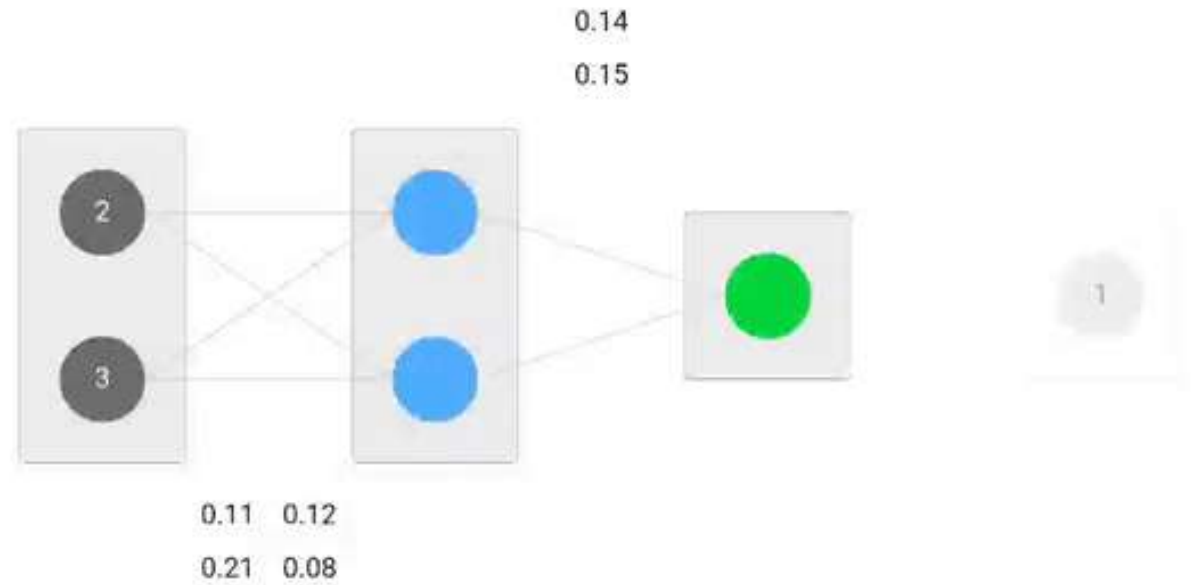
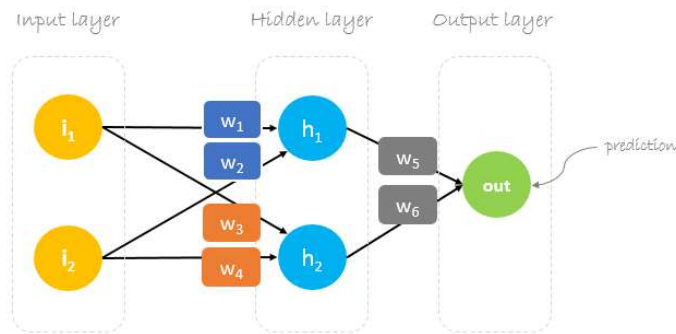




Backpropagation visualization

<https://hmkcode.com/ai/backpropagation-step-by-step/>

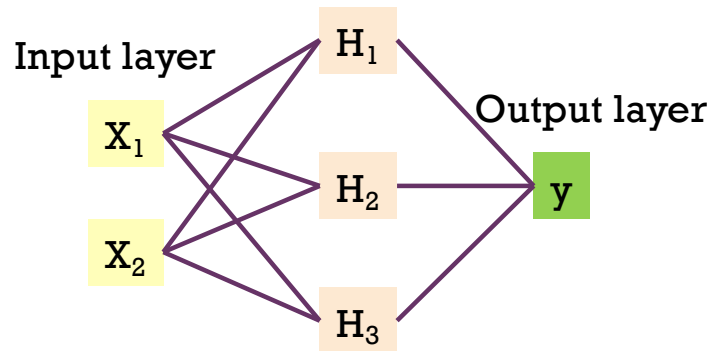
112



+ Gradient descent (solver to find weights)

Calculate gradient → Move in opposite direction → Loss decreases → Repeat

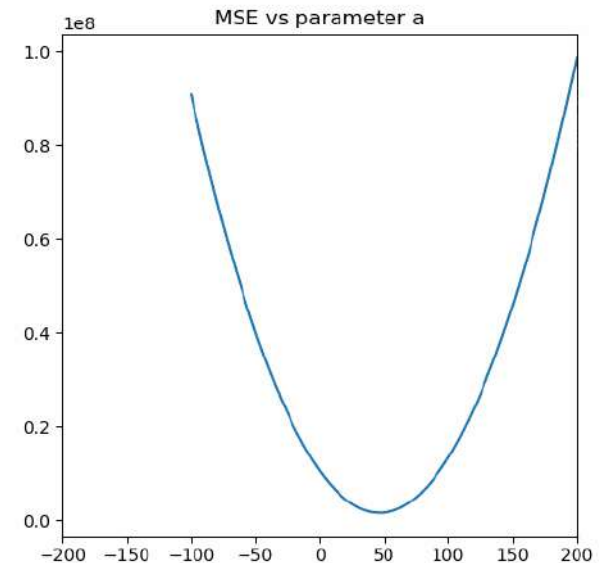
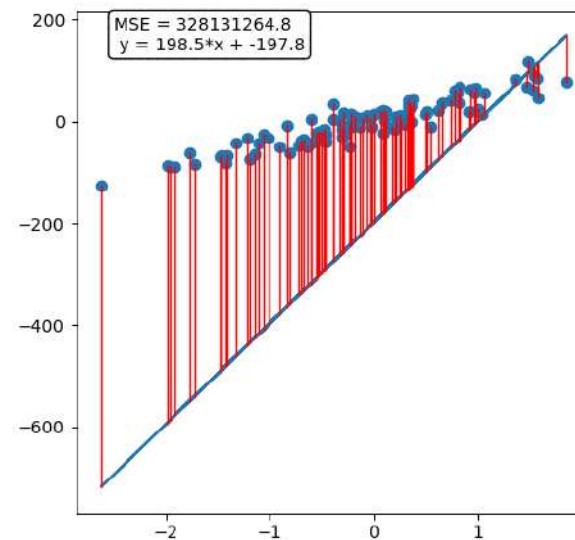
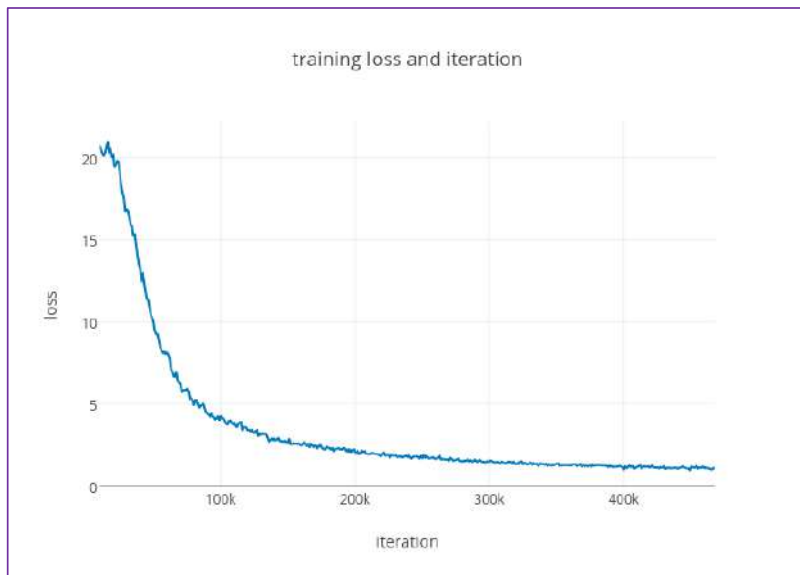
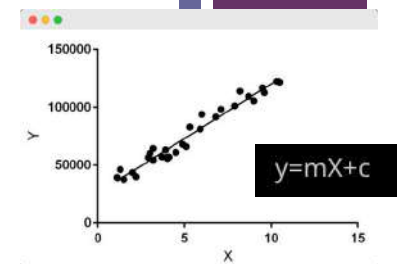
Hidden layer



$$\text{New weight} = \text{weight} - \text{Learning rate} * \text{Gradient}$$

$$*W_x = W_x - a \left(\frac{\partial \text{Loss}}{\partial W_x} \right)$$

Labels in diagram:
- W_x : Old weight
- a : Learning rate
- $\frac{\partial \text{Loss}}{\partial W_x}$: Derivative of Loss with respect to weight
- $*W_x$: New weight





Learning rate

$$*W_x = W_x - a \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

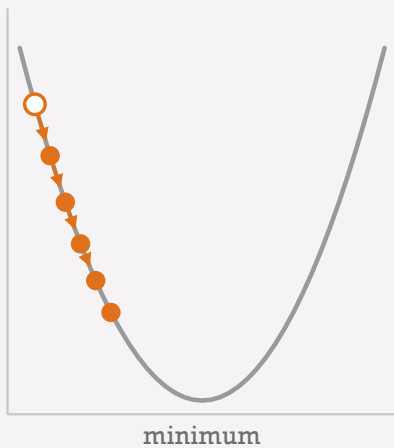
Annotations:
 - W_x : Old weight
 - a : Learning rate
 - $\frac{\partial \text{Error}}{\partial W_x}$: Derivative of Error with respect to weight
 - $*W_x$: New weight

114

Constant learning rate

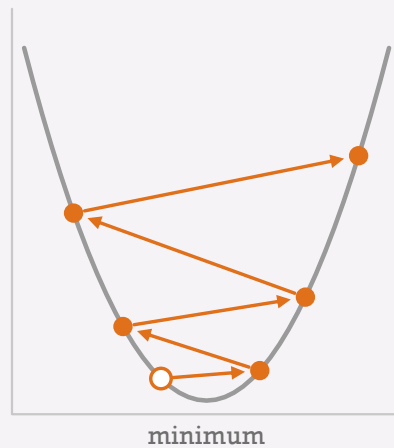
Adaptive learning rate

Too small



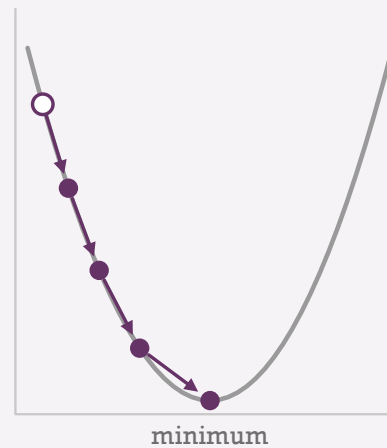
Tiny steps: slow, and training may stop before the minimum

Too large



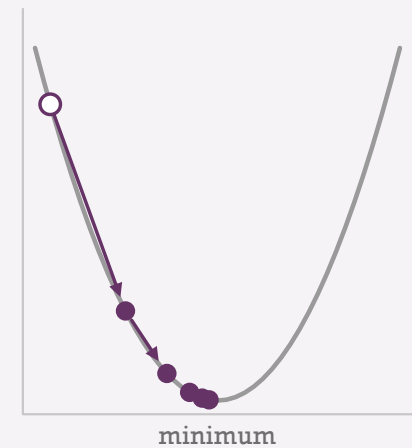
Overshoots the minimum: loss bounces up and can diverge

Just right



Same step size every time, arriving at the minimum

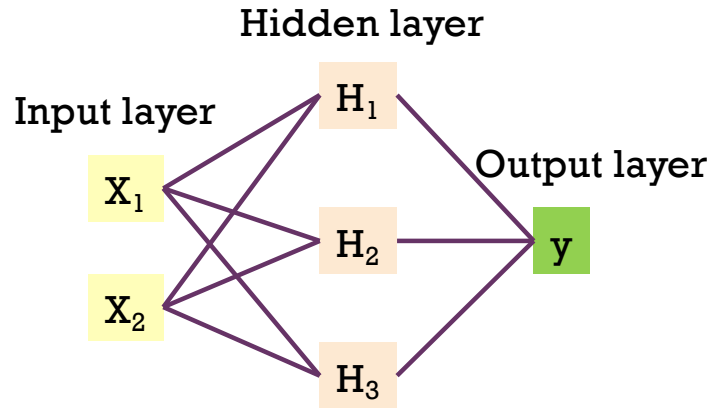
Adaptive



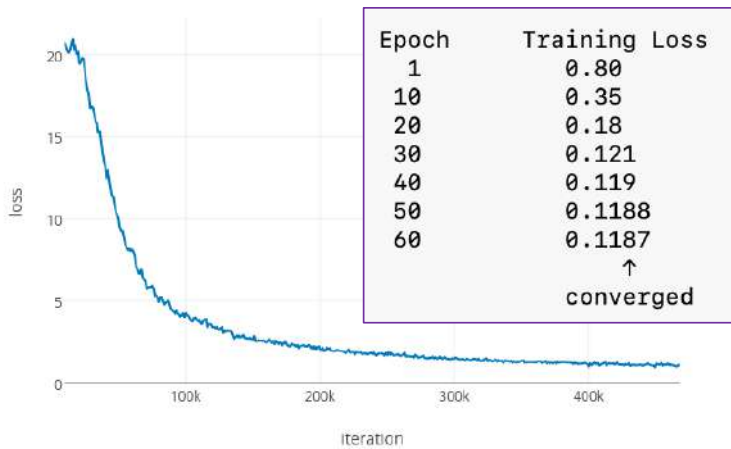
Steps shrink as the rate adapts near the minimum (e.g. Adam, RMSprop)

Each panel: loss (vertical) against a weight (horizontal). Open dot = starting weight; arrows = successive gradient-descent steps.

+ Batch, Iteration, Epoch

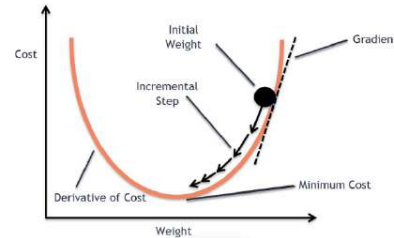


training loss and iteration



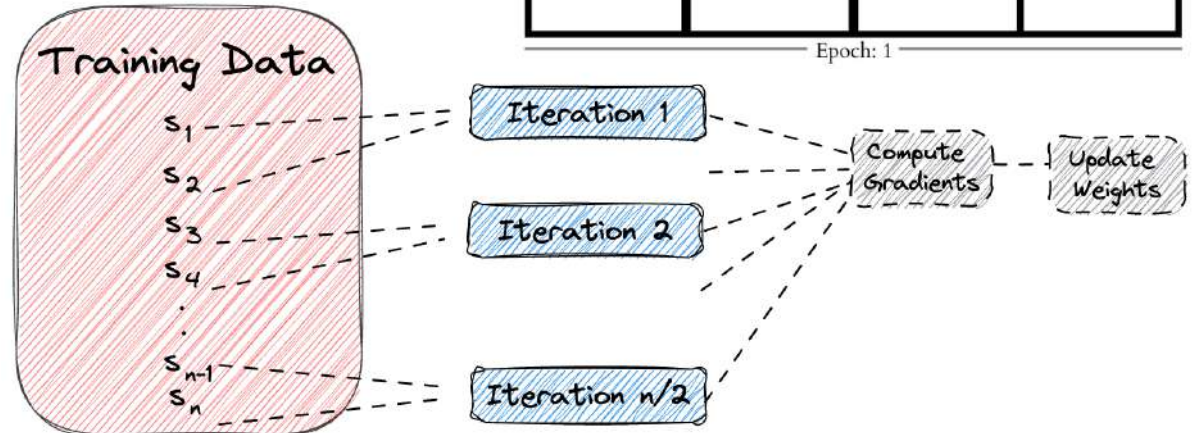
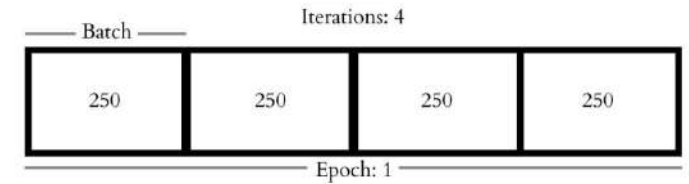
Stop?

- **Converge:** no further improvement (tol: tolerance); e.g., $\text{tol} = 1e-4$
- **Max iteration;** e.g., 500



Epoch vs Iterations vs Batch

Example Dataset: 1000



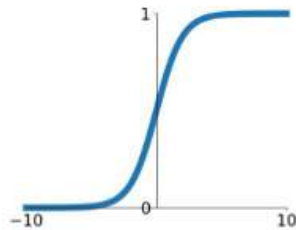
Method	Data used for one update
Stochastic GD (SGD)	1 sample
Batch GD	Entire training set
Mini-batch GD	Small batch



Activation functions

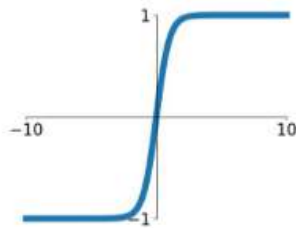
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



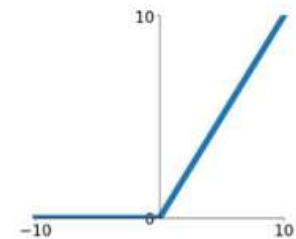
tanh

$$\tanh(x)$$



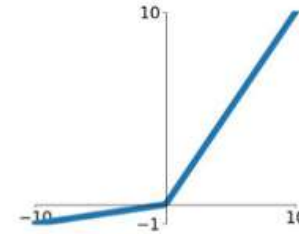
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

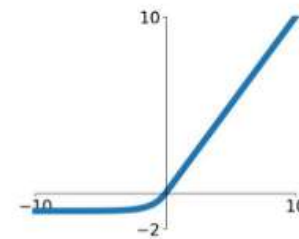


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

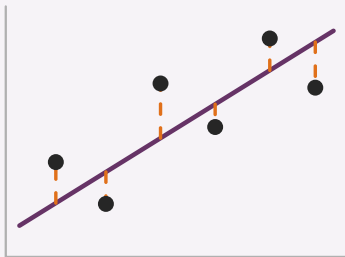


Loss functions: regression vs. classification

Regression → MSE

Use for: Predicting a number (price, temperature)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$



Error = gap to the fitted line

- Squares each error, so big misses cost more
- Always ≥ 0 ; equals 0 only when every prediction is exact
- Units are the target's units squared

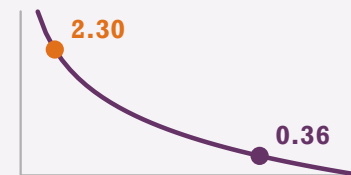
Example actual 3.0, 2.0 · predicted 2.5, 2.1

$$\text{MSE} = (0.5^2 + 0.1^2) \div 2 = \mathbf{0.13}$$

Classification → Cross-entropy

Use for: Predicting a class (cat / dog / bird)

$$\text{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{y}_{ic}$$



Loss = $-\ln p(\text{true class})$

- Only the true class counts
- Confident mistakes cost most
- Used with softmax or sigmoid

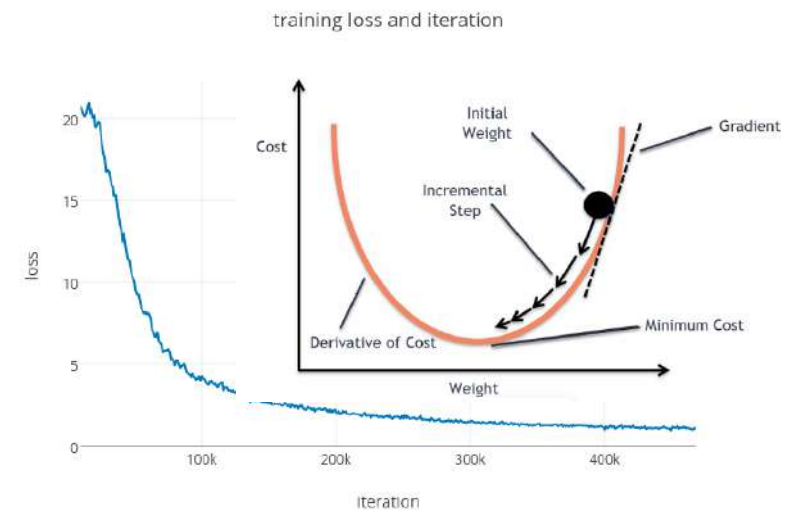
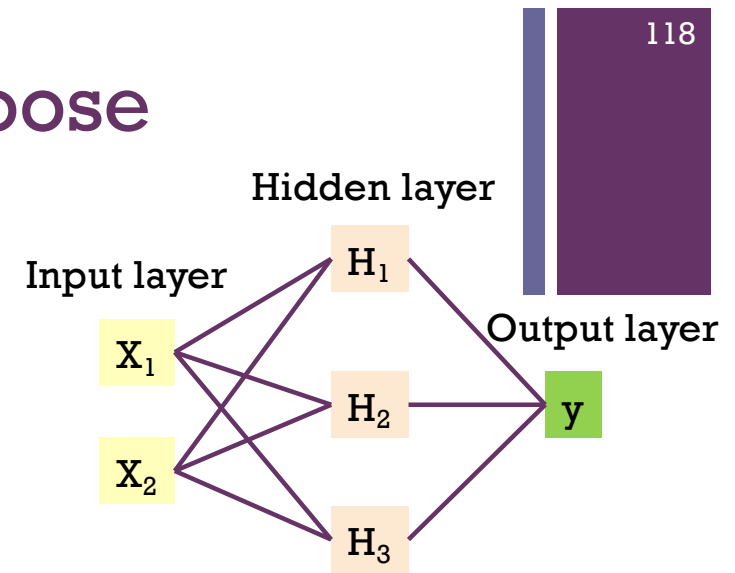
(cat, dog, bird)	True y	Predicted $\hat{y}$	Loss
Cat image	1 0 0	0.7 0.2 0.1	$-\ln 0.7 = 0.36$
Dog image	0 1 0	0.3 0.4 0.3	$-\ln 0.4 = 0.92$
Mean CE = $(0.36 + 0.92) \div 2 = \mathbf{0.64}$			

The $\frac{1}{2}(\text{prediction} - \text{actual})^2$ used on slide 19 is MSE for one sample, halved so the derivative is simpler. $\ln$ = natural log.

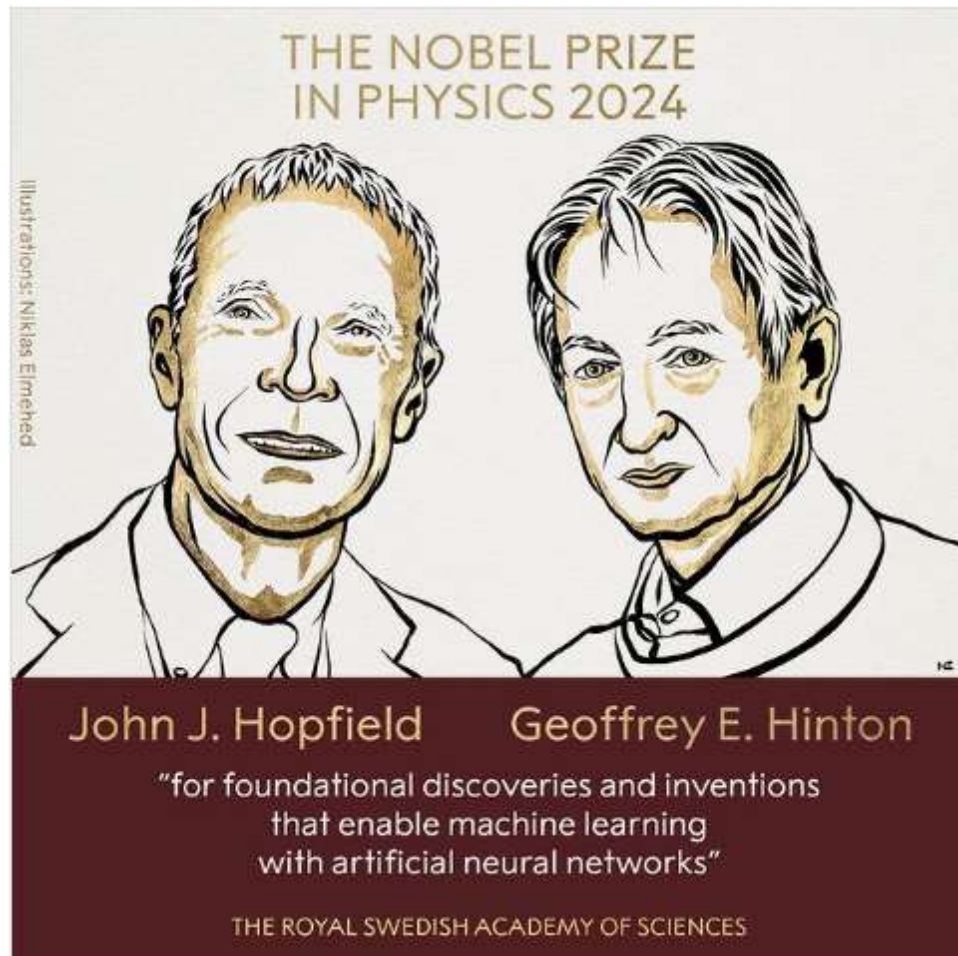


Some key hyperparameters to choose

- Network architecture:
 - Hidden Layers, Hidden Units
 - Activation functions: 'relu', 'tanh', etc.
 - Loss functions: Regression (MSE) or Classification (CE)
- Optimization:
 - Learning rate_init (e.g., 0.01)
 - Learning rate:
 - 'constant' — keep the initial learning rate constant.
 - 'invscaling' — gradually decrease the learning rate.
 - 'adaptive' — keep it constant while training improves; reduce it when improvement stalls.
 - Batch_size
 - Max_iter (e.g., 500)
 - Early_stopping (true/false)
 - alpha = regularization weight



Basic research (foundation research)



8 October 2024

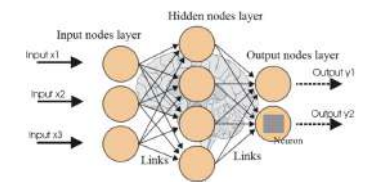
The Royal Swedish Academy of Sciences has decided to award the Nobel Prize in Physics 2024 to

John J. Hopfield
Princeton University, NJ, USA

Geoffrey E. Hinton
University of Toronto, Canada

"for foundational discoveries and inventions that enable machine learning with artificial neural networks"

They trained artificial neural networks using physics



John Hopfield invented a network that uses a method for saving and recreating patterns. We can imagine the nodes as pixels. [The Hopfield network](#) utilises physics that describes a material's characteristics due to its atomic spin – a property that makes each atom a tiny magnet.

Geoffrey Hinton used the Hopfield network as the foundation for a new network that uses a different method: [the Boltzmann machine](#). This can learn to recognise characteristic elements in a given type of data.

Tinker With a **Neural Network** Right Here in Your Browser.

Don't Worry, You Can't Break It. We Promise.

Epoch: 000,000 Learning rate: 0.03 Activation: Tanh Regularization: None Regularization rate: 0 Problem type: Classification

DATA

Which dataset do you want to use?



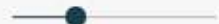
Ratio of training to test data: 50%



Noise: 0



Batch size: 10



[Link](#) [Feedback](#)

FEATURES

Which properties do you want to feed in?

X_1

X_2

X_1^2

X_2^2

$X_1 X_2$

$\sin(X_1)$

2 HIDDEN LAYERS

+ -

4 neurons

h_1

h_2

h_3

h_4

This is the output from one neuron. Hover to see it larger.

+ -

2 neurons

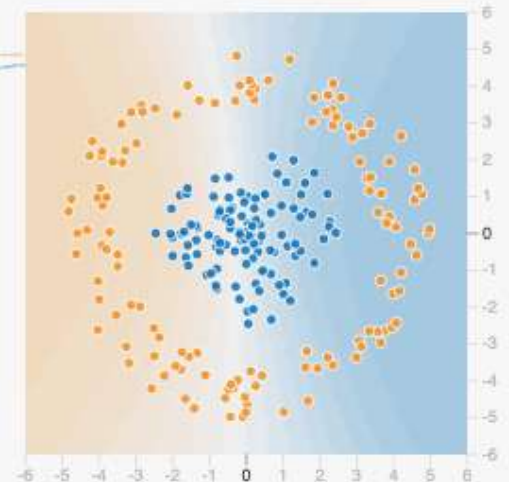
o_1

o_2

OUTPUT

Test loss 0.500

Training loss 0.551



The outputs are mixed with varying weights, shown by the thickness of the lines.

sklearn.neural_network.MLPClassifier

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100, ), activation='relu', solver='adam', alpha=0.0001,
batch_size='auto', learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200, shuffle=True, random_state=None,
tol=0.0001, verbose=False, warm_start=False, momentum=0.9, nesterovs_momentum=True, early_stopping=False,
validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000) ¶
```

[\[source\]](#)

Multi-layer Perceptron classifier.

This model optimizes the log-loss function using LBFGS or stochastic gradient descent.

New in version 0.18.

Parameters:

hidden_layer_sizes : *tuple, length = n_layers - 2, default=(100,)*

The ith element represents the number of neurons in the ith hidden layer.

activation : *{'identity', 'logistic', 'tanh', 'relu'}, default='relu'*

Activation function for the hidden layer.

- 'identity', no-op activation, useful to implement linear bottleneck, returns $f(x) = x$
- 'logistic', the logistic sigmoid function, returns $f(x) = 1 / (1 + \exp(-x))$.
- 'tanh', the hyperbolic tan function, returns $f(x) = \tanh(x)$.
- 'relu', the rectified linear unit function, returns $f(x) = \max(0, x)$

solver : *{'lbfgs', 'sgd', 'adam'}, default='adam'*

The solver for weight optimization

SGD — Basic gradient-based optimization; simple and highly configurable
Adam — Adaptive gradient-based optimizer; good default for most problems
L-BFGS — Quasi-Newton optimizer; often effective for smaller datasets

sklearn.neural_network.MLPRegressor

```
class sklearn.neural_network.MLPRegressor(hidden_layer_sizes=(100,), activation='relu', solver='adam', alpha=0.0001,
batch_size='auto', learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200, shuffle=True, random_state=None,
tol=0.0001, verbose=False, warm_start=False, momentum=0.9, nesterovs_momentum=True, early_stopping=False,
validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000) ¶ \[source\]
```

Multi-layer Perceptron regressor.

This model optimizes the squared-loss using LBFGS or stochastic gradient descent.

New in version 0.18.

Parameters:

hidden_layer_sizes : *tuple, length = n_layers - 2, default=(100,)*

The ith element represents the number of neurons in the ith hidden layer.

activation : *{'identity', 'logistic', 'tanh', 'relu'}, default='relu'*

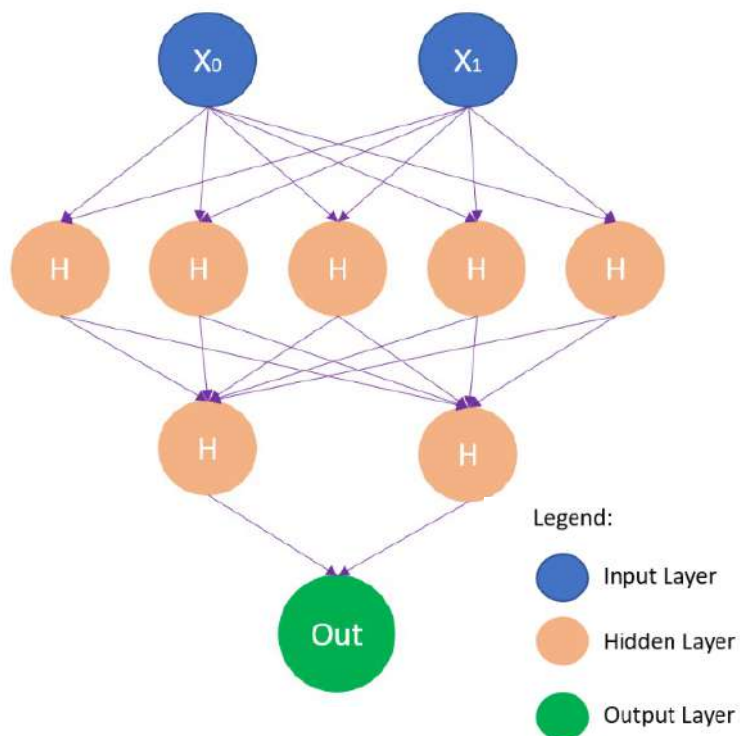
Activation function for the hidden layer.

- 'identity', no-op activation, useful to implement linear bottleneck, returns $f(x) = x$
- 'logistic', the logistic sigmoid function, returns $f(x) = 1 / (1 + \exp(-x))$.
- 'tanh', the hyperbolic tan function, returns $f(x) = \tanh(x)$.
- 'relu', the rectified linear unit function, returns $f(x) = \max(0, x)$

solver : *{'lbfgs', 'sgd', 'adam'}, default='adam'*

The solver for weight optimization.

Example



batch_size : *int*, *default='auto'*

Size of minibatches for stochastic optimizers. If the solver is 'lbfgs', the classifier will not use minibatch. When set to "auto", `batch_size=min(200, n_samples)`.

learning_rate : {'constant', 'invscaling', 'adaptive'}, *default='constant'*

Learning rate schedule for weight updates.

```
mlp_clf = MLPClassifier(hidden_layer_sizes=(5,2),
                        max_iter = 300, activation = 'relu',
                        solver = 'adam')
```

```
param_grid = {
    'hidden_layer_sizes': [(150,100,50), (120,80,40), (100,50)],
    'max_iter': [50, 100, 150],
    'activation': ['tanh', 'relu'],
    'solver': ['sgd', 'adam'],
    'alpha': [0.0001, 0.05],
    'learning_rate': ['constant', 'adaptive'],
}
```

```
grid = GridSearchCV(mlp_clf, param_grid, n_jobs= -1, cv=5)
grid.fit(trainX_scaled, trainY)
```

```
print(grid.best_params_)
```

+

4) kNN



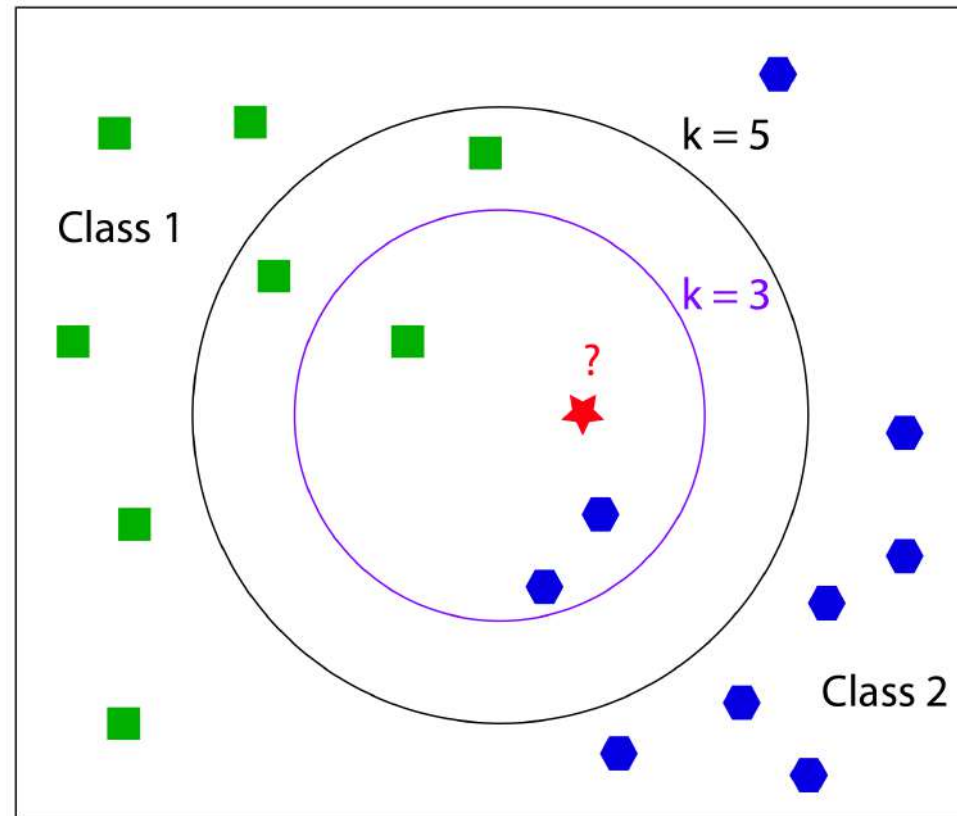
4) k-Nearest Neighbors (kNN)

Important Params:

- k
- Distance function

125

- Memory based learning
- Suitable for small data sets
- Merge
 - Voting
 - Average
 - Maximum prob
- Cautions:
 - Support only numerical variables
 - Need to adjust variable range





Distance Function

■ Euclidean Distance

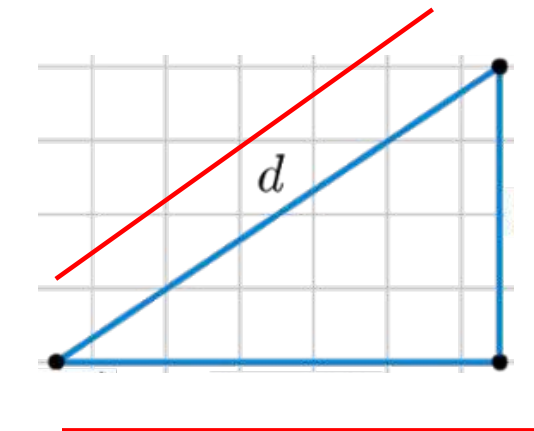
$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

■ Manhattan Distance

$$\sum_{i=1}^k |x_i - y_i|$$

■ Minkowski Distance

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$



$q = 1$ is Manhattan Distance
 $q = 2$ is Euclidean Distance



Distance Function (cont.)

127

- Hamming distance: same (0) vs diff (1)
- **It still supports only numeric variable.**

$$D_H = \sum_{i=1}^k |x_i - y_i|$$

$$x = y \Rightarrow D = 0$$

$$x \neq y \Rightarrow D = 1$$

X	Y	Distance
Male	Male	0
Male	Female	1



Nearest Neighbors

128

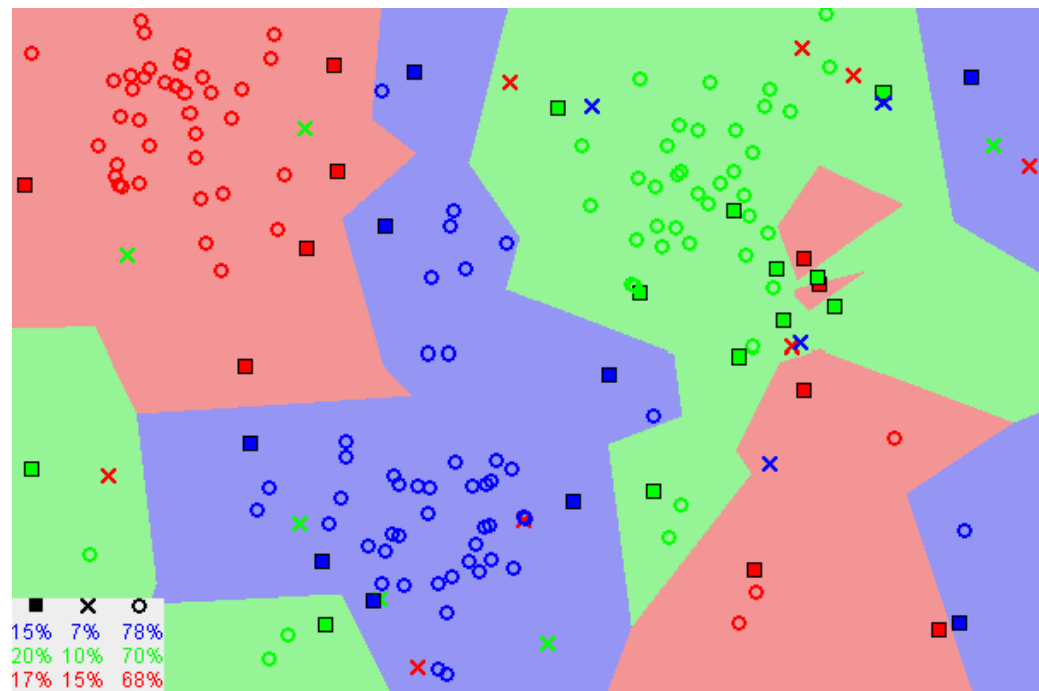


Image showing how similar data points typically exist close to each other

Source : <https://towardsdatascience.com/machine-learning-basics-with-the-k-nearest-neighbors-algorithm-6a6e71d01761>



`sklearn.neighbors.KNeighborsClassifier`

```
class sklearn.neighbors.KNeighborsClassifier(n_neighbors=5, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None, **kwargs)
```

[\[source\]](#)

Classifier implementing the k-nearest neighbors vote.

Read more in the [User Guide](#).

Parameters:

`n_neighbors` : *int, optional (default = 5)*

Number of neighbors to use by default for `kneighbors` queries.

`weights` : *str or callable, optional (default = 'uniform')*

weight function used in prediction. Possible values:

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally.
- 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

`algorithm` : *{'auto', 'ball_tree', 'kd_tree', 'brute'}, optional*

Algorithm used to compute the nearest neighbors:

- 'ball_tree' will use `BallTree`
- 'kd_tree' will use `KDTree`
- 'brute' will use a brute-force search.
- 'auto' will attempt to decide the most appropriate algorithm based on the values passed to `fit` method

sklearn.neighbors.KNeighborsRegressor

```
class sklearn.neighbors.KNeighborsRegressor(n_neighbors=5, weights='uniform', algorithm='auto', leaf_size=30, p=2, metric='minkowski', metric_params=None, n_jobs=None, **kwargs)
```

[\[source\]](#)

Regression based on k-nearest neighbors.

The target is predicted by local interpolation of the targets associated of the nearest neighbors in the training set.

Read more in the [User Guide](#).

New in version 0.9.

Parameters:

n_neighbors : *int, optional (default = 5)*

Number of neighbors to use by default for `kneighbors` queries.

weights : *str or callable*

weight function used in prediction. Possible values:

- 'uniform' : uniform weights. All points in each neighborhood are weighted equally.
- 'distance' : weight points by the inverse of their distance. in this case, closer neighbors of a query point will have a greater influence than neighbors which are further away.
- [callable] : a user-defined function which accepts an array of distances, and returns an array of the same shape containing the weights.

Uniform weights are used by default.



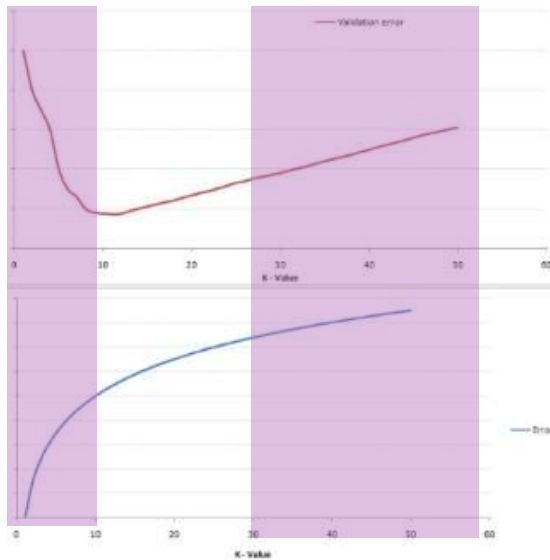
Choosing the right value of K

131

Search the best parameters with ...

Too Underfitted

Too Overfitted



`sklearn.model_selection.GridSearchCV`

```
class sklearn.model_selection.GridSearchCV(estimator, param_grid, *, scoring=None, n_jobs=None, iid='deprecated', refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score=nan, return_train_score=False)
```

[\[source\]](#)

`sklearn.model_selection.RandomizedSearchCV`

```
class sklearn.model_selection.RandomizedSearchCV(estimator, param_distributions, *, n_iter=10, scoring=None, n_jobs=None, iid='deprecated', refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', random_state=None, error_score=nan, return_train_score=False)
```

[\[source\]](#)

```
param_grid=dict(  
    n_neighbors=[1,2,3,4,5,6,7,8,9,10],  
    weights=['uniform', 'distance'],  
)
```

Search with **k: 1-10**
on **uniform/distance** weights



Evaluation Measures



Evaluation (Train/Test Split)

Training Data

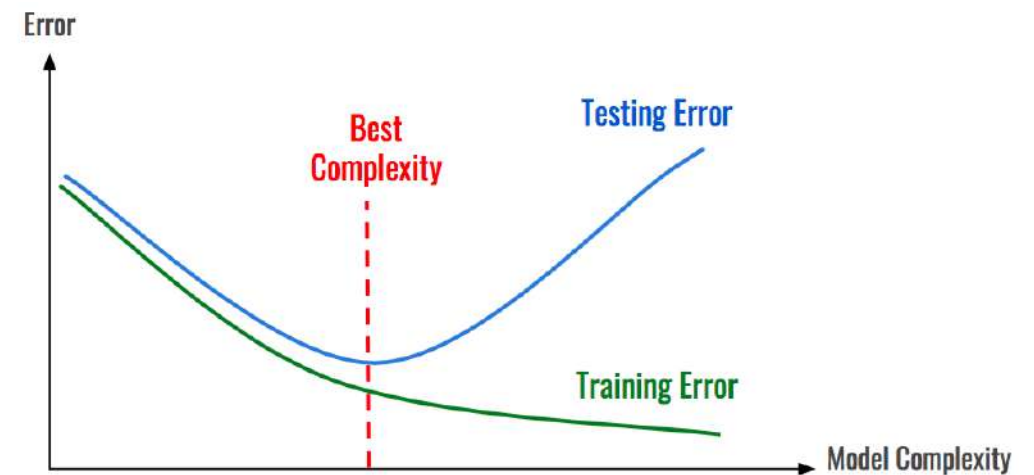


Age	Income	Purchase
25	25,000	Yes
35	50,000	Yes
32	35,000	No

Testing Data

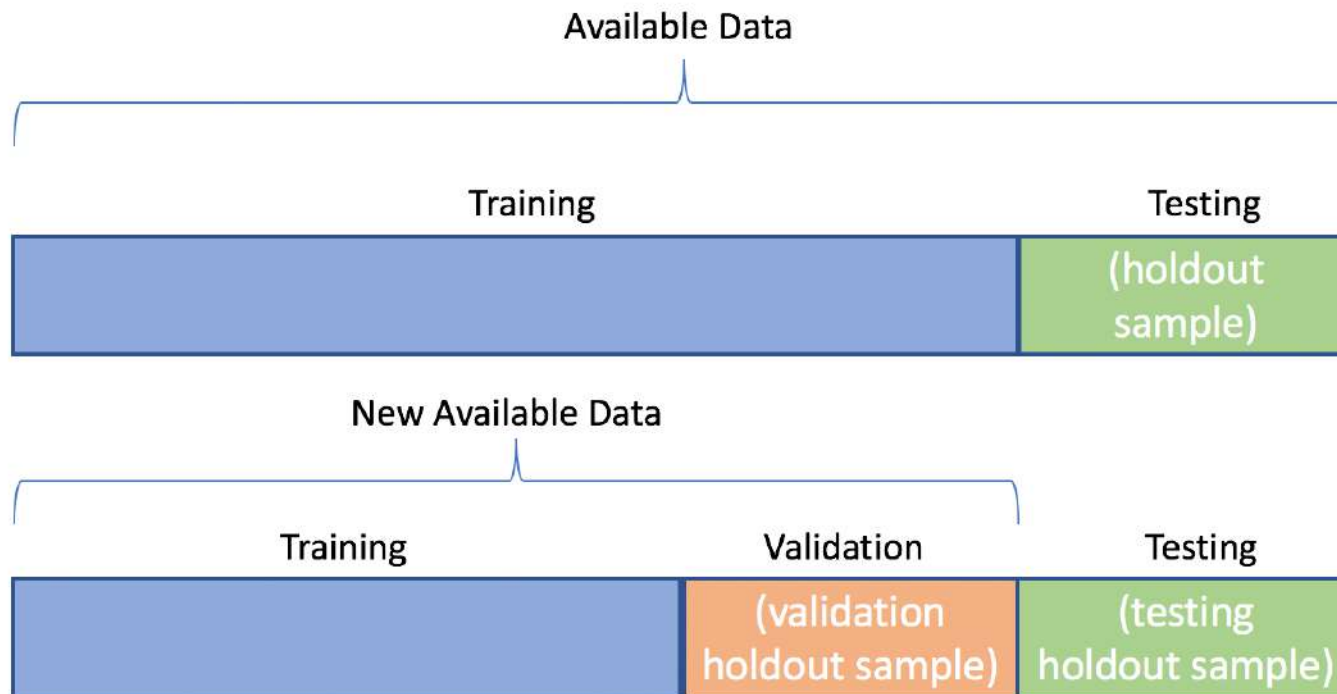
Age	Income	Purchase
27	35,000	Yes
23	20,000	No
45	34,000	No

Age	Income	Purchase
27	35,000	Yes
23	20,000	No
45	34,000	No



+ Train (Validation) & Test

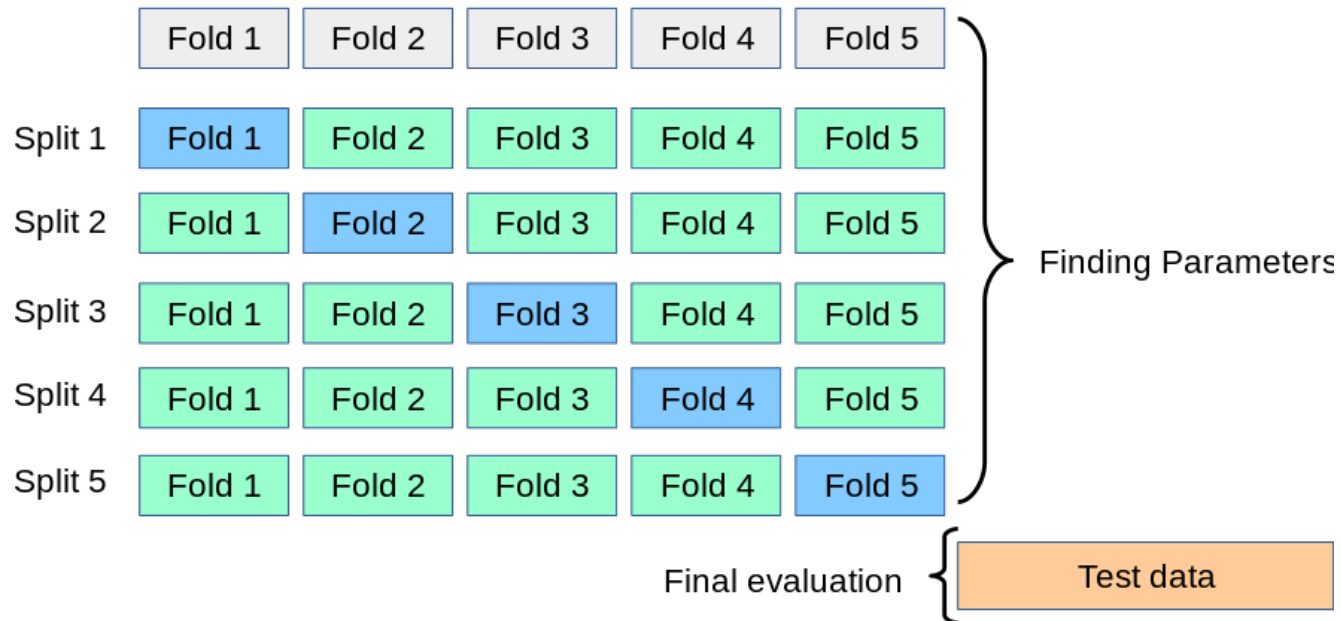
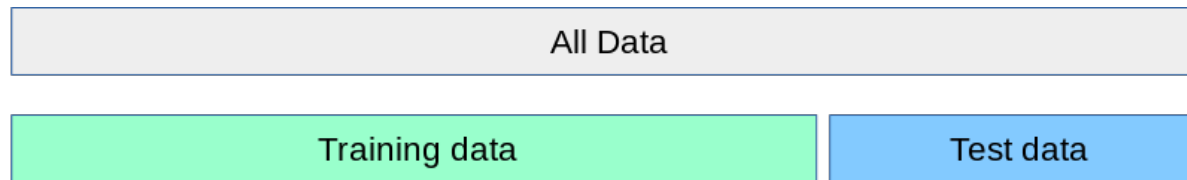
- Training data = Textbook
- Validation data = Exercise
- Testing data = Final exam





Single Validation → Cross Validation (CV)

135



Example: accuracy per fold

	Model A	Model B
Split 1	82%	78%
Split 2	85%	86%
Split 3	80%	75%
Split 4	84%	81%
Split 5	83%	79%
Mean	82.8%	79.8%

sklearn.model_selection.train_test_split

```
sklearn.model_selection.train_test_split(*arrays, **options)
```

[\[source\]](#)

Split arrays or matrices into random train and test subsets

Quick utility that wraps input validation and `next(ShuffleSplit().split(X, y))` and application to input data into a single call for splitting (and optionally subsampling) data in a oneliner.

Read more in the [User Guide](#).

Parameters:

***arrays : sequence of indexables with same length / shape[0]**

Allowed inputs are lists, numpy arrays, scipy-sparse matrices or pandas dataframes.

test_size : float, int or None, optional (default=None)

If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the test split. If int, represents the absolute number of test samples. If None, the value is set to the complement of the train size. If `train_size` is also None, it will be set to 0.25.

train_size : float, int, or None, (default=None)

If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the train split. If int, represents the absolute number of train samples. If None, the value is automatically set to the complement of the test size.

random_state : int, RandomState instance or None, optional (default=None)

If int, random_state is the seed used by the random number generator; If RandomState instance, random_state is the random number generator; If None, the random number generator is the RandomState instance used by `np.random`.



Model Evaluation

137

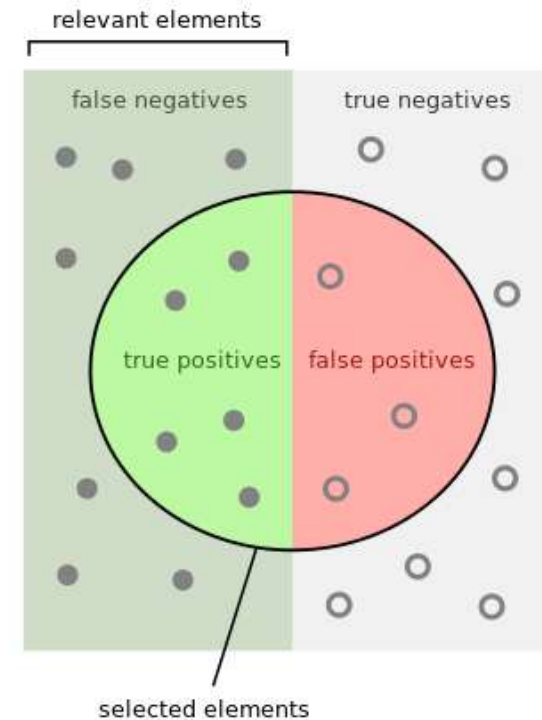
- Regression
 - Sum Squared Error (SSE)
 - Average Squared Error (ASE)
- Classification
 - **Accuracy**
 - Misclassification
- Precision
- Recall
- F1
- Graph
 - ROC Curve
 - Area Under ROC (c-statistic)
 - Lift
 - Gain
 - Response



Precision, Recall, F1

	Predicted: NO	Predicted: YES	
n=165 Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	

- Precision = correctly predict = $TP / (TP + FP)$
- Recall = coverage = $TP / (TP + FN)$
- F1 = $(2 * pre * rec) / (pre + rec)$

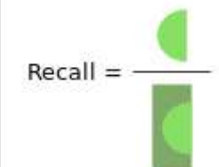


How many selected items are relevant?



Precision =

How many relevant items are selected?



Recall =

3.5.1. The `scoring` parameter: defining model evaluation rules

Model selection and evaluation using tools, such as `grid_search.GridSearchCV` and `cross_validation.cross_val_score`, take a `scoring` parameter that controls what metric they apply to estimators evaluated.

3.5.1.1. Common cases: predefined values

For the most common usecases, you can simply provide a string as the `scoring` parameter. Possible values are:

Scoring	Function
Classification	
'accuracy'	<code>sklearn.metrics.accuracy_score</code>
'average_precision'	<code>sklearn.metrics.average_precision_score</code>
'f1'	<code>sklearn.metrics.f1_score</code>
'precision'	<code>sklearn.metrics.precision_score</code>
'recall'	<code>sklearn.metrics.recall_score</code>
'roc_auc'	<code>sklearn.metrics.roc_auc_score</code>
Clustering	
'adjusted_rand_score'	<code>sklearn.metrics.adjusted_rand_score</code>
Regression	
'mean_absolute_error'	<code>sklearn.metrics.mean_absolute_error</code>
'mean_squared_error'	<code>sklearn.metrics.mean_squared_error</code>
'r2'	<code>sklearn.metrics.r2_score</code>

sklearn.metrics.confusion_matrix

```
sklearn.metrics.confusion_matrix(y_true, y_pred, labels=None, sample_weight=None, normalize=None)
```

[\[source\]](#)

Compute confusion matrix to evaluate the accuracy of a classification.

By definition a confusion matrix C is such that $C_{i,j}$ is equal to the number of observations known to be in group i and predicted to be in group j .

Thus in binary classification, the count of true negatives is $C_{0,0}$, false negatives is $C_{1,0}$, true positives is $C_{1,1}$ and false positives is $C_{0,1}$.

Read more in the [User Guide](#).

Parameters: **y_true** : array-like of shape (n_samples,)

Ground truth (correct) target values.

```
>>> from sklearn.metrics import confusion_matrix
>>> y_true = [2, 0, 2, 2, 0, 1]
>>> y_pred = [0, 0, 2, 2, 0, 2]
>>> confusion_matrix(y_true, y_pred)
array([[2, 0, 0],
       [0, 0, 1],
       [1, 0, 2]])
```

y_pred : array-like of shape (n_samples,)

Predicted target values (as returned by a classifier).

labels : list of shape (n_classes), default=None

List of labels to index the matrix. This may be used to reorder or select a subset of labels. If None is given, those labels appearing in y_true or y_pred are used in sorted order.

sample_weight : array-like of shape (n_samples,), default=None

Sample weights to be used in the calculation.

normalize : {'true', 'pred', 'all'}, default=None

Normalizes confusion matrix over the true (rows), predicted (columns) conditions or all the population. If None, confusion matrix will not be normalized.

sklearn.metrics.classification_report

`sklearn.metrics.classification_report(y_true, y_pred, labels=None, target_names=None, sample_weight=None, digits=2, output_dict=False, zero_division='warn')`

[\[source\]](#)

Build a text report showing the main

Read more in the [User Guide](#).

Parameters: `y_true : 1d array`

Ground truth (c

`y_pred : 1d array`

Estimated target

`labels : array, sh`

Optional list of

`target_names : l`

Optional display

`sample_weight :`

Sample weights

`digits : int`

Number of digits

the returned va

Examples

```
>>> from sklearn.metrics import classification_report
>>> y_true = [0, 1, 2, 2, 2]
>>> y_pred = [0, 0, 2, 2, 1]
>>> target_names = ['class 0', 'class 1', 'class 2']
>>> print(classification_report(y_true, y_pred, target_names=target_names))
              precision    recall  f1-score   support

<BLANKLINE>
 class 0       0.50         1.00         0.67         1
 class 1       0.00         0.00         0.00         1
 class 2       1.00         0.67         0.80         3

<BLANKLINE>
 accuracy              0.60         5
 macro avg           0.50         0.56         0.49         5
 weighted avg        0.70         0.60         0.61         5

<BLANKLINE>
>>> y_pred = [1, 1, 0]
>>> y_true = [1, 1, 1]
>>> print(classification_report(y_true, y_pred, labels=[1, 2, 3]))
              precision    recall  f1-score   support

<BLANKLINE>
           1         1.00         0.67         0.80         3
           2         0.00         0.00         0.00         0
           3         0.00         0.00         0.00         0

<BLANKLINE>
 micro avg         1.00         0.67         0.80         3
 macro avg         0.33         0.22         0.27         3
 weighted avg         1.00         0.67         0.80         3

<BLANKLINE>
```

This documentation is for
scikit-learn **version 0.15-**
git — [Other versions](#)

If you use the software,
please consider [citing](#)
[scikit-learn](#).

`sklearn.metrics.mean_absolute_error`

sklearn.metrics.mean_absolute_error

```
sklearn.metrics.mean_absolute_error(y_true, y_pred, sample_weight=None)
```

Mean absolute error regression loss

Parameters: **y_true** : array-like of shape = [n_samples] or [n_samples, n_outputs]

Ground truth (correct) target values.

y_pred : array-like of shape = [n_samples] or [n_samples, n_outputs]

Estimated target values.

sample_weight : array-like of

Sample weights.

Returns: **loss** : float

A positive floating point

Examples

```
>>> from sklearn.metrics import mean_absolute_error
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> mean_absolute_error(y_true, y_pred)
0.5
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> mean_absolute_error(y_true, y_pred)
0.75
```



Previous
sklearn.metrics.mean_squared_error

Next
sklearn.metrics.mean_squared_error

Up
Reference

This documentation is for
scikit-learn **version 0.15-**
git — [Other versions](#)

If you use the software,
please consider [citing](#)
[scikit-learn](#).

`sklearn.metrics.mean_squared_error`
Examples using
`sklearn.metrics.mean_squared_error`

sklearn.metrics.mean_squared_error

```
sklearn.metrics.mean_squared_error(y_true, y_pred, sample_weight=None)
```

Mean squared error regression loss

Parameters: **y_true** : array-like of shape = [n_samples] or [n_samples, n_outputs]

Ground truth (correct) target values.

y_pred : array-like of shape = [n_samples] or [n_samples, n_outputs]

Estimated target values.

sample_weight : array-like of shape =

Sample weights.

Returns: **loss** : float

A positive floating point value (the

Examples

```
>>> from sklearn.metrics import mean_squared_error
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> mean_squared_error(y_true, y_pred)
0.375
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> mean_squared_error(y_true, y_pred)
0.708...
```



Previous
sklearn.metrics

Next
sklearn.metrics

Up
Reference

This documentation is for
scikit-learn **version 0.15-**
[git](#) — [Other versions](#)

If you use the software,
please consider [citing](#)
scikit-learn.

`sklearn.metrics.r2_score`
Examples using
`sklearn.metrics.r2_score`

sklearn.metrics.r2_score

```
sklearn.metrics.r2_score(y_true, y_pred, sample_weight=None)
```

R² (coefficient of determination) regression score function.

Best possible score is 1.0, lower values are worse.

Parameters: **y_true** : array-like of shape = [n_samples] or [n_samples, n_outputs]

Ground truth (correct) target values.

y_pred : array-like of shape = [n_samples] or [n_samples, n_outputs]

Estimated target values.

sample_weight : array-like of shape = [n_samples]

Sample weights.

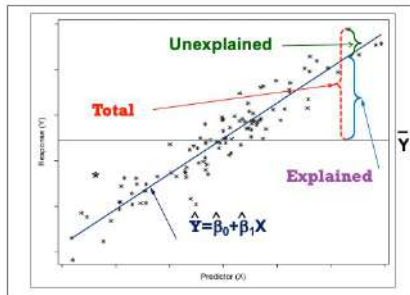
Returns: **z** : float

The R² score.

Examples

```
>>> from sklearn.metrics import r2_score
>>> y_true = [3, -0.5, 2, 7]
>>> y_pred = [2.5, 0.0, 2, 8]
>>> r2_score(y_true, y_pred)
0.948...
>>> y_true = [[0.5, 1], [-1, 1], [7, -6]]
>>> y_pred = [[0, 2], [-1, 2], [8, -5]]
>>> r2_score(y_true, y_pred)
0.938...
```

Coefficient of Determination



Regression

'mean_absolute_error' `sklearn.metrics.mean_absolute_error`

'mean_squared_error' `sklearn.metrics.mean_squared_error`

'r2' `sklearn.metrics.r2_score`

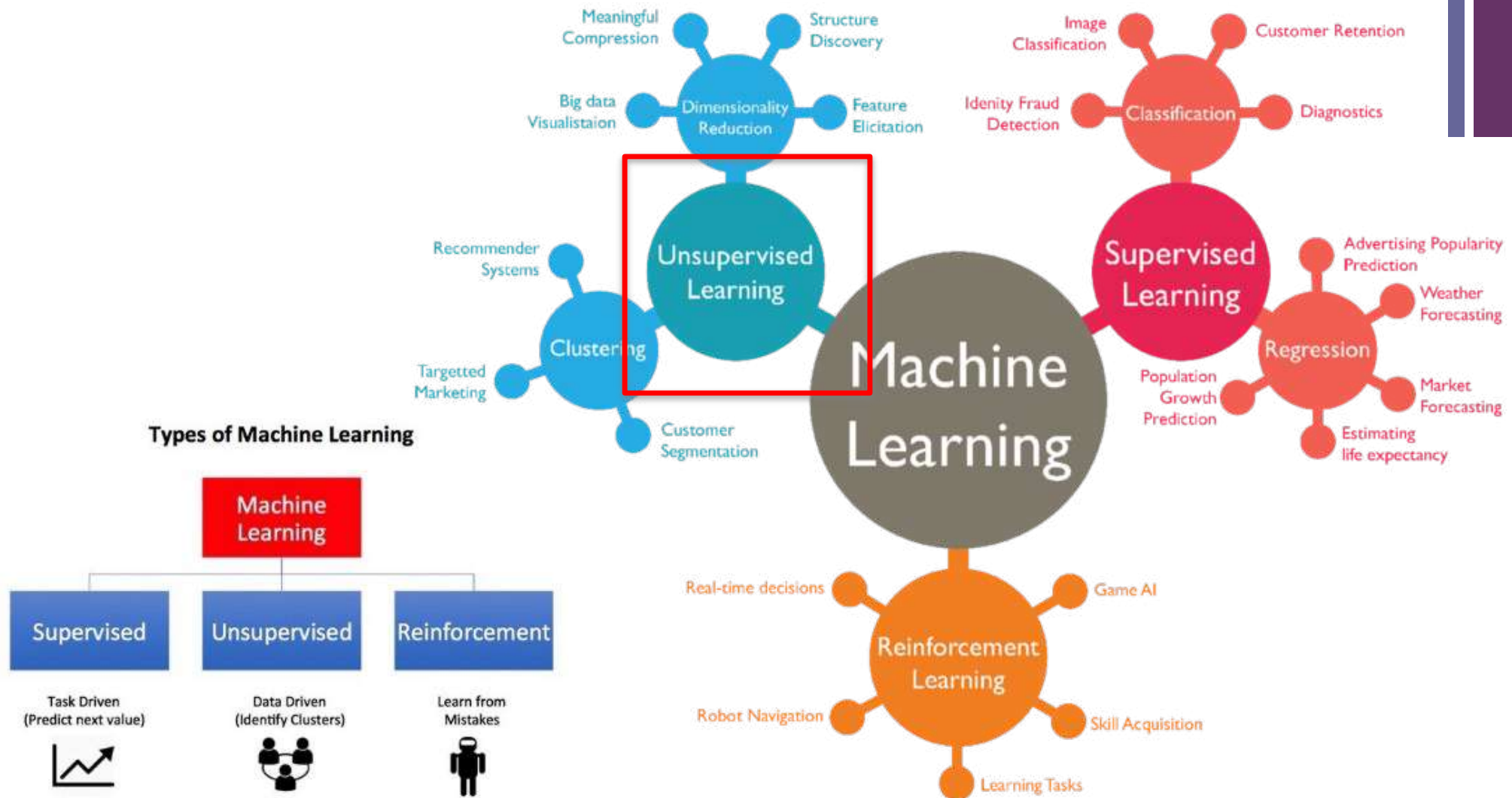
$$R^2 = \frac{SSM}{SST} = 1 - \frac{SSE}{SST}$$

id	chol (x)	bp (y)	predict	error	squred error (SE)	guess	(y - y_bar)	squred total (ST)
1	437	194	196.1897	(2.1897)	4.7948	143.4286	50.5714	2,557.4694
2	264	121	141.4179	(20.4179)	416.8906	143.4286	(22.4286)	503.0408
3	249	131	136.6689	(5.6689)	32.1364	143.4286	(12.4286)	154.4694
4	297	159	151.8657	7.1343	50.8982	143.4286	15.5714	242.4694
5	243	123	134.7693	(11.7693)	138.5164	143.4286	(20.4286)	417.3265
6	272	161	143.9507	17.0493	290.6786	143.4286	17.5714	308.7551
7	161	115	108.8081	6.1919	38.3396	143.4286	(28.4286)	808.1837
average	274.7143	143.4286		SSE	972.2548		SST	4,991.7143
				MSE	138.8935			
				RMSE	11.7853			
	R^2	1 - (SSE/SST)	0.8052					

+ Part3: Unsupervised Learning (Descriptive Task)

+ Machine Learning (cont.)

147





Task2: Unsupervised learning (descriptive task)

148

Training Data



inputs				target
Age	Income	Gender	Province	Purchase
25	25,000	Female	Bangkok	Yes
35	50,000	Female	Nontaburi	Yes
32	35,000	Male	Bangkok	?



Testing Data



Age	Income	Gender	Province	Purchase
25	25,000	Female	Bangkok	?



+ Clustering

149



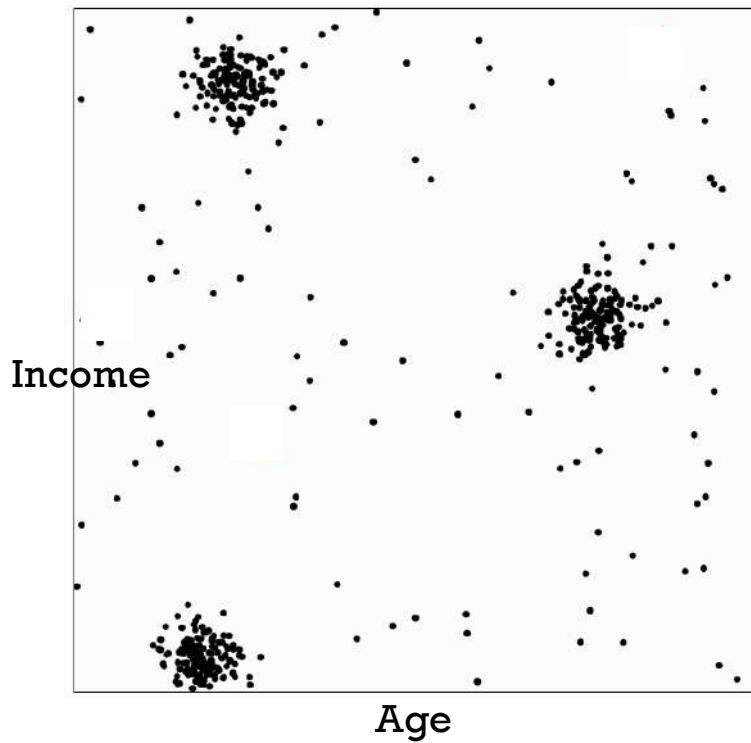
- In our class, there are many participants. Should we teach them using the same method?
- **May be not!** Since they may have different learning behaviors and backgrounds.
- Inputs
 - Education field
 - Level of English communication
 - Level of computer skills
 - Age range
 - Gender



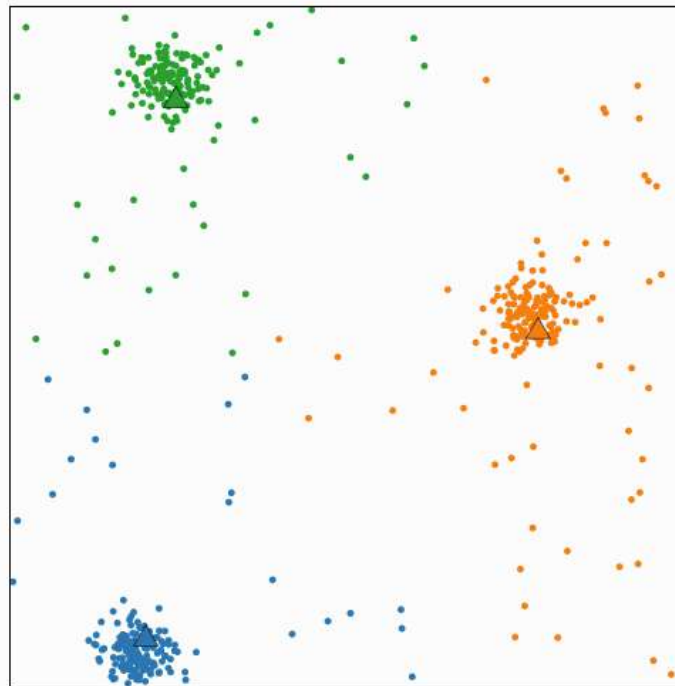
K-means Clustering

150

■ <http://web.stanford.edu/class/ee103/visualizations/kmeans/kmeans.html>



Visualizing K-Means Clustering



The k -means algorithm is an iterative method for clustering a set of N points (vectors) into k groups or clusters of points.

Algorithm

Repeat until convergence:

Find closest centroid

Find the closest centroid to each point, and group points that share the same closest centroid.

Update centroid

Update each centroid to be the mean of the points in its group.

Update centroid

Data

Clustered points ☐ Random

Number of clusters : 3

Number of centroids : 3

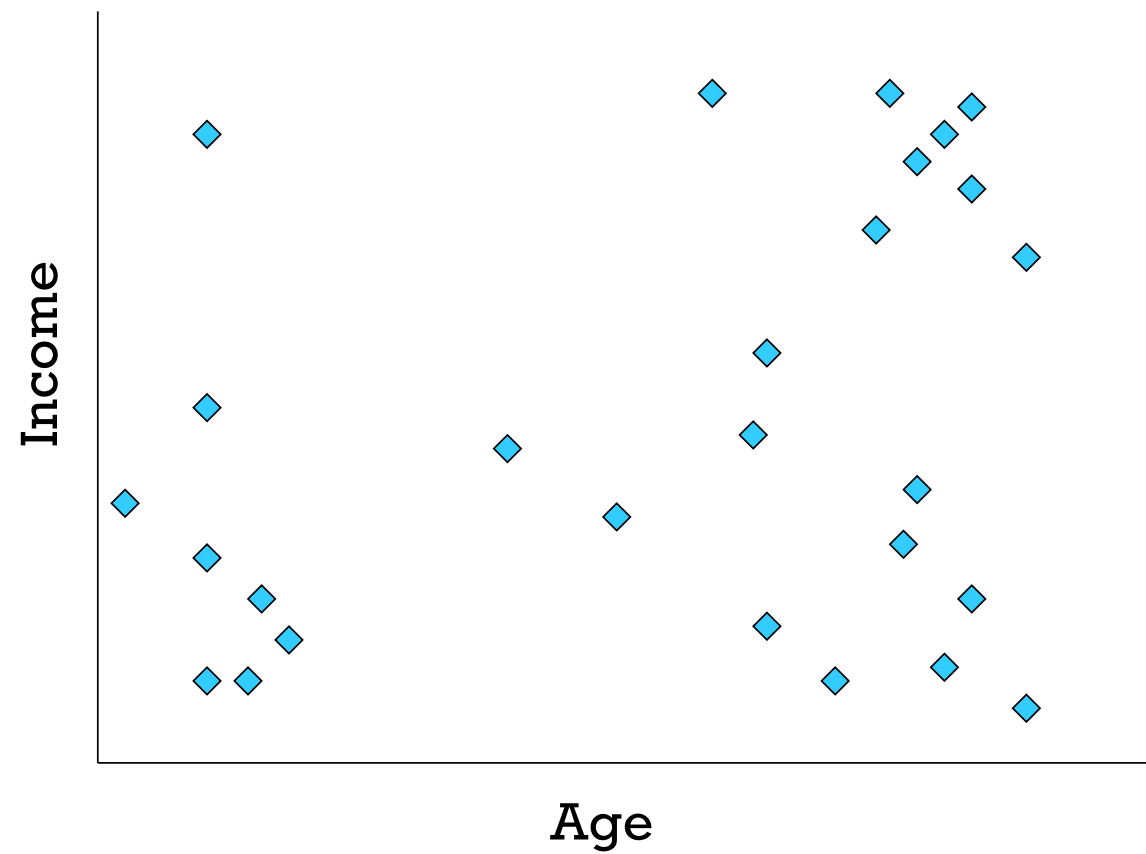
New points

New centroids



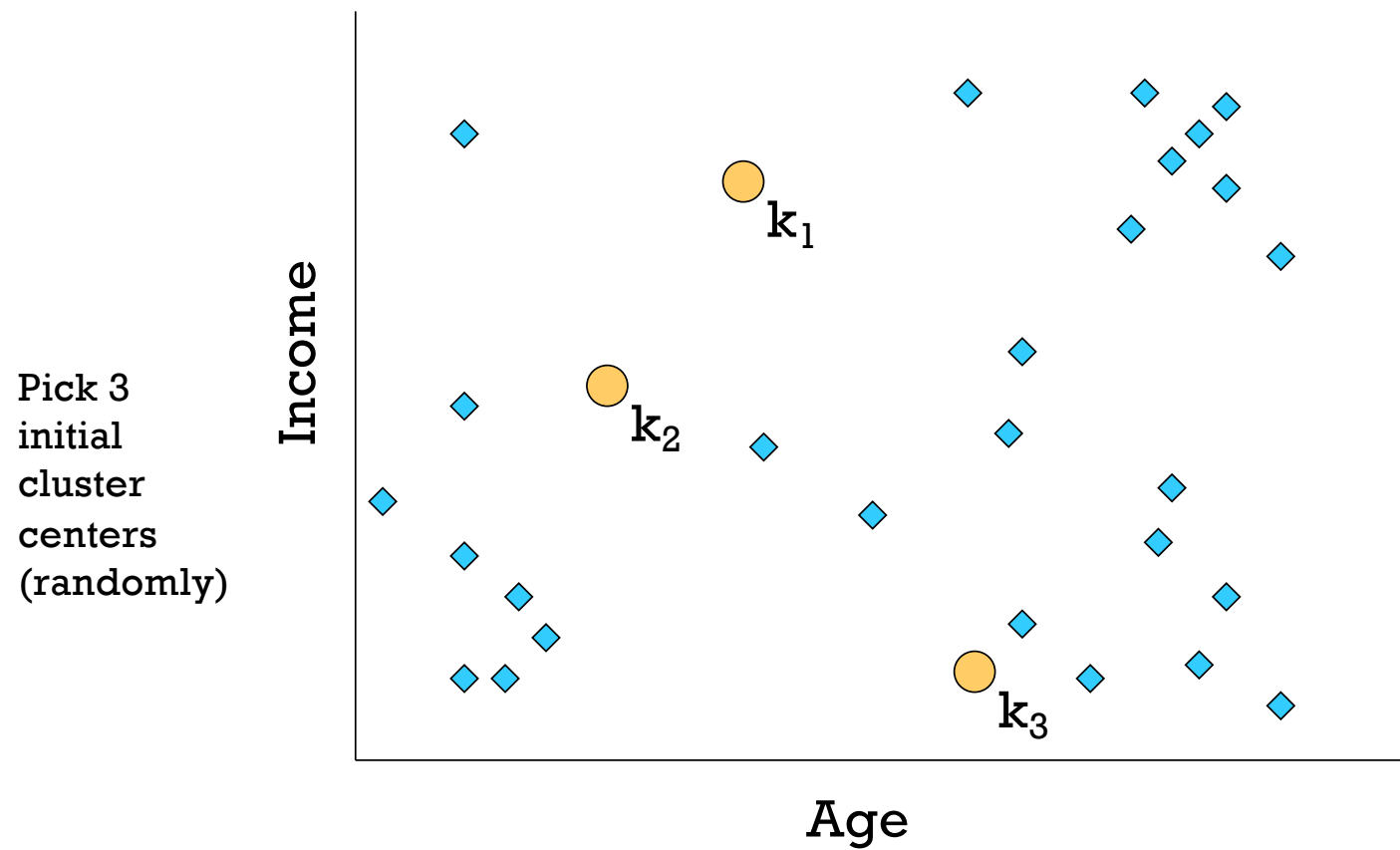
K-means: Step0

151





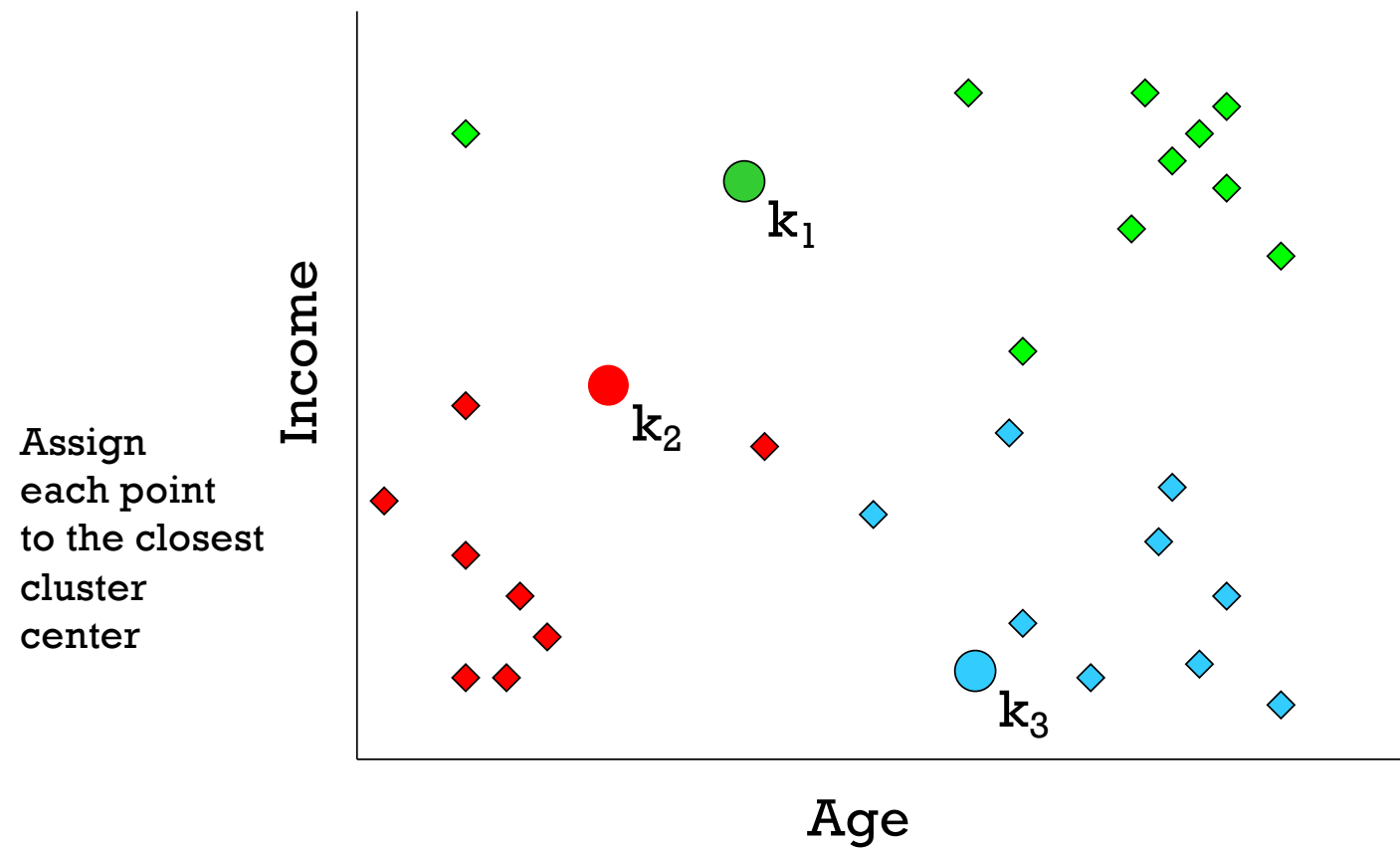
K-means: Step 1





K-means: Step2

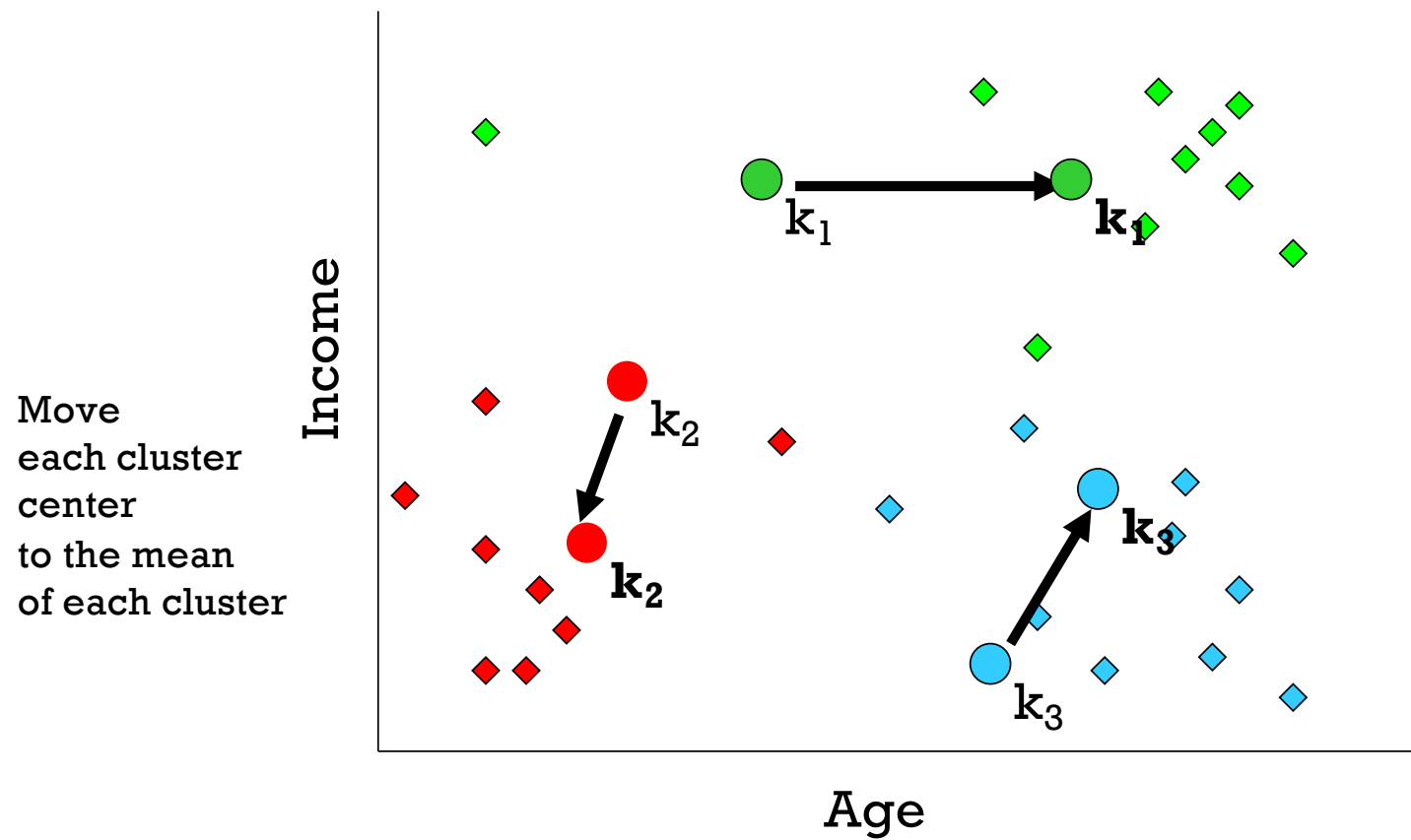
153





K-means: Step3

154

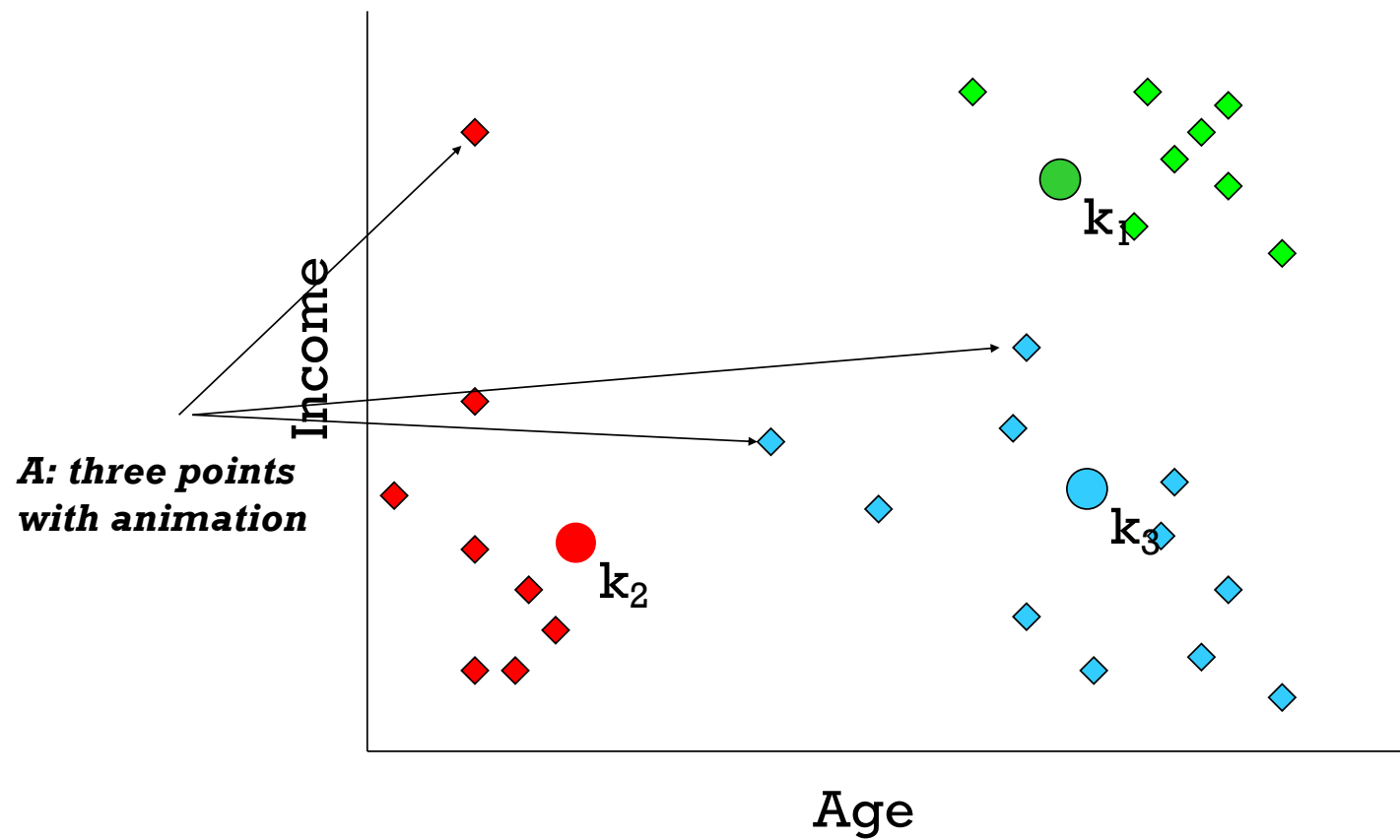


Q: Which points are reassigned?





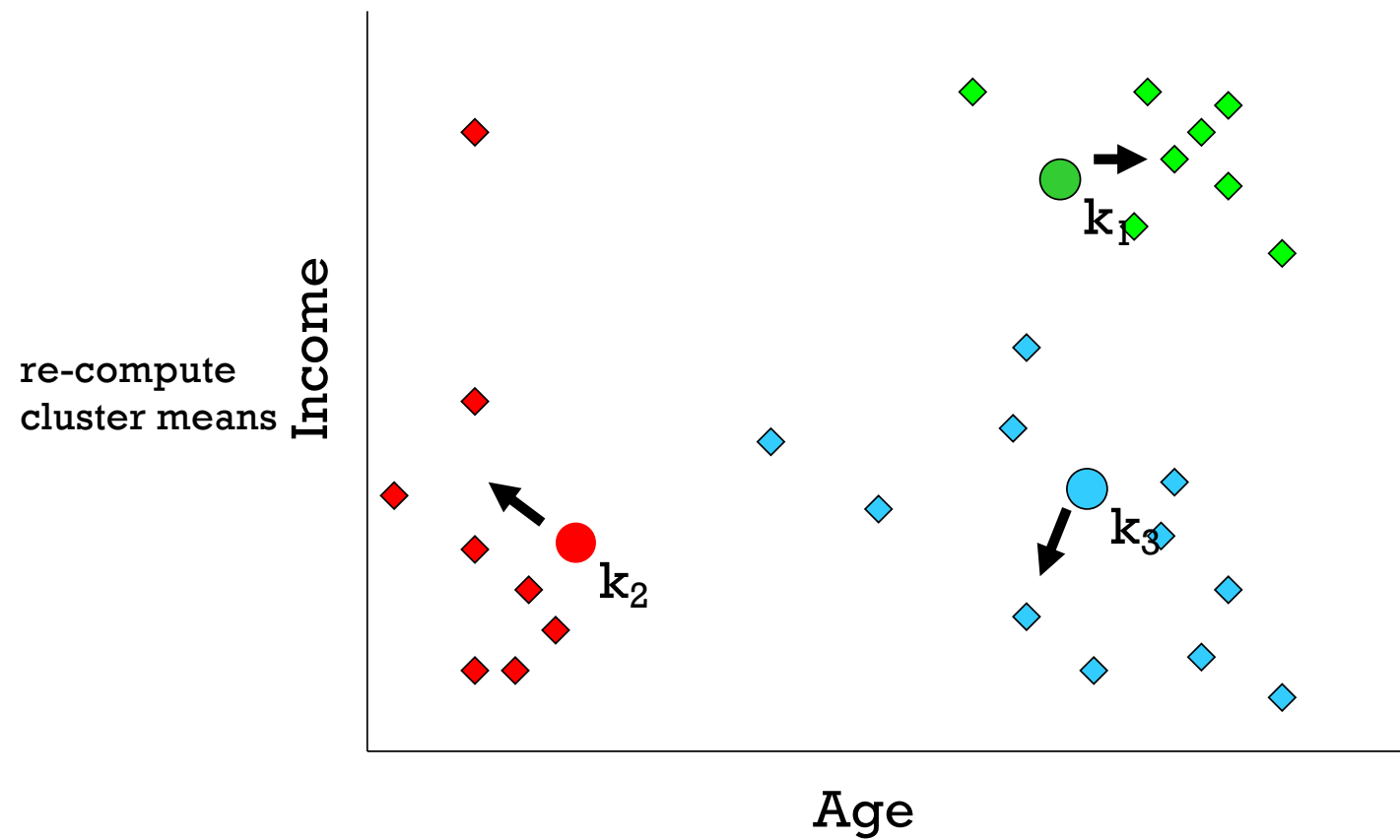
K-means: Step4(a)





K-means: Step5

157





K-means: Step5(a)

Cautions:

- Support only numerical variables
- Need to adjust variable range

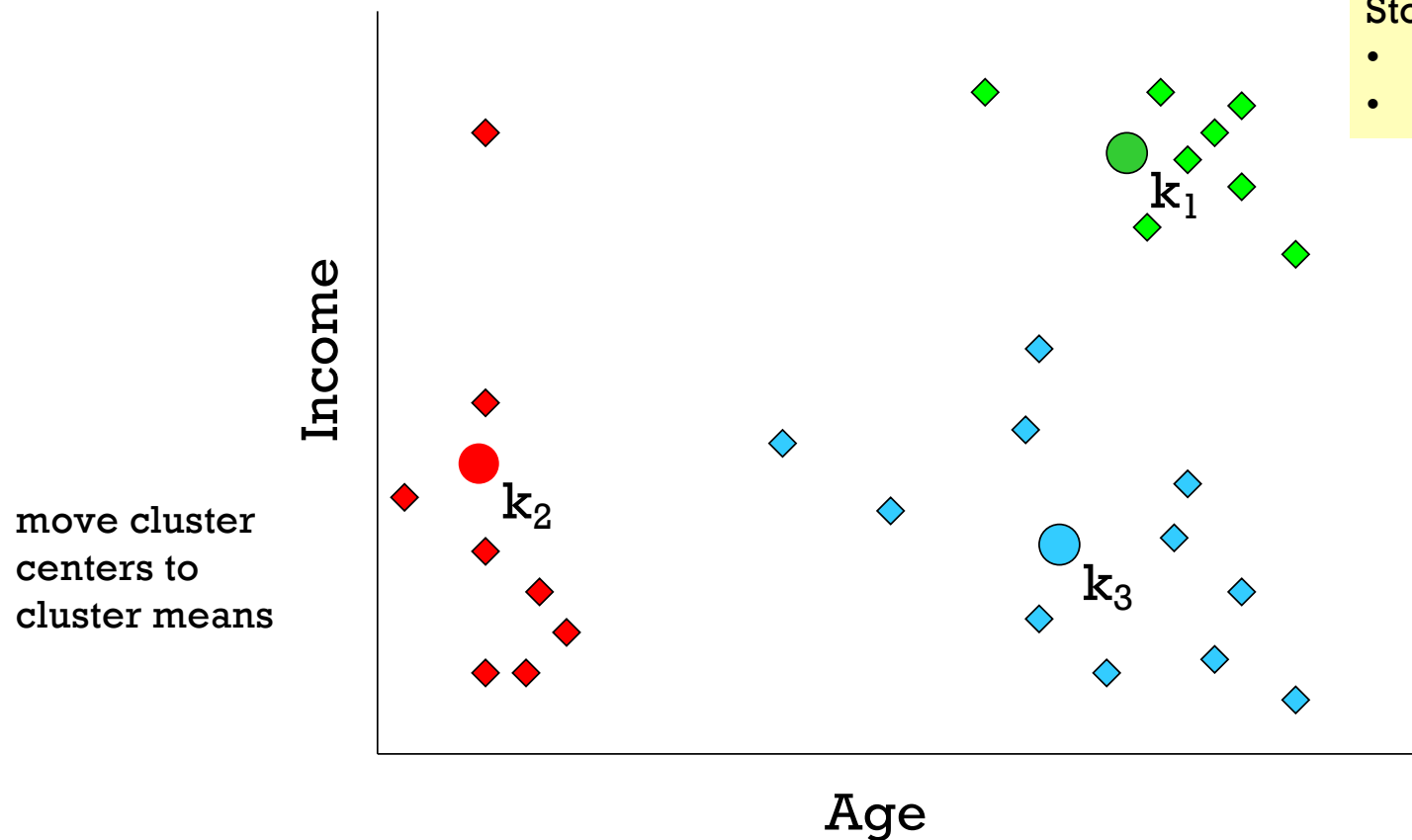
Important Params:

- k, Distance function
- Maximum epochs

158

Stop

- Converge (no change)
- Maximum epochs



[Prev](#) [Up](#) [Next](#)

scikit-learn 0.22.2

[Other versions](#)Please [cite us](#) if you use the software.[sklearn.preprocessing.StandardScaler](#)

Examples using

[sklearn.preprocessing.StandardScaler](#)

sklearn.preprocessing.StandardScaler

```
class sklearn.preprocessing.StandardScaler(copy=True, with_mean=True, with_std=True)
```

[\[source\]](#)

Standardize features by removing the mean and scaling to unit variance

The standard score of a sample x is calculated as:

$$z = (x - u) / s$$

where u is the mean of the training samples or zero if `with_mean=False`, and s is the standard deviation of the training samples or one if `with_std=False`.

Centering and scaling happen independently on each feature by computing the relevant statistics on the samples in the training set. Mean and standard deviation are then stored to be used on later data using [transform](#).

Standardization of a dataset is a common requirement for many machine learning estimators: they might behave badly if the individual features do not more or less look like standard normally distributed data (e.g. Gaussian with 0 mean and unit variance).

For instance many elements used in the objective function of a learning algorithm (such as the RBF kernel of Support Vector Machines or the L1 and L2 regularizers of linear models) assume that all features are centered around 0 and have variance in the same order. If a feature has a variance that is orders of magnitude larger than others, it might dominate the objective function and make the estimator unable to learn from other features correctly as expected.

This scaler can also be applied to sparse CSR or CSC matrices by passing `with_mean=False` to avoid breaking the sparsity structure of the data.

+ Clustering

- K-means

sklearn.cluster.KMeans

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++', n_init=10, max_iter=300, tol=0.0001,
                             precompute_distances='deprecated', verbose=0, random_state=None, copy_x=True, n_jobs='deprecated', algorithm='auto') [source]
```

K-Means clustering.

Read more in the [User Guide](#).

Parameters:

n_clusters : int, default=8

The number of clusters to form as well as the number of centroids to generate.

init : {'k-means++', 'random', ndarray, callable}, default='k-means++'

Method for initialization:

'k-means++': selects initial cluster centers for k-mean clustering in a smart way to speed up convergence. See section Notes in `k_init` for more details.

'random': choose `n_clusters` observations (rows) at random from data for the initial centroids.

If an ndarray is passed, it should be of shape `(n_clusters, n_features)` and gives the initial centers.

If a callable is passed, it should take arguments `X`, `n_clusters` and a random state and return an initialization.

n_init : int, default=10

Number of time the k-means algorithm will be run with different centroid seeds. The final results will be the best output of `n_init` consecutive runs in terms of inertia.

max_iter : int, default=300

Maximum number of iterations of the k-means algorithm for a single run.

```
>>> from sklearn.cluster import KMeans
>>> import numpy as np
>>> X = np.array([[1, 2], [1, 4], [1, 0],
...              [10, 2], [10, 4], [10, 0]])
>>> kmeans = KMeans(n_clusters=2, random_state=0).fit(X)
>>> kmeans.labels_
array([1, 1, 1, 0, 0, 0], dtype=int32)
>>> kmeans.predict([[0, 0], [12, 3]])
array([1, 0], dtype=int32)
>>> kmeans.cluster_centers_
array([[10.,  2.],
       [ 1.,  2.]])
```

[Prev](#) [Up](#) [Next](#)

scikit-learn 1.1.2

[Other versions](#)

Please [cite us](#) if you use the software.

[sklearn.metrics.silhouette_score](#)
[Examples using](#)
[sklearn.metrics.silhouette_score](#)

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

$$S = \frac{1}{N} \sum_{i=1}^N s(i)$$

Plain text

Sample	Cluster	Silhouette
A	1	0.8
B	1	0.6
C	1	0.7
D	2	0.4
E	2	0.5

Overall:

$$\frac{0.8 + 0.6 + 0.7 + 0.4 + 0.5}{5} = 0.60$$

h2>sklearn.metrics.silhouette_score

```
sklearn.metrics.silhouette_score(X, labels, *, metric='euclidean', sample_size=None, random_state=None, **kwargs)
```

[\[source\]](#)

Compute the mean Silhouette Coefficient of all samples.

The Silhouette Coefficient is calculated using the mean intra-cluster distance (a) and the mean nearest-cluster distance (b) for each sample. The Silhouette Coefficient for a sample is (b - a) / max(a, b). To clarify, b is the distance between a sample and the nearest cluster that the sample is not a part of. Note that Silhouette Coefficient is only defined if number of labels is 2 <= n_labels <= n_samples - 1.

This function returns the mean Silhouette Coefficient over all samples. To obtain the values for each sample, use [silhouette_samples](#).

The best value is 1 and the worst value is -1. Values near 0 indicate overlapping clusters. Negative values generally indicate that a sample has been assigned to the wrong cluster, as a different cluster is more similar.

Read more in the [User Guide](#).

Parameters::

- X** : *array-like of shape (n_samples_a, n_samples_a) if metric == "precomputed" or (n_samples_a, n_features) otherwise*
An array of pairwise distances between samples, or a feature array.
- labels** : *array-like of shape (n_samples,)*
Predicted labels for each sample.
- metric** : *str or callable, default='euclidean'*
The metric to use when calculating distance between instances in a feature array. If metric is a string, it must be one of the options allowed by `metrics.pairwise.pairwise_distances`. If X is the distance array itself, use `metric="precomputed"`.
- sample_size** : *int, default=None*
The size of the sample to use when computing the Silhouette Coefficient on a random subset of the data. If `sample_size` is None, no sampling is used.



Please [cite us](#) if you use the software.

[sklearn.metrics.silhouette_samples](#)

[silhouette_samples](#)

[Examples using](#)

[sklearn.metrics.silhouette_samples](#)

sklearn.metrics.silhouette_samples

```
sklearn.metrics.silhouette_samples(X, labels, *, metric='euclidean', **kwargs)
```

[\[source\]](#)

Compute the Silhouette Coefficient for each sample.

The Silhouette Coefficient is a measure of how well samples are clustered with samples that are similar to themselves. Clustering models with a high Silhouette Coefficient are said to be dense, where samples in the same cluster are similar to each other, and well separated, where samples in different clusters are not very similar to each other.

The Silhouette Coefficient is calculated using the mean intra-cluster distance (a) and the mean nearest-cluster distance (b) for each sample. The Silhouette Coefficient for a sample is $(b - a) / \max(a, b)$. Note that Silhouette Coefficient is only defined if number of labels is $2 \leq n_labels \leq n_samples - 1$.

This function returns the Silhouette Coefficient for each sample.

The best value is 1 and the worst value is -1. Values near 0 indicate overlapping clusters.

Read more in the [User Guide](#).

Parameters: *X : {array-like, sparse matrix} of shape (n_samples_a, n_samples_a) if metric == "precomputed" or (n_samples_a, n_features) otherwise*

An array of pairwise distances between samples, or a feature array. If a sparse matrix is provided, CSR format should be favoured avoiding an additional copy.

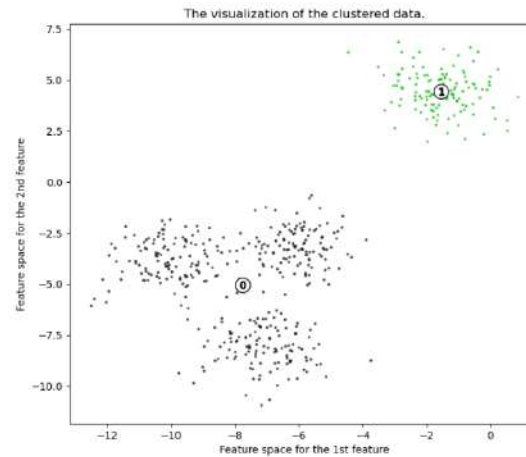
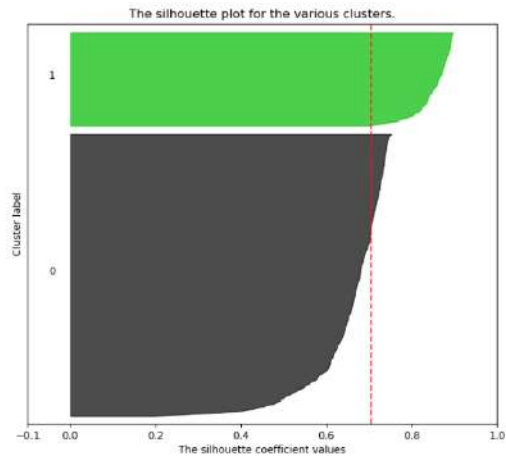
How to choose n_clusters?

Chosen is 2 or 4.

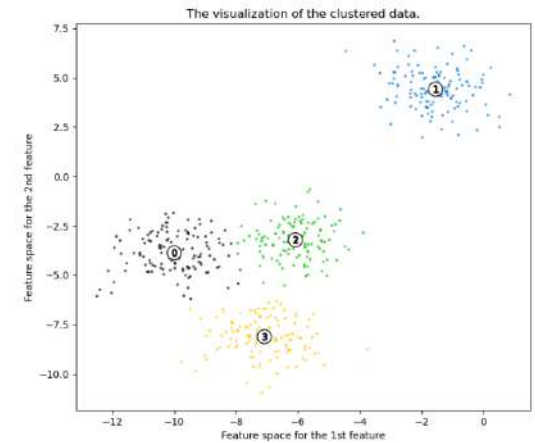
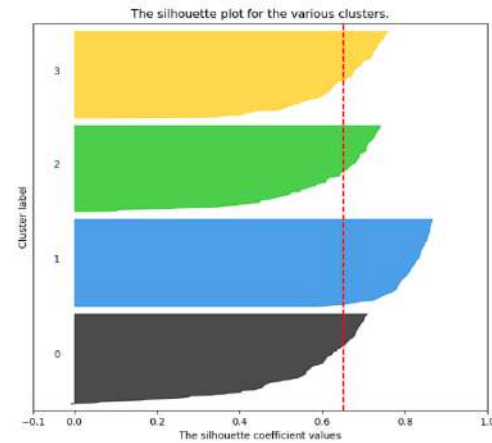
Out:

```
For n_clusters = 2 The average silhouette_score is : 0.7049787496083262
For n_clusters = 3 The average silhouette_score is : 0.5882004012129721
For n_clusters = 4 The average silhouette_score is : 0.6505186632729437
For n_clusters = 5 The average silhouette_score is : 0.561464362648773
For n_clusters = 6 The average silhouette_score is : 0.4857596147013469
```

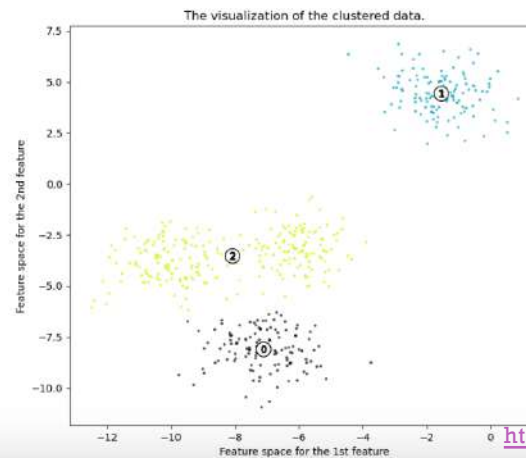
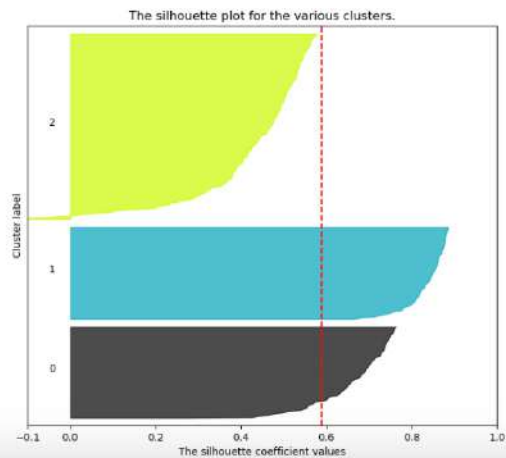
Silhouette analysis for KMeans clustering on sample data with n_clusters = 2



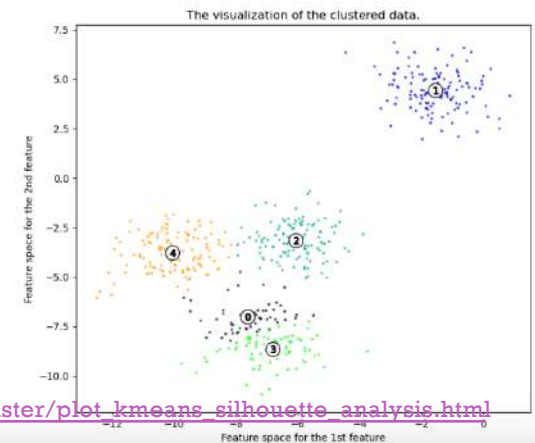
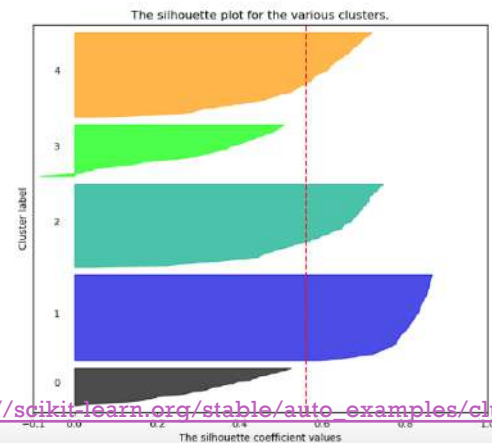
Silhouette analysis for KMeans clustering on sample data with n_clusters = 4



Silhouette analysis for KMeans clustering on sample data with n_clusters = 3



Silhouette analysis for KMeans clustering on sample data with n_clusters = 5



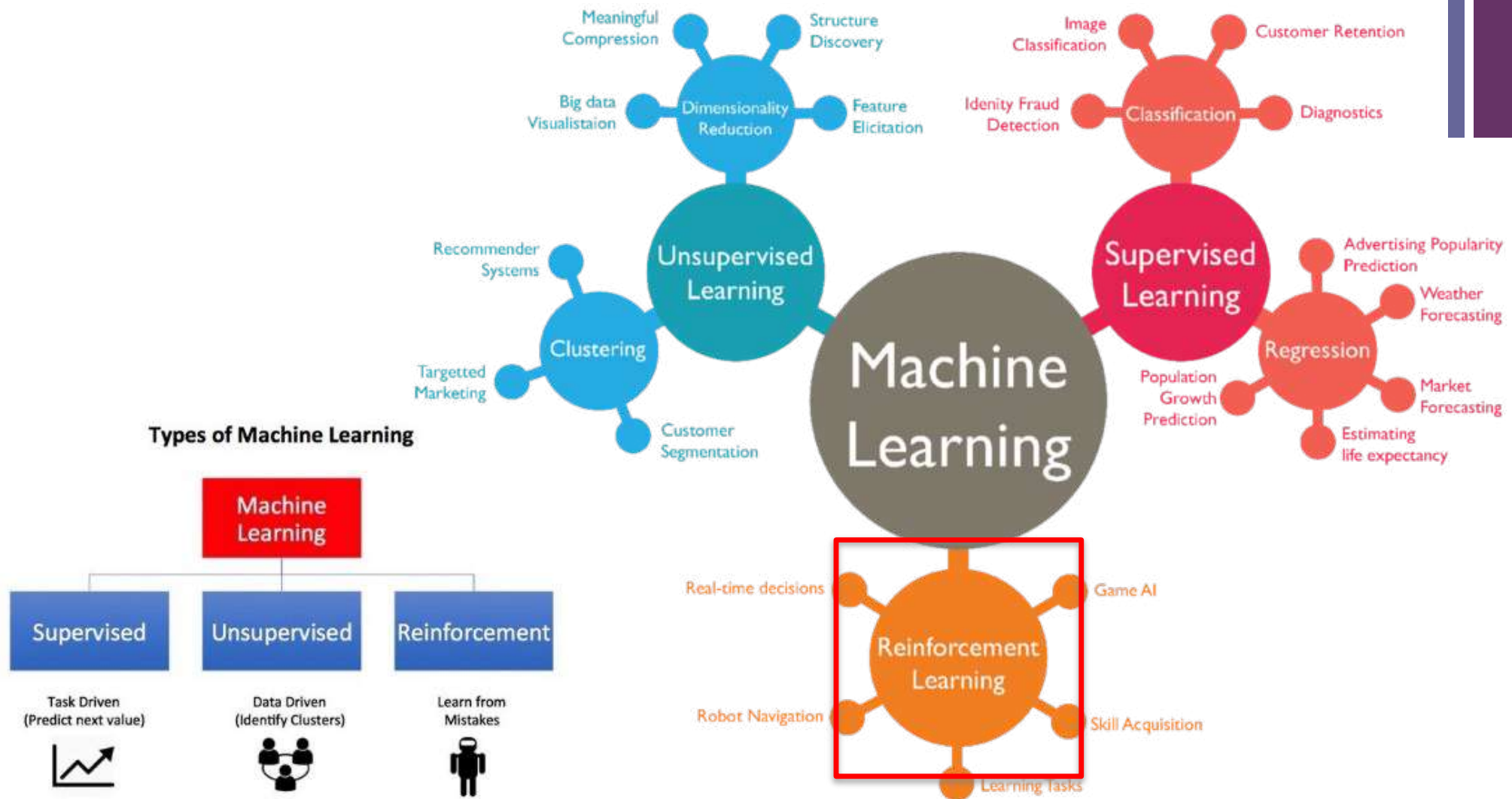
https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html



Part4: Reinforcement Learning (RL)

+ Task3: Reinforcement learning (optimization task)

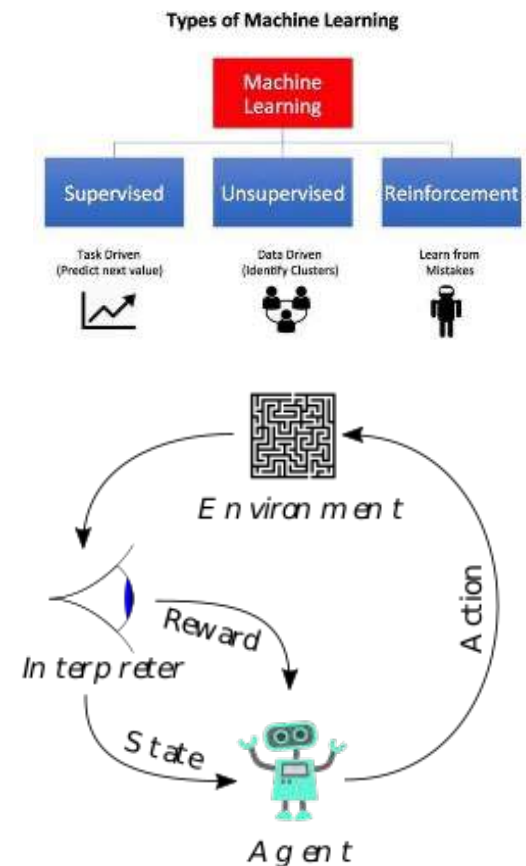
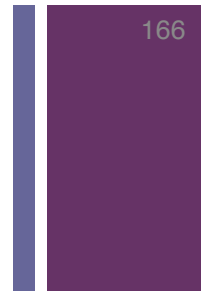
165



+

Task3: Reinforcement Learning (cont.)

<https://www.youtube.com/watch?v=fiQsmdwEGT8>





<https://www.zdnet.com/article/deepmind-alphago-zero-learns-on-its-own-without-meatbag-intervention/>

Reinforcement Learning (cont.)

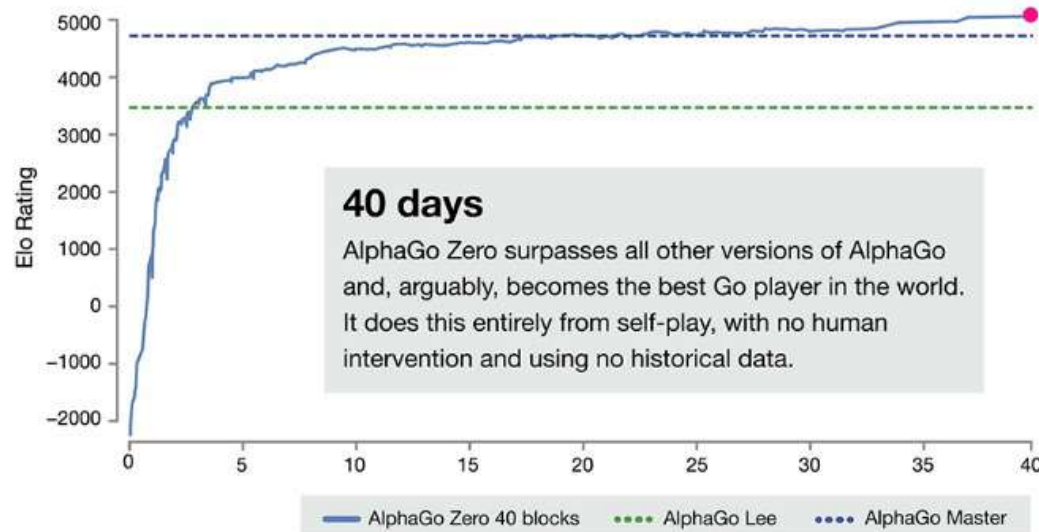
DeepMind AlphaGo Zero learns on its own without meatbag intervention

The latest iteration of DeepMind's Go-playing AI has taught itself and bested other versions of AlphaGo.



By Chris Duckett | October 19, 2017 -- 00:44 GMT (08:44 GMT+08:00) | Topic: Innovation

Is AlphaGo Zero a scientific breakthrough – step towards AGI?

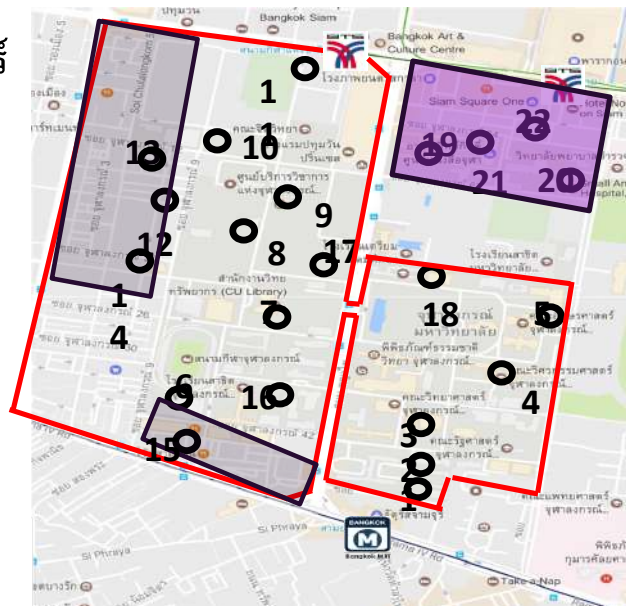


การออกแบบระบบ Relocation สำหรับ CU TOYOTA Ha:mo

พื้นที่การให้บริการ 22 สถานี 60 ช่องจอด

ครอบคลุมบริเวณมหาวิทยาลัย ศูนย์การค้า และ สยามสแควร์

- | | |
|------------------------------|-----------------------|
| 1) ทางออกไปจามจุรีสแควร์ | 12) ระเบียบจามจุรี |
| 2) เศรษฐศาสตร์ | 13) สวนหลวงสแควร์ |
| 3) ศาลาพระเกษียณ | 14) แอมพาร์ค |
| 4) วิศวกรรมศาสตร์ | 15) ยู เซ็นเตอร์ |
| 5) อักษรศาสตร์ | 16) นิเทศศาสตร์ |
| 6) จามจุรี 9 | 17) สำนักงานทรัพย์สิน |
| 7) จามจุรี 5 | 18) อาคารศิลปวัฒนธรรม |
| 8) หอพักวิทยนิเวศน์ | 19) เกษศาสตร์ |
| 9) จามจุรี 10 | 20) สัตวแพทยศาสตร์ |
| 10) จุฬาพัฒน์ 14 | 21) อาคารวิทยกิตติ |
| 11) บีทีเอส สยามกีฬาแห่งชาติ | 22) สยามสแควร์ ซอย 8 |

เปิดให้บริการ

จันทร์ - ศุกร์ 7.00-19.00

อัตราค่าบริการ

การใช้บริการ

30 บาท / 20 นาทีแรก เกินจากนั้นนาทีละ 2 บาท

ค่าลงทะเบียนสมาชิก

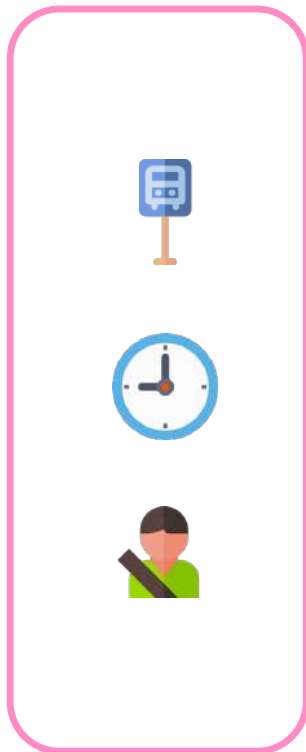
100 บาท (คืนเป็นคะแนนให้ 100 pt.)

วิธีชำระค่าบริการ

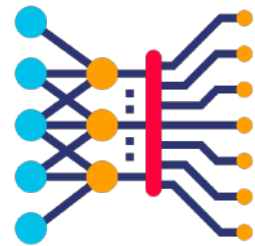
ผ่านระบบอิเล็กทรอนิกส์ ด้วยบัตรเครดิต/เดบิต



Machine Learning



Demand
Forecasting



Deep Learning

Relocation
Optimization



Min Cost Max-Flow

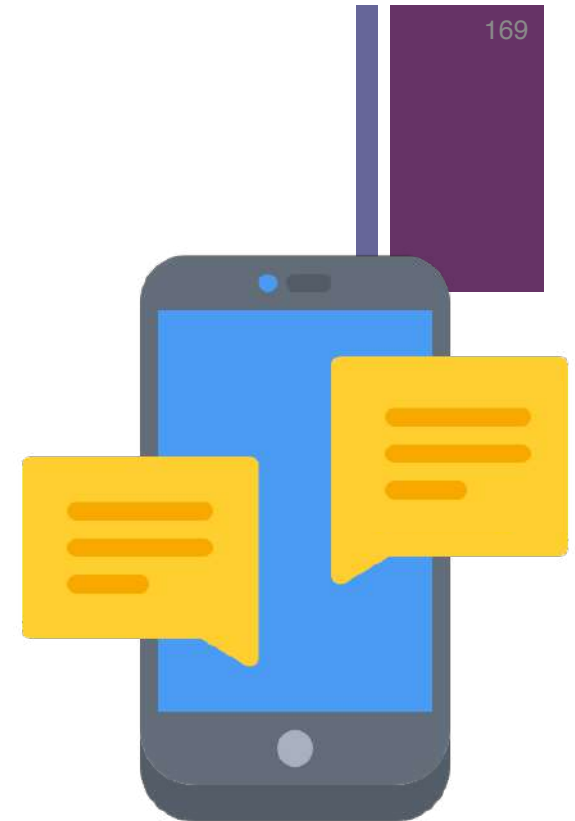
Research Article

Recurrent Neural-Based Vehicle Demand Forecasting and Relocation Optimization for Car-Sharing System: A Real Use Case in Thailand

Peerapon Vateekul¹, Panyawut Sri-iesaranusorn^{1,2}, Pawit Aiemvaravutikul¹,
Adsadawut Chanakitkarnchok¹ and Kultida Rojviboonthai¹

¹Chulalongkorn University Big Data Analytics and IoT Center (CUBIC), Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University, Bangkok, Thailand

²Division of Information Science, Nara Institute of Science and Technology, Nara, Japan



Relocation
Optimization
System

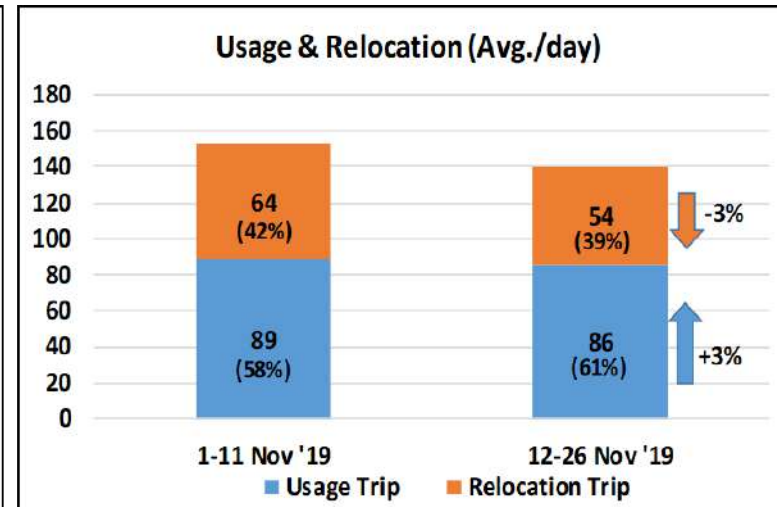
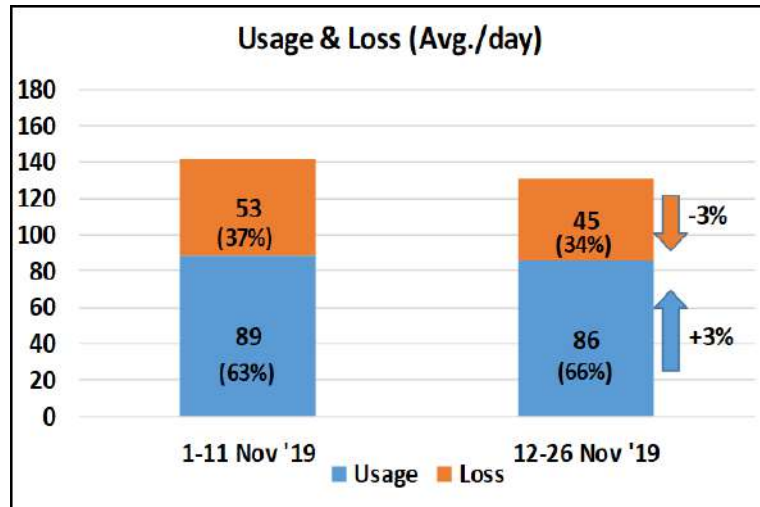
Project 3 : System Design for Vehicle Relocation

4) Service Result :

- Opportunity Loss = 34% (-3%)
- Relocation job reduction = 39% (-3%)
- Reduction of manual job (loss analysis & demand forecasting) = 17 MH/Week

5) Conclusion :

- Our AI assisted system can achieve lower opportunity loss (3%), lower relocation job (3%), and reduce human relocation effort (17 MH/Week).



+ Thank you
& any questions